

Università di Pisa

Dipartimento di Informatica

Refactoring Sostenibile del Codice mediante Large Language Model

Tesi di Laurea in Informatica

Candidato:

Gabriele Marsili

Matricola:

Relatori:

Prof. Jacopo Soldani

Prof. Stefano Forti

Dr.ssa Lucia Passaro

Anno Accademico 2024/2025

Abstract

Questa tesi indaga il potenziale dei Large Language Model (LLM) nel migliorare caratteristiche dei codici al fine di ridurre il consumo energetico, ossia indaga le capacità degli LLM nel refactoring sostenibile del codice, valutando se i modelli allo stato dell'arte (OpenAI GPT-4, Google Gemini Flash 2.0 e Anthropic Claude Sonnet 4) possano migliorare le prestazioni del codice e ridurre il consumo energetico mantenendo al contempo la correttezza funzionale. Attraverso una metodologia sperimentale applicata a un dataset di oltre 2.300 snippet di codice in 7 diversi linguaggi di programmazione, questa ricerca quantifica l'impatto del refactoring basato su LLM riguardo alla riduzione dell'utilizzo di risorse computazionali e consumo energetico. Lo studio impiega quattro diverse tecniche di prompt engineering per valutare l'influenza della progettazione del prompt sull'efficacia del refactoring. I risultati dimostrano che gli LLM possono ottenere significativi miglioramenti delle prestazioni e riduzioni energetiche nei linguaggi di programmazione considerati, con un'efficacia che varia in base al modello, alla versione del prompt e alle caratteristiche del linguaggio. I risultati contribuiscono al campo emergente dell'Ingegneria del Software sostenibile e forniscono indicazioni per integrare il refactoring assistito dall'Intelligenza Artificiale nei flussi di lavoro di sviluppo.

Indice

Abstract	1
1 Introduzione	4
1.1 Contesto	4
1.2 Il Problema e l'Opportunità: Performance del Codice e Sostenibilità Computazionale	4
1.3 Domanda di Ricerca e Obiettivi Scientifici	5
1.4 Contributi della Tesi	6
1.5 Struttura del Documento	7
2 Stato dell'Arte	8
2.1 Large Language Models per la Generazione e Analisi del Codice (Code-LLM)	8
2.2 Tecniche di Refactoring Automatico del Codice	9
2.3 Green Computing e Misurazione della Sostenibilità del Software	11
2.4 Posizionamento della Tesi e Lacune della Letteratura	11
3 Progettazione e Metodologia Sperimentale	13
3.1 Costruzione e Clustering del Dataset	13
3.2 Modelli Selezionati	17
3.3 Progettazione, Ingegnerizzazione e Versioning dei Prompt	18
3.4 Infrastruttura di Esecuzione e Raccolta Metriche	21
3.5 Processo di Valutazione, Validazione e Metriche Derivate	23
4 Risultati Sperimentali	27
4.1 Analisi di Correttezza: Misurazione del Degrado rispetto alla Baseline	27
4.2 Analisi di Performance e Sostenibilità	31
5 Discussione	50
5.1 Risposte alle Principali Domande di Ricerca	50
5.2 Lezioni Apprese	52
5.3 Minacce di Validità dello Studio	53
5.4 Riepilogo dei Principali Risultati	54
6 Conclusioni	56
6.1 Sintesi dei Contributi	56
6.2 Lavori Futuri	57
Bibliografia	61
A Testi Completi dei Prompt (P1-P4)	62
B Esempi Notevoli di Refactoring	64
C Grafici Supplementari: Comparazione P1 vs P4 per Metrica	69

Capitolo 1

Introduzione

1.1 Contesto

L'ambito dell'Ingegneria del Software ha subito importanti cambiamenti a causa dell'avvento dei Large Language Models (LLM). Nell'ultimo decennio il campo dell'Intelligenza Artificiale ha ottenuto importanti progressi nell'elaborazione del linguaggio naturale, riuscendo a sviluppare modelli generativi capaci di comprendere, analizzare e produrre output testuali molto simili a quelli umani, con elevato grado di fluidità e coerenza.

Modelli come GPT di OpenAI, Gemini di Google e Claude di Anthropic fanno parte dell'attuale stato dell'arte in questo campo, dimostrando capacità che superano di gran lunga la tradizionale generazione di testo. Ciò che li distingue dai predecessori è la loro abilità nel comprendere e manipolare i linguaggi formali, compresi i linguaggi di programmazione, con un livello di competenza simile a quello degli sviluppatori umani e in alcuni casi addirittura superiore. Tali modelli sono stati addestrati su ampi dataset di codice e su vaste codebase, comprendenti diversi paradigmi di programmazione, linguaggi e domini, al fine di svolgere compiti e raggiungere obiettivi che in precedenza erano considerati esclusivi dell'esperienza umana: rilevamento di bug, completamento di codice, generazione di documentazione e, in particolare per questa tesi, *refactoring del codice*.

L'uso e l'integrazione degli LLM nei workflow di sviluppo del software hanno comportato un cambiamento di paradigma nella metodologia con cui il codice viene scritto, revisionato e mantenuto. Ambienti di sviluppo integrati moderni (IDE, Integrated Development Environment) incorporano sempre più frequentemente l'utilizzo di assistenti basati sull'Intelligenza Artificiale in grado di restituire suggerimenti e indicazioni in tempo reale, oltre che capaci di svolgere refactoring automatico del codice.

I progressi e l'evoluzione che questo campo ha visto recentemente portano a sollevare domande fondamentali sul ruolo dell'automazione nell'Ingegneria del Software: con quale impatto gli LLM possono migliorare o sostituire le pratiche di sviluppo software tradizionali? In che misura i codici generati dagli LLM migliorano la qualità del software in termini di prestazioni e di altri requisiti non funzionali? A causa della progressiva scarsità delle risorse ambientali e in relazione alle politiche dell'Unione Europea volte a raggiungere la neutralità climatica entro il 2050 [1], diviene di particolare importanza la seguente domanda: come e quanto gli LLM possono contribuire a ridurre l'impatto ambientale legato all'esecuzione del software?

Questa tesi si colloca esattamente nell'intersezione tra l'Ingegneria del Software guidata dall'uso dell'AI e l'ambito del Green Computing, valutando se il refactoring basato sugli LLM possa essere un valido strumento volto a migliorare sia la qualità del codice che la sua sostenibilità. Il punto di incontro tra questi due ambiti—la trasformazione automatizzata del codice mediante LLM e l'attenzione all'ambiente durante il processo di progettazione software—rappresenta un'interessante opportunità di ricerca, ancora per buona parte inesplorata, che questo lavoro mira ad approfondire.

1.2 Il Problema e l'Opportunità: Performance del Codice e Sostenibilità Computazionale

L'inefficienza software non è puramente una questione accademica, bensì ha concrete conseguenze riguardanti le risorse computazionali e la sostenibilità ambientale. Anche semplici code snippet, se eseguiti su larga scala o ripetutamente in ambienti di produzione, possono causare sostanziali consumi di energia e relative emissioni di carbonio.

La relazione tra efficienza del codice e impatto energetico è basata su vincoli fisici: i processori moderni consumano energia proporzionalmente al carico computazionale e anche le operazioni relative

alla memoria hanno costi energetici significativi. Le metriche di performance di basso livello—uso di CPU, uso di RAM, tempo di esecuzione—possono essere aggregate per derivare il consumo energetico e, di conseguenza, l'impronta ambientale in termini di emissioni di carbonio.

La premessa fondamentale di questo lavoro è che il codice inefficiente non è immutabile. Attraverso il refactoring—il processo di ristrutturazione del codice esistente senza alterarne il comportamento—le performance possono essere significativamente migliorate. Tradizionalmente questo processo viene effettuato manualmente da sviluppatori esperti; tuttavia, con l'avvento degli LLM, si è aperta la possibilità di un refactoring automatico anche su larga scala. Se gli LLM si dimostrano in grado di migliorare uso di CPU, RAM e tempo di esecuzione mantenendo la correttezza funzionale, queste tecnologie rappresenterebbero concrete opportunità per ottenere benefici sostanziali in termini di efficienza energetica. Al contrario, se il codice generato dagli LLM introducesse regressioni rispetto alle prestazioni e/o alla correttezza funzionale, dovute ad esempio ad allucinazioni, eccessive astrazioni o pattern applicati in modo errato, allora i presunti vantaggi dello sviluppo assistito dall'AI dovrebbero essere riconsiderati criticamente. Questa tesi cerca quindi di quantificare empiricamente tale potenziale, fornendo una risposta basata sui dati alla domanda: gli LLM possono contribuire a un software più sostenibile attraverso il refactoring automatizzato?

1.3 Domanda di Ricerca e Obiettivi Scientifici

In merito all'impatto degli LLM su campi quali l'Ingegneria del Software e il Green Computing, ancor prima dell'avvio del lavoro di tesi, sono emerse le seguenti domande di ricerca, utilizzate poi come punti guida nell'intero studio:

Gli LLM—nello specifico OpenAI GPT-4, Gemini Flash 2.0 e Anthropic Claude Sonnet 4—possono migliorare le performance e la sostenibilità del codice mediante processi di refactoring mantenendone la correttezza? Se sì, con quale impatto e come (ossia attraverso quali tecniche) raggiungono tale obiettivo?

È quindi possibile analizzare nel dettaglio questi quesiti al fine di valutare differenti punti interni al processo di refactoring via LLM:

Se: i Large Language Models migliorano il codice?

Il primo obiettivo è volto a comprendere se gli LLM siano effettivamente in grado di migliorare il codice scritto da persone. Tale quesito ha un esito binario: in media gli LLM generano codice più efficiente rispetto all'originale? Dove l'efficienza attraversa più dimensioni, incluse quelle considerate all'interno di questo studio, ossia uso di CPU, consumo di memoria (uso di RAM) e tempo di esecuzione, oltre che le metriche derivate quali il consumo energetico.

L'analisi di questo primo punto considera e tiene fortemente di conto anche la correttezza funzionale: è essenziale che il codice generato dagli LLM sia in grado di mantenere il funzionamento del codice originario senza introdurre errori, bug o altre problematiche e senza alterare il comportamento. In assenza di questi fondamentali criteri un miglioramento, anche se misurato nelle performance, non può effettivamente essere considerato tale.

Come: meccanismi e pattern di refactoring

Una volta stabilita la portata delle miglorie introdotte dagli LLM, viene naturale chiedersi in che modo e attraverso quali tecniche gli LLM migliorino i codici originari. A tal fine è necessario analizzare differenze tra modelli, linguaggi considerati e versioni del prompt utilizzate, oltre che identificare i pattern di refactoring applicati da ogni modello e, possibilmente, correlarli alle osservazioni relative ai miglioramenti delle performance.

Comprendere come gli LLM agiscano nel processo di refactoring fornisce dati interessanti sia per ulteriori ricerche nel campo dell'Intelligenza Artificiale che nell'ambito dell'Ingegneria del Software.

Quanto: quantificare l'impatto del miglioramento

Assumendo che la risposta al primo punto sia “sì, gli LLM migliorano il codice originario attraverso

refactoring”, diviene interessante quantificare la portata di tale miglioramento. Viene quindi misurata e calcolata la distribuzione di tali miglioramenti percentuali presenti nei codici generati rispetto agli originari. Ciò permette di comprendere se i miglioramenti nelle performance introdotti dagli LLM siano modesti o sostanziali e, di conseguenza, se possano rappresentare una concreta opportunità di risparmio energetico quando portati su larga scala.

Ecco che questa quantificazione diviene un punto critico per stabilire l'utilità reale del refactoring basato sugli LLM: miglioramenti piccoli o marginali potrebbero non giustificare l'impiego di risorse e costi associati all'utilizzo stesso degli LLM, mentre, al contrario, guadagni consistenti e considerevoli potrebbero invece porre basi solide e motivazioni concrete per l'impiego di queste tecnologie anche e soprattutto su larga scala.

1.4 Contributi della Tesi

Il contributo scientifico di questa tesi risiede nell'analisi empirica e sistematica del refactoring basato su LLM, con un focus specifico sulla sostenibilità, una dimensione ampiamente assente nella letteratura esistente sulla generazione di codice assistita dall'Intelligenza Artificiale.

Questa tesi fornisce molteplici contributi all'intersezione degli ambiti dell'Ingegneria del Software e del Green Computing.

Anzitutto contribuisce mediante la **costruzione di un dataset strutturato e multi-linguaggio per analisi di sostenibilità**, ossia svolge il design e l'assemblaggio di un dataset appositamente costruito per valutare la sostenibilità del refactoring del codice. Dataset pre-esistenti danno priorità alla correttezza funzionale e alla diversità algoritmica, non comprendendo quanto necessario a misurare metriche di performance a grana fine. Questo lavoro introduce un dataset con **diverse fonti** in quanto gli snippet di codice provengono prevalentemente dalla piattaforma Exercism¹, ma da diversi utenti, garantendo così implementazioni realistiche con relative imperfezioni e opportunità di miglioramento. Tale dataset inoltre comprende **diversi linguaggi di programmazione**, rappresentando un'ampia gamma di paradigmi (interpretato vs. compilato, tipizzato staticamente vs. dinamicamente, gestione automatica vs. manuale della memoria). All'interno del dataset ogni frammento di codice è abbinato a una **test suite** per convalidare automaticamente la correttezza funzionale dopo il refactoring e le entries sono raggruppate in **clusters** per tipo di esercizio, facilitando l'analisi comparativa tra linguaggi e modelli. La costruzione del dataset è documentata e riproducibile. Oltre al dataset è stata implementata una pipeline automatizzata per l'aggiunta di nuove entries, la generazione dei codici rifattorizzati e la raccolta delle metriche. Tale pipeline è descritta in dettaglio nella Sezione 3.5 ed è a sua volta un contributo in quanto permette di ampliare il dataset utilizzato in questo studio.

Un ulteriore apporto riguarda il **design ed esecuzione di un esperimento controllato**, ossia la metodologia sperimentale stessa, la quale stabilisce un protocollo rigoroso per la valutazione del refactoring basato su LLM. Per ogni codice di base sono state generate varianti rifattorizzate da più modelli LLM utilizzando diverse versioni del prompt, permettendo di isolare gli effetti del refactoring per modello e per formulazione del prompt. Tutte le esecuzioni sono state effettuate in container Docker isolati, con ambienti specifici per linguaggio per garantire consistenza. Ogni snippet di codice è stato eseguito più volte, considerando la media delle metriche raccolte al fine di ottenere robustezza statistica. Per ogni esecuzione vengono raccolte metriche grezze (CPU, RAM, tempo di esecuzione, pass rate) e calcolato il consumo energetico. L'infrastruttura è completamente automatizzata, garantendo che l'esperimento possa essere replicato ed esteso.

Un altro contributo è dato dalle **analisi quantitative su correttezza, performance e impatto ambientale**, relativamente ai risultati degli esperimenti, le quali forniscono risposte basate sui dati alle domande di ricerca poste nella Sezione 1.3. A tal proposito sono state effettuate analisi di correttezza valutando i pass rate tra modelli, prompt e linguaggi e identificando pattern di fallimento specifici. Sono state inoltre condotte analisi di performance, quantificando i miglioramenti per ogni metrica e tenendo in considerazione unicamente le esecuzioni in cui la test suite veniva passata senza errori, assicurando quindi che tutte le misure rappresentino la correttezza funzionale del codice. La

¹Exercism è una piattaforma open source gratuita che offre esercizi di programmazione in molteplici linguaggi, con soluzioni revisionate dalla community. Sito web: <https://exercism.org>

distribuzione dei miglioramenti rivela che gli LLM possono raggiungere riduzioni energetiche medie anche superiori al 50%, ma presentano tuttavia code di regressioni prestazionali non trascurabili. Le metriche prestazionali sono state inoltre tradotte in consumo energetico per valutare l'impatto sulla sostenibilità del refactoring via LLM. Infine sono stati analizzati gli effetti del prompt engineering tramite comparazioni delle diverse versioni del prompt per comprendere il ruolo della formulazione sulla qualità del refactoring.

Dai risultati emersi in questo studio è possibile comprendere che gli LLM siano effettivamente in grado di svolgere refactoring sostenibile del codice, ottenendo miglioramenti significativi, specialmente per linguaggi fortemente tipati quali TypeScript e Go. Tali risultati sono presentati attraverso una combinazione di analisi statistiche, box plot, scatter plot e heatmap, consentendo sia l'identificazione di trend di alto livello sia un'ispezione dettagliata per linguaggio e per modello. Insieme, questi contributi stabiliscono una base metodologica per la valutazione della sostenibilità del codice generato dagli LLM, fornendo prove empiriche che alimentano il dibattito in corso sul ruolo degli LLM negli ambiti dell'Ingegneria del Software e del Green Computing.

Infine, per giungere ai contributi precedentemente riportati, questa tesi sfrutta un rigoroso **quadro di valutazione comparativa e relativa metodologia**. Il benchmark dataset è costituito da una collezione curata di snippet di codice reali ottenuti da piattaforme di esercizi (quali Exercism), comprendenti sette diversi linguaggi di programmazione (Python, Java, JavaScript, TypeScript, C, C++, Go), ciascuno con la relativa test suite per accertarne la correttezza funzionale. Vengono considerati tre modelli appartenenti all'attuale stato dell'arte (OpenAI GPT-4, Google Gemini Flash 2.0, Anthropic Claude Sonnet), valutandoli e comparandoli per comprendere se vi siano performance specifiche associate all'uso di un particolare modello. Per il processo di ingegnerizzazione del prompt vengono impiegate e comparate tra loro diverse versioni del prompt, il cui design ambisce a migliorare la qualità del refactoring. Vengono infine raccolte metriche associate alle performance (uso di CPU, uso di RAM, tempo di esecuzione, pass rate relativo alle test suite) sia per i codici originari che per quelli generati dagli LLM, e viene calcolato il consumo energetico come metrica derivata.

1.5 Struttura del Documento

La parte restante di questa tesi è organizzata come segue:

- Il **Capitolo 2** introduce lo stato dell'arte, esaminando la letteratura su LLM per la generazione di codice, tecniche di refactoring tradizionali e basate sull'Intelligenza Artificiale, e Green Computing. Il capitolo si conclude posizionando questa tesi nel panorama della ricerca ed evidenziando le lacune che motivano il lavoro.
- Il **Capitolo 3** presenta la progettazione e la metodologia sperimentale, descrivendo la costruzione del dataset, la scelta dei modelli LLM, il processo di ingegnerizzazione del prompt, l'ambiente di esecuzione containerizzato e le metriche di valutazione.
- Il **Capitolo 4** riporta i risultati sperimentali, analizzando la correttezza del codice generato e le performance in termini di uso di CPU, RAM, tempo di esecuzione e consumo energetico. Le analisi sono disaggregate per modello, versione del prompt e linguaggio di programmazione.
- Il **Capitolo 5** discute i risultati ottenuti, fornendo risposte alle domande di ricerca ed esplorando le implicazioni per l'Ingegneria del Software e il Green Computing. Vengono inoltre esaminate le minacce alla validità e i limiti dello studio.
- Il **Capitolo 6** conclude la tesi riassumendo i contributi principali e delineando le direzioni per la ricerca futura.

Le appendici forniscono materiali supplementari: il testo completo di tutte e quattro le versioni dei prompt (Appendice A), esempi significativi di coppie di codice di base rispetto a quelle refactored (Appendice B) e dati statistici e visualizzazioni aggiuntivi (Appendice C).

Capitolo 2

Stato dell'Arte

Questo capitolo racchiude concetti fondamentali presenti in letteratura e pertinenti agli argomenti trattati in questo studio. Inizia andando ad introdurre gli LLM, con particolare attenzione sui relativi campi di applicazione e abilità nella generazione e analisi del codice. Successivamente viene posta enfasi sul panorama del processo di refactoring automatico del codice, comparando le tecniche tradizionali con quelle moderne basate sull'AI. In seguito questo capitolo indaga i principi del Green Computing, concentrandosi sulle metriche utilizzate per valutare la sostenibilità di un software. Infine viene motivata la posizione che questa tesi trova all'interno dello stato attuale della ricerca, identificando lo spazio nella letteratura corrente che questa tesi è volta a riempire.

2.1 Large Language Models per la Generazione e Analisi del Codice (Code-LLM)

Negli ultimi anni i Large Language Models (LLM) sono emersi nel campo delle tecnologie associate all'AI, guadagnando progressivamente importanza e attenzione grazie alle incredibili capacità e abilità che posseggono. In questa sezione viene fornita una panoramica riguardante gli LLM, concentrandosi sul relativo campo applicativo e sugli aspetti incentrati sull'Ingegneria del Software che queste invenzioni hanno contribuito a modificare.

Definizione e meccaniche fondamentali

Un Large Language Model è un sofisticato modello probabilistico allenato su vaste quantità di dati testuali al fine di comprendere, generare e processare il linguaggio umano. Attualmente la maggior parte degli LLM, come quelli presenti nel gruppo GPT (Generative Pre-trained Transformer) [2], sono basati sull'architettura Transformer, introdotta da Vaswani et al. nel 2017 [3]. Tale architettura ha introdotto un'innovazione cruciale, insita nel meccanismo di self-attention, il quale permette al modello di calcolare peso e rilevanza di ogni parola (o "token") rispetto alle altre in una data sequenza, indipendentemente dalla loro distanza posizionale. Ciò ha consentito di ottenere dipendenze a lungo raggio e relazioni di contesto complesse più efficacemente rispetto alle reti neurali ricorrenti (RNN).

Il ciclo di vita di un LLM comprende due principali fasi:

1. **Pre-training:** il modello viene addestrato in modo auto-supervisionato su un ampio dataset non strutturato con l'obiettivo di minimizzare la funzione di perdita nella previsione del successivo token w_i data la sequenza $w_1, \dots, w_{i-1}$. Attraverso questa fase il modello apprende sintassi, semantica, conoscenza del mondo e capacità di ragionamento.
2. **Fine-Tuning e Allineamento:** Il modello viene adattato a compiti specifici, sfruttando ad esempio Reinforcement Learning from Human Feedback (RLHF) o supervised fine-tuning su dataset associati a task specifici.

L'importanza dell'ingegnerizzazione del prompt

Gli LLM non sono programmi deterministici, bensì sistemi generativi stocastici con capacità di in-context learning, ossia l'abilità di imparare direttamente da esempi o istruzioni presenti nel testo ricevuto, piuttosto che avere la necessità di modificare i pesi del modello. Riescono quindi ad adattare il proprio comportamento in relazione al contesto e, di conseguenza, il loro output è interamente condizionato dall'input ricevuto, definito **prompt**. Caratteristiche quali la qualità, la specificità e la struttura semantica del prompt sono essenziali e determinanti per ottenere risposte corrette e pertinenti. Il processo di creazione e sistemazione del prompt, noto come **prompt engineering**, è quindi la pratica di raffinare l'input ricevuto dal modello al fine di ottenere l'output desiderato [4].

Un prompt vago (ad esempio: “Migliora questo codice”) produrrà risultati generici e al di sotto delle aspettative, mentre un prompt ben strutturato e specifico (es. “Effettua il refactoring della seguente funzione Python per migliorarne l’efficienza energetica riducendo i cicli di CPU, senza alterarne la correttezza funzionale verificata dalle test suite allegate”) impone vincoli e direttive che miglioreranno la qualità del risultato.

Questo lavoro di tesi fa particolare attenzione al processo di prompt engineering e ai relativi risultati, come descritto in seguito nella Sezione 3.3.

Capacità e rilevanza attuale

Attualmente gli LLM posseggono capacità e abilità maggiori rispetto alla semplice generazione di testo, manifestando quelle che in letteratura vengono definite “capacità emergenti” (emergent abilities) [5]. Tra queste vi è l’**In-Context Learning**, ossia l’abilità di sfruttare nuove informazioni fornite nel contesto corrente per risolvere un problema senza aggiornare i pesi del modello. A ciò si affianca lo **Zero-shot e Few-shot Learning**, che consente di risolvere task per i quali il modello non è stato esplicitamente addestrato, utilizzando una singola interazione con la descrizione del compito da svolgere (zero-shot) o un numero limitato di interazioni con esempi forniti nel prompt (few-shot) [2]. Infine il **Ragionamento Logico e la Pianificazione** permettono di decomporre problemi complessi in sotto-task più semplici, una capacità essenziale per l’Ingegneria del Software.

Questa versatilità ha catalizzato un enorme interesse di ricerca, posizionando gli LLM come Foundation Models [6] in grado di automatizzare compiti cognitivi complessi e di offrire elevati incrementi di produttività.

Applicazioni nell’Ingegneria del Software

È possibile addestrare e/o specializzare gli LLM su ampi dataset di codici al fine di concentrarne capacità e abilità nell’ambito del coding. Tali modelli (tra cui, ad esempio, CodeGen [7]) hanno dimostrato particolari competenze in ambito di sviluppo software, tra cui: la **Code Generation**, ossia la scrittura di funzioni o classi intere a partire da descrizioni in linguaggio naturale; la **Code Completion**, relativa al suggerimento di righe o blocchi di codice mentre uno sviluppatore digita; la **Bug Detection e Repair**, che consente l’identificazione di errori logici e la proposta di correzioni; la **Code Summarization e Documentation**, per la generazione di descrizioni in linguaggio naturale per frammenti di codice; e infine il **Code Refactoring**, il focus di questa tesi, che comporta la ristrutturazione del codice esistente per migliorarne gli attributi non funzionali (come leggibilità, manutenibilità o prestazioni) preservandone il comportamento esterno.

Questa ricerca sfrutta quest’ultima capacità per investigare miglioramenti non solo nelle prestazioni tradizionali, ma specificamente nella sostenibilità computazionale.

2.2 Tecniche di Refactoring Automatico del Codice

Il refactoring del codice è un procedimento volto a ristrutturare del codice esistente, cambiandone la struttura interna, al fine di migliorarne dei requisiti non funzionali, senza alterarne il comportamento esterno [8]. In questa sezione vengono comparati l’approccio tradizionale di refactoring con quello automatico basato sull’utilizzo dell’Intelligenza Artificiale (nello specifico degli LLM).

Tecniche di refactoring tradizionale

Gli approcci tradizionali prevedono tecniche deterministiche e basate su un insieme di regole, con lo scopo di operare direttamente sulla struttura del codice. La prima tecnica è la **manipolazione degli Abstract Syntax Tree (AST)**, in cui specifici strumenti analizzano il codice sorgente trasformandolo in un AST e applicando particolari trasformazioni (ad esempio estrazione di metodi o cambiamenti di nome alle variabili); gli IDE moderni (come IntelliJ IDEA o Visual Studio) integrano ampiamente questi strumenti. La seconda tecnica si basa su **metodi formali e analisi dei programmi**, sfruttando tecniche di analisi statica per dimostrare che le trasformazioni da applicare al codice preservano la semantica; tale procedimento assicura la correttezza, ma è spesso limitato ad un insieme predefinito di variabili e/o di pattern di refactoring.

Vantaggi: Il primo e principale punto di forza del refactoring tradizionale è la sua affidabilità e correttezza. Poiché queste tecniche sono basate su regole formali e modifiche strutturali dirette, esse spesso garantiscono che il codice risultante dal processo di refactoring rimanga semanticamente lo stesso rispetto all'originale, preservandone quindi le funzionalità.

Svantaggi: la limitazione maggiore per questo approccio è l'incapacità di comprendere a fondo la semantica del codice e la mancanza di flessibilità. Queste tecniche sono spesso limitate a piccole porzioni di codice, faticando ad essere applicate ad ampie codebase in cui è spesso necessario comprendere a fondo l'intento dello sviluppatore per effettuare cambiamenti con un'elevata portata d'impatto.

Tecniche di refactoring basate sull'AI

Con la comparsa degli LLM (come discusso nella Sezione 2.1) sono emerse nuove opportunità per il refactoring del codice: tecniche basate sull'AI che sfruttano le capacità degli LLM per il refactoring e per cui divengono particolarmente importanti le indicazioni fornite in linguaggio umano.

Anziché seguire un insieme di regole fisso e statico, gli LLM sfruttano le conoscenze acquisite durante la fase di addestramento per portare a termine i compiti richiesti nel prompt. Attraverso la comprensione dei comandi espressi tramite linguaggio umano i modelli capiscono quali siano gli obiettivi e le richieste, andando a generare o modificare il codice per soddisfarle.

Confronto e Trade-Off

I trade-off tra l'approccio tradizionale e quello basato sull'AI sono molteplici e significativi, come riassunto nella Tabella 2.1.

Tabella 2.1: Confronto tra tecniche di refactoring tradizionali e basate sull'AI.

Aspetto	Tecniche Tradizionali	Tecniche Basate su AI
Correttezza	Alta (spesso garantita)	Variabile (richiede verifica)
Ambito	Limitato a pattern predefiniti	Ampio; può eseguire cambiamenti semantici complessi
Input	Comando o regola specifica	Linguaggio naturale, prompt orientati agli obiettivi
Flessibilità	Bassa; rigida	Alta; può inferire intento e contesto
Sfida Chiave	Scalabilità e ambito limitato	Allucinazioni e regressione funzionale

Il **principale vantaggio** introdotto dall'uso degli LLM è la flessibilità e la comprensione semantica, che permettono a questi modelli di effettuare refactoring complessi anche in ampi progetti. Inoltre il refactoring tradizionale richiede spesso elevate competenze e capacità da parte degli sviluppatori, oltre che interventi manuali, per poter identificare opportunità di ottimizzazione e applicare design pattern e test per migliorare il codice. Gli LLM offrono la possibilità di automatizzare questo processo, trasformandolo in una pipeline autonoma e adattabile a diversi scenari.

Tuttavia lo **svantaggio maggiore** introdotto dall'uso dell'AI è la mancanza di garanzie relative al mantenimento della correttezza del codice originario, in quanto gli LLM possono avere "allucinazioni" durante il processo di refactoring, restituendo risultati diversi da quelli attesi e introducendo problematiche, bug e/o errori. Ciò introduce la necessità di un robusto framework di test (Sezione 3.5).

La combinazione di entrambi gli approcci può rappresentare un interessante campo di ricerca, rilevante anche per questo progetto: le soluzioni proposte dagli LLM possono (e dovrebbero) essere poi revisionate mediante un intervento umano e/o l'uso di strumenti di analisi statica o basati su AST per assicurare il mantenimento della correttezza funzionale a seguito del refactoring.

2.3 Green Computing e Misurazione della Sostenibilità del Software

Il campo del Green Computing è relativo alla minimizzazione dell'impatto ambientale associato all'ambito ICT [9]. Mentre inizialmente l'attenzione veniva posta sugli aspetti hardware, il cui margine di miglioramento era maggiore, nel tempo il focus si è spostato verso il software, che gioca un ruolo critico nel consumo energetico. La seguente sezione sottolinea e motiva le metriche scelte per valutare la sostenibilità del codice utilizzate in questo lavoro di tesi.

Metriche di Efficienza (CPU, RAM, Tempo di Esecuzione)

È fondamentale misurare l'utilizzo delle risorse legate al software per valutarne la sostenibilità. Per tale motivo sono state selezionate tre metriche da monitorare prima e dopo il refactoring applicato dagli LLM al fine di valutarne l'efficacia e poter stimare l'impatto che hanno gli LLM nel ridurre il consumo energetico legato alle esecuzioni di codice. L'**uso di CPU** (misurato in %) rappresenta la porzione della capacità del processore impiegata durante l'esecuzione del programma: maggiore è l'uso di CPU maggiore sarà il consumo energetico legato al processore. L'**uso di RAM** misura la quantità di memoria (in kilobytes) utilizzata dal programma; solitamente il consumo di potenza associato alla memoria è più statico rispetto a quello associato alla CPU, tuttavia un'elevata quantità di allocazioni può aumentare l'impiego di energia necessario all'esecuzione. Infine il **tempo di esecuzione** è la quantità totale di tempo necessaria al programma per essere completamente eseguito, misurata in millisecondi: un programma particolarmente veloce sfrutta le risorse per un ammontare di tempo breve, rilasciandole non appena termina e, di conseguenza, impiega meno energia. Al contrario, un codice la cui esecuzione richiede maggiore tempo per terminare utilizza risorse per un periodo temporale più lungo, con relativo consumo energetico più elevato.

Queste metriche fondamentali sono state scelte perché universalmente misurabili in diversi linguaggi di programmazione e poiché forniscono un'indicazione diretta del carico computazionale associato ad un frammento di codice. La metodologia specifica per la raccolta di queste metriche è descritta in dettaglio nella Sezione 3.4.

Stima del Consumo Energetico (Joule)

Una misura diretta dell'impatto ambientale legato all'utilizzo del software è il consumo energetico associato alle esecuzioni, tipicamente quantificato in Joule (J) e utilizzabile per calcolare anche l'impronta di carbonio relativa all'utilizzo software. Misurare direttamente l'esatta quantità di energia utilizzata per l'esecuzione di un codice è un processo complesso che spesso richiede hardware specializzato.

Non potendo far affidamento su tali dispositivi di misura mediante hardware, il consumo energetico è stato stimato usando modelli basati sul software, considerando le metodologie più comuni in letteratura:

- **Modelli basati su TDP (Thermal Design Power):** Stimano la potenza assorbita come funzione dell'utilizzo della CPU rispetto al suo TDP, che rappresenta la massima potenza termica che il sistema di raffreddamento è progettato per dissipare [10].
- **Modelli a livello di sistema:** Considerano la potenza relativa all'utilizzo della RAM, correlandola ad un fattore di efficienza globale (PUE) [11, 12].

In questo lavoro di tesi, è stato adottato un modello analitico deterministico che combina le metriche di esecuzione raccolte (utilizzo CPU, occupazione RAM, tempo di esecuzione) con costanti hardware di riferimento. Maggiori dettagli riguardo al calcolo dell'energia come metrica derivata sono descritti nella Sezione 3.5.

2.4 Posizionamento della Tesi e Lacune della Letteratura

Lo stato corrente della letteratura rivela importanti ricerche nei seguenti ambiti:

- Lo sviluppo e la valutazione di LLM specializzati per la generazione di codice, auto-completamento e risoluzione di bug (Sezione 2.1);

- Lo studio relativo al processo di refactoring automatico mediante precisi strumenti, sfruttando sia tecniche tradizionali che basate sull'AI (Sezione 2.2);
- La misurazione e ottimizzazione del software legata al campo del Green Computing, incentrandosi su metriche come il consumo energetico (Sezione 2.3).

Tuttavia l'intersezione di questi campi fa emergere un'importante mancanza: **nessuno studio fino ad ora ha indagato sistematicamente l'impatto del refactoring basato su LLM specificamente sul consumo energetico e sulla sostenibilità ambientale.**

Nello stato dell'arte attuale i lavori recenti si sono concentrati prevalentemente su aspetti parziali del problema. Niu et al. [13] hanno valutato l'efficienza del codice generato dagli LLM su benchmark quali HumanEval e LeetCode, evidenziando come i modelli attuali producano spesso codice funzionalmente corretto ma sub-ottimale in termini di performance computazionale, senza tuttavia considerare l'impatto energetico. Shypula et al. [14] hanno proposto un framework per l'ottimizzazione del codice mediante LLM, introducendo un dataset di oltre 77.000 coppie di codici C++ con relative modifiche migliorative, limitandosi tuttavia ad un singolo linguaggio e a metriche di performance computazionale. Vartziotis et al. [15] hanno analizzato la "green capacity" di diversi LLM, mostrando che i modelli analizzati producono codice leggermente meno efficiente rispetto alle soluzioni umane ottimizzate; tuttavia lo studio riguarda la generazione di codice da zero piuttosto che il refactoring. Cursaru et al. [16] hanno condotto un esperimento controllato sull'efficienza energetica del codice generato da Code Llama su C++, JavaScript e Python, ma coinvolgendo un singolo modello e tre linguaggi. Sul versante teorico, Lano et al. [17] hanno analizzato l'impatto dei design pattern sulla sostenibilità del software, proponendo un pattern language per la riduzione energetica, senza tuttavia coinvolgere LLM né condurre un'analisi empirica. Infine Poy et al. [18] hanno analizzato l'impatto di pattern, code smell e tecniche di refactoring sul consumo energetico, concludendo che non vi è un effetto univoco, ma senza considerare il ruolo degli LLM nel processo.

Gli attuali studi considerano quindi singoli linguaggi e performance che non comprendono il consumo energetico [14], oppure si incentrano sulla generazione del codice piuttosto che sull'impatto del refactoring [15], su analisi teoriche anziché empiriche [17], coinvolgono pochi modelli e linguaggi [16] o non considerano l'impatto degli LLM [17, 18].

Questa tesi apporta il proprio contributo attraverso un'analisi empirica che compara più modelli affiancando l'utilità del processo di refactoring all'utilizzo degli LLM per compiere tale procedimento, combinando di conseguenza le capacità degli LLM ai benefici del refactoring e analizzandone gli effetti su sette diversi linguaggi mediante misure dei consumi energetici e individuazione dei pattern introdotti, oltre che ponendo particolare attenzione al ruolo del prompt engineering. Inoltre vengono valutate sia le capacità degli LLM di effettuare refactoring corretto e valido, sia l'impatto di tale refactoring in relazione all'efficienza energetica che può derivarne, misurando metriche quali uso di CPU, uso di RAM, tempo di esecuzione e soprattutto stimando il consumo energetico pre e post refactoring.

Questo lavoro si posiziona quindi come nuovo contributo nell'ambito del Green AI e sostenibilità relativa all'Ingegneria del Software [19], fornendo un'analisi empirica volta a verificare se gli LLM possano essere sfruttati come strumenti efficaci nella realizzazione di software più ecologico.

Capitolo 3

Progettazione e Metodologia Sperimentale

In questo capitolo viene presentata la fase di progettazione e realizzazione del dataset, accompagnata dalle scelte relative alla metodologia sperimentale utilizzata nella ricerca sul refactoring sostenibile mediante LLM. Questa prima fase del lavoro si suddivide nelle seguenti cinque sotto-fasi (ciascuna associata alla relativa sotto-sezione): costruzione del dataset e clustering (Sezione 3.1), selezione dei modelli LLM coinvolti nello studio (Sezione 3.2), raffinamento e ingegnerizzazione del prompt (prompt engineering) mediante realizzazione di più versioni (Sezione 3.3), creazione dell'infrastruttura per l'esecuzione e la raccolta delle metriche (Sezione 3.4), processo di validazione e valutazione dei risultati (Sezione 3.5).

Il design sperimentale è volto all'ottenimento di metriche valide e affidabili: ogni snippet di codice è associato alla relativa test suite al fine di garantirne la correttezza, mentre la stabilità delle misure è ottenuta eseguendo cinque volte ciascuna test suite sul relativo codice.

Gli scopi di queste scelte sono quindi: la creazione di un dataset ampio e bilanciato, la scelta oculata e mirata di linguaggi e modelli LLM, l'ottenimento di metriche significative per lo studio in questione e la relativa validazione volta alla valutazione (oggetto del Capitolo 4).

3.1 Costruzione e Clustering del Dataset

L'intero studio si appoggia su un dataset creato da zero, contenente coppie di codici e relative test suite (di seguito nominate “entry”) in vari linguaggi di programmazione. In questa sezione viene spiegato nel dettaglio l'intero processo di creazione, clustering e validazione del dataset che ha poi permesso di valutare le capacità degli LLM selezionati nel refactoring sostenibile di codici scritti interamente da umani.

Fonte dei Dati e Criteri di Selezione

La ricerca si è mossa verso l'individuazione di codici che potessero soddisfare molteplici requisiti. In primo luogo la **brevità**, ossia snippet di codici indipendenti e non annessi ad altri moduli, accompagnata dalla **presenza di test suite**, fondamentale per controllare e valutare se gli LLM siano in grado di mantenere la correttezza passando i “regression test”. Era inoltre necessaria una **copertura di più linguaggi** di programmazione con caratteristiche differenti, per garantire la significatività dello studio. Poiché lo scopo di questo lavoro è valutare le capacità di refactoring degli LLM, è stato essenziale ottenere codici **scritti da umani** (human made), possibilmente con diversi livelli di abilità e capacità: codici corretti che passassero le test suite ma potenzialmente inefficienti e quindi con margine di miglioramento nel processo di refactoring. Infine si è tenuto conto della **reperibilità e possibilità di utilizzo**, privilegiando file facilmente scaricabili in locale e utilizzabili senza vincoli di licenza, e della **rilevanza dei codici**, ossia programmi e script che utilizzassero strutture dati, algoritmi e implementazioni di soluzioni associate a problemi noti in modo da riflettere scenari reali di refactoring.

Per tali motivazioni la scelta si è ristretta ad un gruppo di piattaforme online mediante cui gli utenti possono esercitarsi scrivendo codici che vadano a risolvere uno o più problemi e che possono essere testati tramite apposite test suite. A riguardo sono state individuate tre principali piattaforme di esercizi che potessero soddisfare i requisiti: LeetCode, Codewars ed Exercism.

Tali siti tuttavia non forniscono accesso diretto alle proprie test suite e/o ai codici relativi alle soluzioni sottomesse dagli utenti, quindi è stato utilizzato GitHub ai fini di ricerca e creazione del dataset,

sfruttando query mirate volte alla ricerca di repository correlate alle piattaforme di esercizi individuate. Nello specifico la quasi interezza dei codici presenti nel dataset è relativa alla piattaforma **Exercism**, di cui è stata individuata una grande quantità di repository GitHub pubbliche appartenenti a utenti diversi; ciò ha reso inoltre possibile la realizzazione di script che automatizzassero il processo di ricerca e ottenimento delle entries (coppie <codice, test suite>) di cui si compone il dataset. Ogni entry (ed annessi file utili o necessari all'esecuzione dei codici) è stata salvata all'interno del dataset sfruttando una struttura volta alla categorizzazione per linguaggio e sotto-categorizzazione per coppia <esercizio, autore>.

Linguaggi di Programmazione

Sono stati individuati e scelti sette diversi linguaggi di programmazione al fine di coprire molteplici paradigmi di programmazione, modelli di gestione della memoria, differenti tipologie di performance e caratteristiche. La Tabella 3.1 riassume i linguaggi selezionati descrivendone brevemente le principali proprietà.

Tabella 3.1: Linguaggi di programmazione nel dataset

Linguaggio	Paradigma	Modello Memoria	Compilazione	Tipizzazione
Python	Multi-paradigma	Reference counting + GC ciclico	Interpretato	Dinamica
JavaScript	Multi-paradigma	GC automatico	Interpretato/JIT	Dinamica
TypeScript	Multi-paradigma	GC automatico	Transpilato/JIT	Statica
Java	Orientato agli oggetti	GC automatico	Compilato (JVM)	Statica
Go	Procedurale/Concorrente	GC automatico	Compilato	Statica
C	Procedurale	Manuale	Compilato	Statica
C++	Multi-paradigma	Manuale/RAII	Compilato	Statica

Le motivazioni collegate alla scelta dei linguaggi sono molteplici. Prima tra tutte, l'inclusione di **linguaggi interpretati** (JavaScript, Python) e **compilati** (C, C++, Go, Java) permette di valutare il comportamento degli LLM nel refactoring di queste diverse tipologie, mentre la decisione di inserire sia linguaggi con una **gestione manuale della memoria** che linguaggi supportati dalla presenza del **Garbage Collector** consente di apprezzare le differenze nel refactoring di queste categorie, specialmente nell'utilizzo della RAM. Infine la presenza di linguaggi con **sistemi di tipi eterogenei** (dinamici e statici, debolmente e fortemente tipizzati) e con elevate divergenze di **livelli di astrazione** (alto livello come Python fino a basso livello come C) arricchisce ulteriormente la qualità dei risultati, dando la possibilità di analizzare a fondo le capacità e il comportamento degli LLM rispetto al refactoring di codice associato alle diverse caratteristiche dei linguaggi scelti.

Organizzazione in Cluster e Scala del Dataset

Ai fini dell'analisi il dataset è stato suddiviso in “cluster”: gruppi di entries i cui codici risolvono lo stesso esercizio di Exercism (ad esempio “Two Sum”, “Prime Factors”, “RNA Transcription”), implementati in linguaggi differenti e da diversi autori.

Ogni cluster raggruppa le “entry di base”, ossia codici relativi alle soluzioni proposte dalla community che costituiscono il dataset oggetto di studio. A fini sperimentali, ad ogni entry di base sono state successivamente associate **dodici varianti generate dagli LLM** (3 modelli $\times$ 4 versioni del prompt).

Struttura dei Cluster

I cluster vengono modellati mediante file JSON (`cluster_{exerciseName}.json`) che ne mantengono le informazioni e i metadati. La struttura prevede una **divisione per linguaggio**, con array di entries presenti nel cluster per ogni linguaggio (non necessariamente un cluster ha tutti e sette i linguaggi scelti). Per ogni entry di base vengono salvati **metadati** quali percorsi ai file di codice e test, metriche (quantità di caratteri e di parole), data di download, fonte e tipo di licenza. Infine le **varianti generate dagli LLM** sono associate ad un'entry di base e raggruppate in un array; per ogni entry LLM vi sono metadati quali: tipo del LLM, percorso del file, quantità di parole e di caratteri, metriche di similarità rispetto alla corrispondente entry di base. Tali metriche di similarità (fuzzy

score, cosine similarity e similarity index) vengono calcolate da appositi script e rispecchiano quanto il codice generato dagli LLM sia diverso rispetto al codice originario (di base); vengono sfruttate per analisi approfondite sui miglioramenti delle performance.

Scala del Dataset

Il dataset iniziale comprendeva 2568 entries raggruppate in 401 cluster. La Figura 3.1 mostra la distribuzione delle entries per linguaggio nel dataset iniziale, prima dell'applicazione del filtro di validazione relativo al pass rate (descritto nella Sezione 3.1).

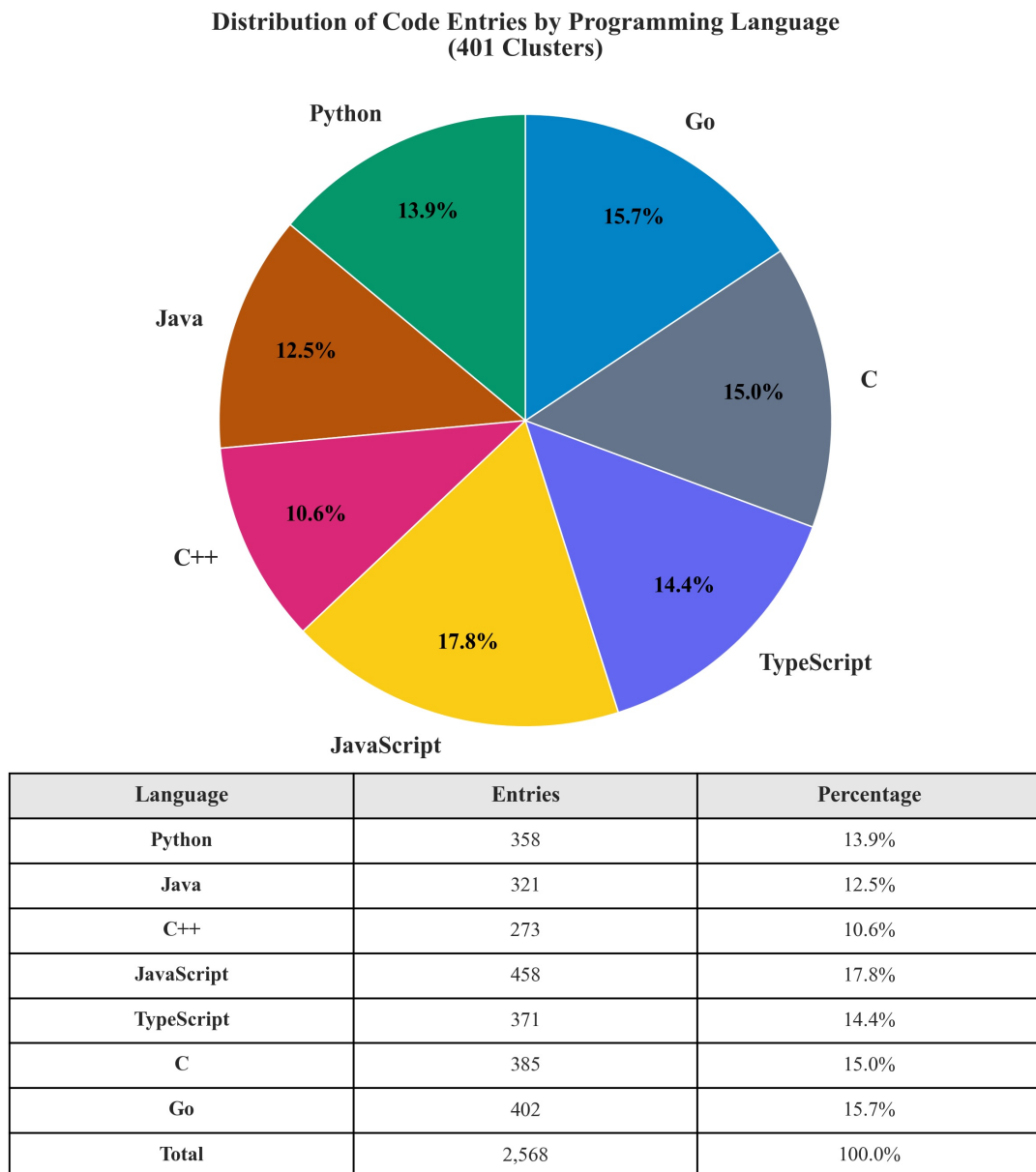
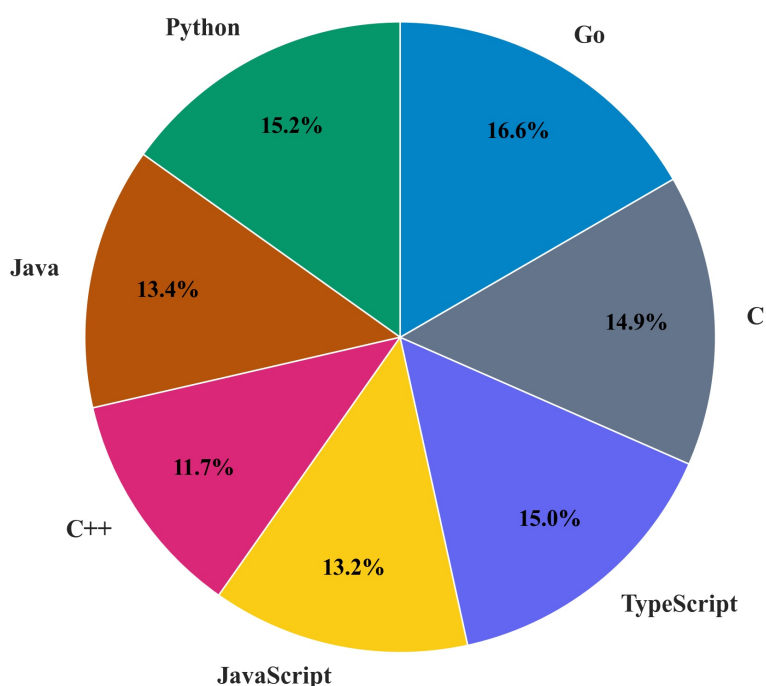


Figura 3.1: Distribuzione delle entries nel dataset per linguaggio di programmazione (dataset iniziale, non filtrato).

Tuttavia, a seguito della validazione volta a mantenere unicamente le entries (di base) con 100% di pass rate su 5 esecuzioni, le dimensioni del dataset si sono leggermente ridotte, ottenendo un totale di **2.343 entries** suddivise in **392 cluster**. La Figura 3.2 illustra la distribuzione finale delle entries per linguaggio nel dataset validato.

**Distribution of Code Entries by Programming Language
(Entries with 100% Pass Rate Only - 392 Clusters)**



Language	Entries	Percentage
Python	355	15.2%
Java	315	13.4%
C++	273	11.7%
JavaScript	309	13.2%
TypeScript	351	15.0%
C	350	14.9%
Go	390	16.6%
Total	2,343	100.0%

Figura 3.2: Distribuzione delle entries nel dataset per linguaggio di programmazione (dataset filtrato, solo entries con 100% pass rate).

È interessante notare come a 2.343 base entries corrispondano 28.116 (2.343×12) codici generati dagli LLM per un totale di oltre **150.000 esecuzioni** (5 per codice) considerando sia codici LLM che codici originari; ammontare che garantisce robustezza statistica.

Processo di Validazione del Dataset

La qualità e affidabilità dell'intero dataset sono garantite da un processo di validazione articolato in più fasi e volto ad assicurare la completezza dei dati e metadati, oltre che la correttezza dei codici (entries di base) considerati.

Pipeline di Validazione

Per raggiungere tale scopo è stata sfruttata una pipeline articolata nelle seguenti fasi:

1. **Verifica di compilazione:** è stato verificato che ogni entry di base compilasse correttamente (per i linguaggi compilati) o che non avesse errori di sintassi (per i linguaggi interpretati), depurando in tal modo il dataset da codici che presentassero inesattezze già in questa fase.

2. **Esecuzione e filtro di correttezza:** ogni test suite appartenente ad un'entry di base è stata eseguita cinque volte sul relativo codice e sono stati mantenuti nel dataset solamente codici con il 100% di pass rate (5 esecuzioni passate con successo su 5).
3. **Completezza dei metadati:** un rigoroso controllo dei metadati associati alle entries è stato effettuato mediante scansioni dei file JSON relativi ai cluster al fine di assicurarne la completezza e correttezza, strettamente necessarie ai fini delle analisi ed esecuzione degli script.
4. **Validazione delle metriche:** durante la fase di esecuzione e raccolta delle relative metriche, valori invalidi (quali ad esempio None, Null) sono stati scartati per garantire l'ottenimento solamente di metriche valide per le analisi.

Impatto della Validazione

A seguito della pipeline di validazione l'intero dataset ha raggiunto:

- 100% di pass rate su ogni entry di base
- 100% di completezza e correttezza dei metadati
- Utilizzo di metriche validate per la creazione delle analisi e relativi grafici

Osservazione sul Filtro del Pass Rate

La decisione di mantenere all'interno del dataset unicamente entries di base con un pass rate del 100% è un punto critico per la precisione complessiva del dataset in quanto permette di ottenere una **condizione di correttezza iniziale certa** (o *Ground Truth*), fondamentale ai fini dello studio in quanto lo scopo è quello di osservare il comportamento degli LLM nel refactoring di codici privi di errori e non di valutare le capacità degli LLM nel risolvere eventuali problematiche presenti nei codici.

Se il codice di base presenta errori che causano un'esecuzione scorretta non è possibile valutare la qualità del refactoring da parte degli LLM e in particolare non è possibile valutarne la capacità di mantenere il corretto comportamento del programma iniziale (regression test).

L'applicazione di questo filtro permette quindi di apprezzare la veridicità dei risultati: codici generati dagli LLM che non ottengono un pass rate del 100% su cinque esecuzioni rappresentano una **degradazione di correttezza** introdotta dall'utilizzo dei modelli LLM.

3.2 Modelli Selezionati

Il processo di selezione dei modelli si è basato su molteplici criteri: la dimostrata capacità di generazione di codice corretto e funzionante in vari linguaggi di programmazione, l'accessibilità via API per l'automazione del processo di generazione, la rilevanza e l'ampio utilizzo dei modelli nel campo dello sviluppo software, l'eterogeneità dei fornitori (OpenAI, Anthropic, Google) per evitare bias legati a un singolo ecosistema, e l'interesse nel valutare e comparare le prestazioni su un ampio spettro di linguaggi di programmazione.

Tutti i modelli selezionati appartengono alla famiglia degli LLM basati su architettura Transformer e sono stati scelti in quanto rappresentano lo stato dell'arte (SOTA) al momento dello studio. Essi condividono caratteristiche fondamentali quali la natura multi-modale (capacità di processare input non solo testuali), l'accesso tramite API per l'integrazione in pipeline automatiche e l'addestramento su vasti corpora di dati, inclusi codice sorgente e documentazione. Di seguito vengono dettagliate le specificità che hanno motivato l'inclusione di ciascun modello nello studio.

OpenAI GPT-4

OpenAI GPT-4 [20] (riferito come "ChatGPT4") è stato selezionato in quanto rappresenta il benchmark di riferimento attuale per le capacità di ragionamento complesso e general-purpose. Privilegia la qualità del ragionamento e l'aderenza alle istruzioni (allineamento mediante RLHF), fungendo da baseline di alta qualità per lo studio. La sua architettura ad alta densità di parametri (MoE – Mixture

of Experts) è ottimizzata per task logici complessi, sebbene con costi e latenze maggiori rispetto ad altri modelli. La convenzione adottata per i nomi dei file è `ChatGPT4_{exerciseName}_v{1-4}.ext`, dove `v{1-4}` indica la versione del prompt.

Anthropic Claude Sonnet 4

Anthropic Claude Sonnet 4 [21] (riferito come “claude”) è un modello sviluppato da Anthropic mediante la metodologia di training “Constitutional AI”, che sfrutta agenti per lo sviluppo di codice. Eccelle in task associate al coding, tra cui il refactoring, e possiede elevate capacità di comprensione del contesto grazie ad un’ampia finestra di contesto (200k+ token) che permette di analizzare interi moduli di codice e documentazione senza perdita di informazioni. La convenzione adottata per i nomi dei file è `ClaudeSonnet_{exerciseName}_v{1-4}.ext`.

Google Gemini Flash 2.0

Questo modello [22] (riferito come “gemini”) è stato rilasciato ad inizio 2025 da Google DeepMind come modello multi-modale a bassa latenza e alto throughput, particolarmente indicato per analisi dati e applicazioni real-time. Rappresenta la categoria dei modelli ottimizzati per la velocità e il throughput, utilizzati prevalentemente in scenari di sviluppo real-time o su larga scala. La sua architettura è ottimizzata per ridurre il “Time to First Token” (TTFT) e gestire finestre di contesto estremamente ampie (fino a 1 milione di token), offrendo un approccio alternativo incentrato sulla rapidità di esecuzione. La convenzione adottata per i nomi dei file è `GeminiFlash_{exerciseName}_v{1-4}.ext`.

Comparazione dei Modelli

I tre modelli selezionati presentano differenze nell’approccio dello sviluppo degli LLM, specialmente per quanto concerne il trade-off ragionamento-velocità: **GPT-4** privilegia generalità e ragionamento profondo rispetto a latenza e costi, che vengono invece posti al primo posto da **Gemini Flash 2.0**, mentre **Claude Sonnet 4** mantiene un trade-off più bilanciato e ha capacità specifiche e ottimali nell’uso di agenti per il coding.

Le differenze che emergono dalla comparazione, le diverse architetture e processi di apprendimento dei modelli considerati permettono di valutare quanto l’uso di LLM eterogenei influisca nel refactoring sostenibile, non limitando lo studio a singoli fornitori, bensì apprezzando i distinti aspetti e caratteristiche appartenenti ai modelli di diverse organizzazioni (OpenAI, Anthropic, Google).

Configurazione dei Modelli

Per assicurare una corretta comparazione tra i modelli selezionati, ogni LLM è stato configurato con i medesimi parametri. La temperatura è stata posta a 0.1 al fine di avere un comportamento deterministico e ridurre la variabilità tra gli output delle richieste di generazione di codice. La quantità massima di token è stata configurata a 4000, in base alla lunghezza massima attesa per i codici generati dagli LLM. Infine ogni modello riceve in input le medesime istruzioni (i cui dettagli si trovano nella Sezione 3.3) per garantire la consistenza. Le precedenti scelte di configurazione sono mirate ad ottenere controllo sperimentale e riproducibilità.

3.3 Progettazione, Ingegnerizzazione e Versioning dei Prompt

Il design del prompt è un aspetto critico nel processo di interazione con gli LLM e in questo studio rappresenta una variabile a sé da cui dipende fortemente il comportamento degli LLM e la relativa qualità dei risultati ottenuti.

In questo lavoro di tesi il processo di prompt engineering attraversa la creazione e rifinitura di quattro progressive versioni del prompt volte a valutare l’impatto che hanno la specificità, l’inquadramento e l’enfasi delle istruzioni nei risultati del refactoring.

Metodologia del Processo di Ingegnerizzazione dei Prompt

Questo procedimento segue un approccio iterativo, volto al progressivo aumento della specificità delle istruzioni contenute nel prompt e alla rifinitura dell’architettura che avvolge tali istruzioni. Ogni versione del prompt è stata creata per soddisfare alcuni requisiti fondamentali: la **chiarezza nella definizione dell’obiettivo del refactoring**, ossia l’ottimizzazione del codice al fine di ridurre il

consumo energetico e l'utilizzo di risorse; la necessità di **preservare la correttezza**, richiedendo esplicitamente di mantenere il comportamento del codice originario attraverso l'utilizzo delle stesse interfacce; l'imposizione di **vincoli specifici** che proibiscano modifiche a interfacce esterne e firme di funzioni presenti nei codici originali; e infine la definizione della **tipologia di output**, richiedendo agli LLM di ritornare unicamente il codice senza allegare spiegazioni testuali, in modo da semplificare il processo di automazione.

Le quattro versioni del prompt rappresentano quindi una progressione dall'utilizzo di istruzioni generiche e concise (P1) fino all'impiego di direttive strutturate con formattazione esplicita (P4).

Prompt Versione 1: Baseline

La prima versione del prompt stabilisce la baseline mediante l'impiego di semplici istruzioni che enfatizzano i principali criteri di ottimizzazione:

You are an experienced software engineer. Below is a code snippet written in {language} delimited by triple backticks (""), without the tests. Your task is to return an optimized version of the snippet, which keeps the original behavior and interface (functions' signatures), while reducing the runtime energy consumption:

Optimize the code according to these criteria:

- 1. Minimize execution times*
- 2. Minimize resource usage (e.g., memory, CPU)*
- 3. Avoid modifying functions' signatures*
- 4. Follow sustainable software engineering best practices, guidelines and pattern*

Respond only with updated code, without further explanation.

La versione 1 utilizza un linguaggio chiaro e professionale e sfrutta l'enumerazione dei criteri. È presente un riferimento all'"Ingegneria del Software sostenibile" che introduce una terminologia specifica del settore, in linea con gli standard promossi dalla Green Software Foundation [23].

Prompt Versione 2: Semplificazione

La seconda versione del prompt è volta a semplificare il linguaggio e la struttura, mantenendo i requisiti fondamentali:

You are an experienced software engineer. Below is a code snippet written in {language}, delimited by triple backticks (""), it does not include any tests. Your task is to return an optimized version of the code, that preserves the functionality and the function signatures, while reducing the runtime energy consumption.

Follow these optimization guidelines:

- Improve execution efficiency, minimizing execution time*
- Reduce resource consumption (e.g., memory, CPU)*
- Apply sustainable software engineering principles where relevant*

Respond only with updated code, without further explanation.

Questa versione snellisce il linguaggio, sfruttando "linee guida" anziché "criteri" e riducendo la quantità dei punti da quattro a tre.

Prompt Versione 3: Ruolo dell'Ingegnere Software

La terza versione del prompt introduce il concetto di "expertise framing" e pone attenzione agli standard professionali:

You are a senior-level software engineer with expertise in sustainable computing and performance optimization. A code snippet written in {language} is provided below, marked by triple backticks (""). Your task is to optimize it while preserving its original functionality and interface.

Follow these instructions:

- *Refactor the code to improve runtime performance and reduce overall energy consumption*
- *Limit memory and CPU usage where possible*
- *Preserve all existing function names and signatures (do not alter the external interface)*
- *Apply clean and maintainable coding practices in line with sustainable software engineering standards*

Respond only with the updated code, without explanations or comments.

Questa versione eleva il ruolo del modello a “ingegnere senior con esperienza”, sfruttando i risultati della ricerca secondo cui l’inquadramento basato sul ruolo può migliorare le prestazioni dei modelli LLM [4].

Prompt Versione 4: Strutturato ed esplicito

L’ultima versione del prompt rappresenta la formulazione più dettagliata e strutturata, con un’esplicita quotazione dei vincoli e un’enumerazione degli obiettivi:

You are a senior software engineer with expertise in performance optimization and sustainable computing. Below is a code snippet written in {language}, delimited by triple backticks (“”), and it does not include any tests.

Your task is to return an optimized version of the code that:

- *“Preserves the original functionality”*
- *“Does not alter any function names or signatures”*
- *“Maintains the external interface exactly as provided”*

Optimize the code following these clearly defined goals:

1. *“Improve execution performance (minimize execution time)”*
2. *“Reduce runtime resource usage (e.g., CPU and memory)”*
3. *“Minimize energy consumption”*
4. *“Apply clean, maintainable coding practices”*
5. *“Follow sustainable software engineering principles and standards”*

Respond only with the updated and optimized code, without any explanation or comments.

La versione 4 è quella con la massima specificità, sfruttando l’uso delle virgolette per evidenziare le frasi chiave e aumentando potenzialmente l’importanza dei vincoli mediante l’uso dell’elenco a punti.

Accortezze per la Coerenza

Per garantire un’equa valutazione tra differenti modelli e linguaggi in tutte le versioni del prompt sono state applicate diverse accortezze. La **parametrizzazione del linguaggio** prevede che {language} venga sostituito con l’effettivo nome del linguaggio utilizzato nel codice preso in considerazione. Per l’**inserimento del codice originario**, tale codice è stato inserito immediatamente dopo il prompt, delimitato da tripli backtick (‘‘‘), come specificato dal prompt stesso. La **formulazione indipendente dal modello** assicura che ogni prompt sia generico ed eviti di rivolgersi al modello stesso, al fine di poter essere utilizzato tramite API di ogni modello. Infine ogni prompt chiedeva esplicitamente di restituire in output unicamente il codice.

Riepilogo Comparativo delle Versioni del Prompt

La Tabella 3.2 riassume le principali differenze tra le quattro versioni del prompt, evidenziando l’evoluzione delle caratteristiche di design.

Strategia di Generazione: Approccio One-Shot

Ogni codice generato mediante l’uso dei Large Language Model è stato creato sfruttando una strategia “one-shot”, ossia mediante l’utilizzo di una singola chiamata API (singolo messaggio) al modello in questione, andando a ricreare la sessione di interazione con il modello per ogni invio e ogni generazione. Ciò garantisce che non vi siano bias di contesto dovuti a strategie di chain of thoughts, feedback

Tabella 3.2: Confronto delle caratteristiche di design tra le versioni del prompt

Caratteristica	P1	P2	P3	P4
Inquadramento Ruolo	Esperto	Esperto	Senior	Senior
Dominio Expertise	—	—	Sustainable computing	Performance + Sustainable
Formato Vincoli	Lista criteri	Linee guida	Istruzioni	Vincoli quotati
Numero di Obiettivi	4	3	4	5
Energia Esplicita	Implicita	Implicita	Sì	Sì (obiettivo separato)
Livello di Struttura	Medio	Basso	Medio	Alto

iterativi o, in generale, riferimenti a conversazioni precedenti: ogni refactoring è stato effettuato senza l'influenza di esempi o interazioni precedenti.

L'approccio one-shot assicura quindi:

- **Confronto equo:** tutti i modelli e tutte le versioni del prompt sono state valutate sotto medesimi vincoli di interazione con gli LLM;
- **Realismo in ambito pratico:** la strategia utilizzata riflette scenari reali in cui non è possibile (per via di limiti e risorse) ottenere un refactoring mediante altre tipologie di interazione con gli LLM.

3.4 Infrastruttura di Esecuzione e Raccolta Metriche

Al fine di garantire la riproducibilità del lavoro, per poter raccogliere le metriche in modo affidabile e soprattutto indipendente dalla macchina su cui venivano eseguiti i test, l'intero apparato di esecuzione sfrutta un'infrastruttura containerizzata.

In questa sezione vengono descritti il processo di esecuzione e la relativa infrastruttura, l'uso di Docker per i container, il procedimento di raccolta delle metriche e la relativa validità statistica.

Ambiente di Test Isolato (Docker/Colima)

L'utilizzo di container Docker (mediante Colima per utilizzo lightweight) per l'esecuzione delle test suite su ogni codice è volto alla creazione di un ambiente di esecuzione indipendente e riproducibile, oltre che isolato dalle caratteristiche della macchina.

L'impiego di Docker garantisce quindi criteri fondamentali ai fini dello studio. In primo luogo un **ambiente isolato**, in cui ogni esecuzione viene svolta impedendo interferenze dovute a processi esterni e/o stati residui da precedenti esecuzioni. In secondo luogo la **riproducibilità**, poiché la coerenza delle configurazioni tra diverse esecuzioni e l'indipendenza dalla macchina garantiscono la possibilità di riprodurre i test su qualsiasi macchina e in qualsiasi momento. Docker è stato inoltre sfruttato per il **monitoraggio delle risorse**, ottenendo metriche di esecuzione quali tempo di esecuzione, uso di CPU e RAM. Infine la possibilità di definire **configurazioni specifiche** per ogni linguaggio consente di scegliere e impostare dipendenze, versioni delle librerie e tutto ciò che è necessario ai fini dell'esecuzione dei test.

Architettura dei Container

L'infrastruttura di esecuzione utilizza immagini Docker specifiche per linguaggio, ciascuna configurata con l'ambiente di runtime e gli strumenti di compilazione necessari:

- **Python:** immagine ufficiale Python 3.11 con libreria standard e framework di test (pytest, unittest);
- **JavaScript/TypeScript:** Node.js LTS con framework di test Jest e compilatore TypeScript;
- **Java:** OpenJDK con framework di test JUnit e impostazioni JVM ottimizzate;
- **Go:** immagine Go ufficiale con compilatore Go e pacchetto di test integrato;

- **C/C++:** immagine di base con suite di compilatori GCC e sistema di compilazione make, insieme a framework di test.

I file Dockerfile sono gestiti nella directory `/src/docker/`, garantendo il controllo della versione e la riproducibilità della configurazione dell'ambiente di esecuzione.

Affidabilità delle misurazioni mediante Esecuzioni Multiple

Su tutti i codici, sia quelli originari che quelli generati dagli LLM, sono state effettuate **cinque esecuzioni**:

- **Entries di base:** i risultati delle cinque esecuzioni dei codici “di base” sono stati salvati opportunamente in file JSON con la nomenclatura:
`{clusterName}_results_{1-5}.json`;
- **Entries LLM:** per ogni codice generato dagli LLM (3 modelli $\times$ 4 versioni del prompt, ossia 12 varianti per ogni codice originario) le cinque esecuzioni hanno prodotto risultati salvati in file JSON `{clusterName}_results_v{1-4}_{1-5}.json`;
- **Quantità totale di esecuzioni per entry:** $5 + (3 \times 4 \times 5) = 65$ esecuzioni.

Ciò, avendo un dataset di oltre 2300 entries, ha portato ad ottenere un totale di oltre **150.000 esecuzioni** e relative metriche, permettendo quindi di svolgere analisi approfondite su tali risultati.

La scelta di cinque esecuzioni è motivata da due fattori: la **stabilità e affidabilità**, in quanto tramite la raccolta di più risultati per ogni entry è possibile calcolare con maggior grado di accuratezza le statistiche associate alle analisi dei risultati; e il **costo computazionale**, poiché cinque esecuzioni rappresentano una quantità adeguata per un trade-off tra solidità delle misurazioni e tempo necessario alle esecuzioni.

Raccolta delle Metriche Grezze

Durante la fase di esecuzione delle entries sono state raccolte alcune metriche essenziali per le analisi e scelte appositamente per questo studio; metriche volte a definire la correttezza e le performance del codice:

Uso di CPU

Viene ottenuta la percentuale di CPU impiegata durante l'esecuzione, relativa al container Docker in cui viene eseguita l'entry.

Uso di RAM

Viene misurato il consumo di memoria in kilobyte relativo all'esecuzione dei codici all'interno dell'ambiente containerizzato.

Tempo di Esecuzione

Viene ottenuto il tempo necessario all'esecuzione con una precisione in millisecondi. Tale metrica non è il semplice wall-clock time, bensì rappresenta il **process time della CPU**, ossia la somma tra user e system time.

Stato del Test di Regressione

Si tratta di un booleano che indica la capacità di un codice di passare tutti i test presenti nella test suite associata. È importante osservare che per le entries di base questo valore di verità è positivo per ogni esecuzione (come precedentemente descritto), mentre è particolarmente importante per i codici generati dagli LLM in quanto questo flag indica se il singolo codice è riuscito a passare o meno la singola esecuzione.

Formato dei Risultati Salvati

Le metriche ottenute dalle esecuzioni vengono memorizzate in file JSON che sfruttano il seguente formato ai fini di velocizzare e automatizzare i processi di analisi mediante l'impiego di una struttura fissa e coerente:

```
1 {  
2   "id": "cpp_10-darts_ARPIT73881",  
3   "filename": "darts.cpp",  
4   "language": "cpp",  
5   "execution_time_ms": 1.946949,  
6   "CPU_usage": 68.0,  
7   "RAM_usage": 10496,  
8   "regressionTestPassed": true,  
9   "success": true,  
10  "error_message": null  
11 }
```

Listing 3.1: Struttura dell'output JSON delle metriche grezze

I risultati delle esecuzioni vengono salvati all'interno della cartella `/src/execution.outputs`.

Pipeline per la Collezione delle Metriche

Il flusso di lavoro associato alla raccolta delle metriche percorre le seguenti fasi:

1. **Inizializzazione del container:** per ogni linguaggio viene inizializzato un apposito container Docker con opportuni accorgimenti e impostazioni necessarie al monitoraggio delle metriche;
2. **Caricamento del codice:** vengono individuati i file da eseguire (test suite + codice originale o codice generato dagli LLM) e ne viene copiato il contenuto all'interno dell'ambiente containerizzato;
3. **Esecuzione:** viene avviata l'effettiva esecuzione della test suite sul relativo codice, misurando il tempo e monitorando CPU e RAM;
4. **Raccolta:** le metriche vengono individuate e opportunamente salvate nei relativi file JSON;
5. **Pulizia del container:** al termine di questa pipeline il container impiegato per l'esecuzione viene terminato in modo da garantire un ambiente nuovo per future esecuzioni.

L'infrastruttura containerizzata e l'impiego di un protocollo che garantisca molteplici esecuzioni assicurano che le metriche raccolte mediante questo procedimento siano affidabili e che l'intero processo sia riproducibile.

3.5 Processo di Valutazione, Validazione e Metriche Derivate

Le metriche grezze costituiscono il punto di partenza per analisi approfondite che sfruttano metriche aggregate per poter valutare il comportamento degli LLM nel refactoring sostenibile.

In questa sezione vengono descritte la pipeline di validazione, la metodologia scelta e le relative motivazioni riguardo all'esclusiva analisi di base entries corrette, le metriche derivate (quali l'energia) con le relative formule e infine viene definita la "riduzione percentuale".

Validazione e Controlli di Qualità dei Dati

È stato realizzato un modulo dedicato alla validazione rigorosa dei dati raccolti mediante la fase di esecuzione, volto ad invalidare risultati scorretti e/o parziali.

Il modulo **Unified Analyzer** (disponibile in `/src/unified_analyzer/`) implementa una serie di fasi di validazione che assicurano l'integrità, completezza e correttezza delle metriche raccolte.

Pulizia dei Dati

Un'esecuzione viene ritenuta non valida e non viene considerata durante l'analisi se presenta almeno una delle seguenti problematiche:

- **Valori mancanti:** risultati delle esecuzioni con valori None o Null per una qualsiasi metrica;
- **Valori invalidi:** qualsiasi valore negativo o uguale a zero (fisicamente impossibile) rappresenta un'inesattezza nella misurazione;
- **Risultati incompleti:** qualora non sia presente lo status relativo all'esecuzione della test suite o qualora siano presenti errori di esecuzione che non permettono di raccogliere le metriche.

Filtro sulla Correttezza del Codice

In questa sezione viene descritta una decisione critica nel processo di valutazione di questo studio: la differenza dei filtri di correttezza applicata tra codici originari (base entries) e codici generati dagli LLM.

Base Entries: Requisito del 100% di Pass Rate

Al fine di costruire un dataset di riferimento valido su cui basare lo studio è necessario che ogni codice di base (scritto da persone) passi tutte e cinque le esecuzioni della relativa test suite.

Il codice originario rappresenta una “fonte di verità” e in quanto tale qualsiasi comportamento non deterministico deve essere categorizzato come invalido. Per tale motivo un pass rate inferiore al 100% per il codice di base indica errori nel codice, test suite difettose e/o errori legati all'ambiente di esecuzione.

L'utilizzo di tale filtro è volto anche e soprattutto ad una scelta critica per lo studio: analizzare le capacità degli LLM nell'effettuare il refactoring di codici mantenendone la correttezza, piuttosto che osservare le capacità degli LLM nella risoluzione di problematiche già presenti nei codici originari.

Codici LLM: Filtri per le Metriche

A differenza delle entries di base, un codice generato dagli LLM non deve necessariamente passare tutte e cinque le esecuzioni per essere considerato ai fini delle analisi. Il **filtro sulle metriche** prevede che uso di CPU, uso di RAM e tempo di esecuzione vengano raccolti unicamente dalle esecuzioni in cui la test suite ha un esito positivo. Per quanto riguarda l'**inclusione di una variante LLM**, questa viene considerata nelle analisi relative alle performance se è in grado di passare almeno un'esecuzione su cinque (viene sempre considerata ai fini delle analisi associate al pass rate). Infine la **metodologia di analisi mediante media** prevede che ogni entry LLM venga eseguita cinque volte e che ai fini delle analisi venga calcolata la media (per ogni metrica) sulle esecuzioni avvenute con successo.

Riepilogo del Protocollo di Filtraggio

La Tabella 3.3 sintetizza le regole di filtraggio asimmetriche applicate ai codici di base e ai codici generati dagli LLM.

Tabella 3.3: Riepilogo delle regole di filtraggio asimmetriche per codici Base e LLM		
Tipo di Codice	Soglia di Inclusione	Calcolo Metrica
Codice Base	100% pass rate (5/5)	Media di 5 esecuzioni
Codice LLM (Performance)	$\geq 20\%$ pass rate ($\geq 1/5$)	Media di N esecuzioni passate ($1 \leq N \leq 5$)
Codice LLM (Pass Rate)	Tutte le varianti (0-100%)	Percentuale esecuzioni passate

Calcolo del Consumo Energetico (Joule)

Il consumo di energia è la principale metrica derivata utilizzata in questo studio per racchiudere in un unico valore l'impatto delle metriche uso di CPU, uso di RAM e tempo di esecuzione. L'approccio utilizzato è conforme alle metodologie recenti per il calcolo dell'energia del software [24].

Configurazione e Parametri Hardware

Per poter calcolare il consumo di energia è stato necessario definire alcuni parametri:

- **TDP CPU WATT**: “Thermal Design Power” relativo alla CPU, impostato a 95.0 watts;
- **POWER PER GB RAM WATT**: rappresenta la potenza per gigabyte di RAM, impostato a 0.375 watts (standard medio per modelli RAM DDR4);
- **PUE FACTOR**: “Power Usage Effectiveness”, impostato a 1.5 (valore assimilabile alla media Europea).

Formula del Calcolo dell'Energia

Il consumo di energia (in Joule) è calcolato sfruttando le seguenti formule [24]:

$$t_{sec} = \frac{t_{ms}}{1000} \quad (3.1)$$

$$P_{CPU} = TDP \times \frac{CPU_{usage}}{100} \quad (3.2)$$

$$P_{RAM} = \frac{RAM_{KB}}{1048576} \times P_{per.GB} \quad (3.3)$$

$$P_{total} = (P_{CPU} + P_{RAM}) \times PUE \quad (3.4)$$

$$E_{Joule} = P_{total} \times t_{sec} \quad (3.5)$$

Definizione della Riduzione Percentuale

Per poter quantificare l'impatto del refactoring svolto dagli LLM viene calcolata la variazione percentuale per ogni metrica, comparando quindi il codice generato dagli LLM rispetto al codice originario.

Formula della Riduzione

Fissando un'entry di base e una delle relative 12 entry LLM, data una metrica M tra quelle considerate (uso di CPU, uso di RAM, tempo di esecuzione, pass rate o consumo di energia) la riduzione percentuale è calcolata come:

$$Reduction_{\%} = \frac{M_{LLM} - M_{base}}{M_{base}} \times 100 \quad (3.6)$$

dove M_{LLM} e M_{base} rappresentano il valore della media sulle esecuzioni relative alla metrica M .

Interpretazione Semantica

Per le metriche di performance (uso di CPU, uso di RAM, tempo di esecuzione, consumo di energia):

- Un valore **negativo** rappresenta una **riduzione** della metrica in questione (ad esempio, -42% indica una riduzione del 42%);
- Un valore **positivo** indica un **peggioramento**, ossia un aumento per la metrica considerata (ad esempio, $+20\%$ indica un incremento del 20%).

Pipeline di Automazione

Nel processo di creazione e miglioramento del dataset l'intero procedimento di ricerca, validazione e aggiunta di nuove entries è stato automatizzato mediante la creazione di una pipeline che svolge in sequenza i seguenti passaggi:

1. **Aggiunta di nuove base entries:** mediante l'utilizzo dello script `github_ingestor.py` vengono aggiunte nuove base entries al dataset, una volta validate;
2. **Generazione delle LLM entries:** attraverso l'utilizzo dello script `orphan_orchestrator` vengono individuate tutte le nuove base entries e vengono generati i codici LLM;
3. **Esecuzione mirata delle nuove entries LLM:** mediante lo script `new_llm_selective_runner.py` vengono eseguite le entries LLM che non hanno tutti i risultati;
4. **Rigenerazione delle statistiche e grafici:** mediante il file `generate_all_plots.sh` vengono eseguiti in sequenza gli script per creare nuovamente le statistiche.

L'intera pipeline è completamente automatizzata e configurabile mediante il file `main_pipeline.sh`. Questa pipeline rappresenta un contributo significativo al lavoro di tesi in quanto permette l'ulteriore ampliamento del dataset.

Capitolo 4

Risultati Sperimentali

In questo capitolo vengono presentati i risultati sperimentali ottenuti dalle analisi sugli esiti delle esecuzioni e le relative metriche raccolte. L'obiettivo principale di questa fase dello studio è quello di rispondere alle domande di ricerca poste come base dell'intero lavoro:

- “Gli LLM sono in grado di migliorare le performance e la sostenibilità del codice scritto da persone mediante un processo di refactoring?”
- “Qual è l'impatto e la proporzione di queste migliorie?”
- “Come gli LLM riescono a migliorare il codice mediante refactoring?”

Il processo di analisi dei risultati è suddiviso in due sezioni complementari, ognuna volta allo studio di un preciso aspetto della qualità del codice:

Sezione 4.1: Analisi di correttezza stabilisce un criterio fondamentale: l'analisi riguardante le performance ha validità solo se il codice oggetto di tale analisi è corretto, ossia se gli LLM sono in grado di generare un codice che mantenga il comportamento originale, passando le test suite. In questa sezione vengono quindi analizzati i pass rate relativi al regression test dei codici generati dagli LLM tra diversi linguaggi, modelli e versioni del prompt.

Sezione 4.2: Analisi di performance e sostenibilità, incentrata a rispondere alle domande principali di questo studio. In particolare, in merito alle performance, emerge che la fase di ingegnerizzazione del prompt ha un impatto considerevole sui codici generati ed è quindi necessario approfondire questo aspetto analizzando a fondo le differenze tra le versioni del prompt, non limitandosi ad analisi aggregate a riguardo. La Sezione 4.2 si suddivide ulteriormente in una **Comparazione Globale**, che dimostra il fatto che gli LLM siano in grado di produrre codice con performance migliori rispetto all'originale; un'analisi sull'**Impatto del Processo di Ingegnerizzazione del Prompt**, che mette in risalto l'importanza della fase di ingegnerizzazione del prompt giustificando il confronto dei risultati associati alla prima e l'ultima versione del prompt; un'**Analisi per Modello e per Linguaggio**, che compara sistematicamente i risultati della prima e dell'ultima versione del prompt evidenziandone le differenze; e infine l'**Identificazione di Pattern Associati al Refactoring**, che sposta il focus da un'analisi quantitativa ad una qualitativa, tentando di rispondere alla domanda: “Come gli LLM migliorano il codice mediante refactoring sostenibile?”

Queste sezioni forniscono quindi risposte su più livelli alle domande di ricerca poste in questo studio: non solo se e quanto gli LLM possano migliorare il codice, ma anche perché, quali modelli e linguaggi ne traggono i maggiori benefici e come questi miglioramenti vengono introdotti nei codici generati dagli LLM.

4.1 Analisi di Correttezza: Misurazione del Degrado rispetto alla Baseline

Prima di procedere alla valutazione dei miglioramenti associati alle performance occorre analizzare il comportamento dei codici generati dagli LLM riguardo al mantenimento della correttezza. Come descritto e stabilito nella metodologia (Sezioni 3.1, 3.5), tutti i codici di base considerati hanno un pass rate del 100% su tutte e cinque le esecuzioni, questo al fine di incentrare lo studio sulla misurazione di una degradazione funzionale (in termini di correttezza) rispetto alla baseline.

Percentuali inferiori al 100% indicano quindi una regressione introdotta dal processo di refactoring: un fallimento nel mantenere il corretto comportamento deterministico presente nel codice originario,

al fine di comprendere quanto gli LLM siano affidabili nel preservare la perfezione e l'esattezza nelle esecuzioni del codice.

L'intera sezione in questione analizza quindi il pass rate dei codici generati dagli LLM, aggregando linguaggi di programmazione, modelli e versioni del prompt e interpretando i risultati ottenuti come un degrado dalla baseline in cui è accertata e assicurata la perfezione del 100% di pass rate.

Pass Rate Aggregato per Linguaggio: Misura di Regressione dal 100%

Come primo punto dell'analisi di correttezza l'attenzione viene posta nel comprendere se e quanto l'abilità da parte degli LLM di mantenere la correttezza del codice sia dipendente dal linguaggio di programmazione considerato. Ai fini di tale analisi sono stati creati i grafici visibili nelle Figure 4.1 e 4.2, i quali rispettivamente sono volti a mostrare il pass rate aggregato per modello e diversificato per linguaggio rispetto alla baseline e il pass rate per linguaggio disaggregato anche per modello.

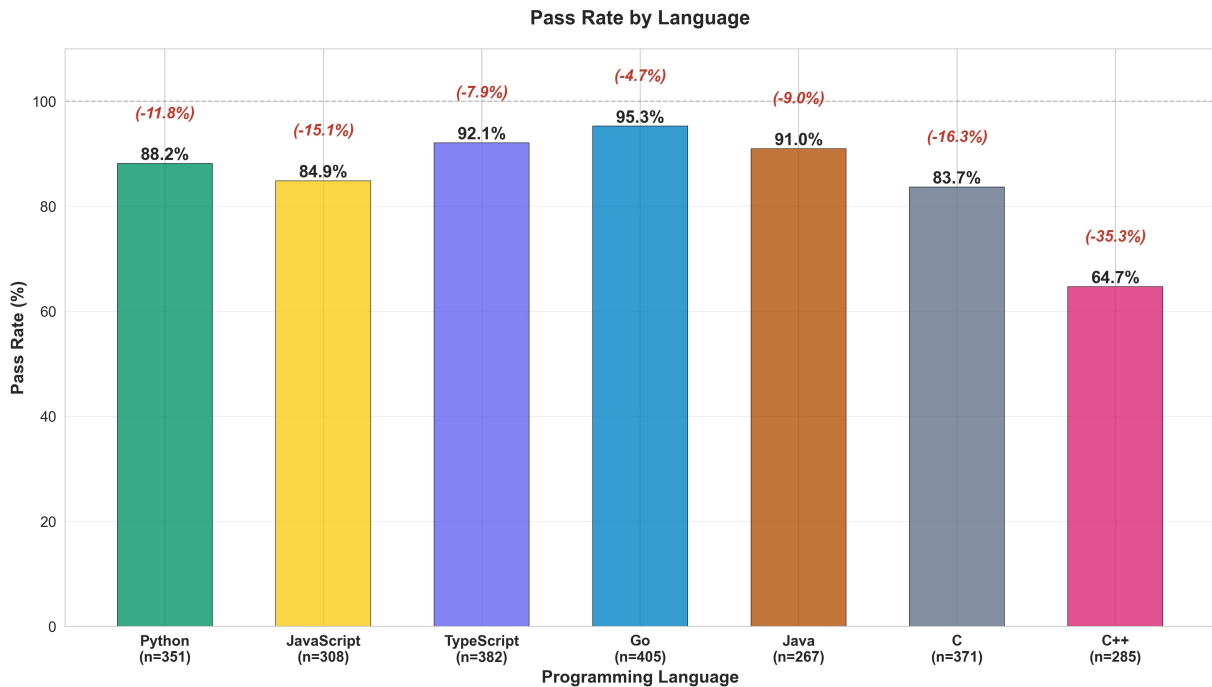


Figura 4.1: Pass Rate (%) per Linguaggio: Degrado di correttezza del codice generato dagli LLM rispetto alla baseline. Il riferimento orizzontale al 100% rappresenta il tasso di correttezza garantito di tutto il codice base. Valori inferiori al 100% quantificano le regressioni funzionali introdotte dal refactoring mediante uso degli LLM. Le dimensioni del campione (n) si riferiscono al numero di voci di codice base, ciascuna associata a 12 varianti generate da LLM (3 modelli $\times$ 4 prompt).

Dai risultati emersi nell'analisi della Figura 4.1 è possibile individuare tre scenari distinti. Nel **best case scenario** (degradazione minimale), Go e TypeScript si sono rivelati i linguaggi con il più alto tasso di pass rate, rispettivamente 95.3% e 92.1%, mostrando degradazioni solamente del -4.7% e -7.9% rispetto alla baseline; seppur non perfetti, i refactoring per questi linguaggi hanno un tasso di errore accettabile ed è quindi possibile considerare l'uso degli LLM per linguaggi fortemente tipati. All'estremo opposto si osserva una **degradazione severa** (alto rischio di incorrettezza): C++ mostra il minor pass rate, raggiungendo solamente il 64.7%; con ben il -35.3% di degrado, C++ si dimostra il linguaggio più complesso per il refactoring da parte degli LLM, che circa una volta su tre falliscono nel mantenere la correttezza del codice originario, presumibilmente alterando le firme di funzione e/o introducendo errori individuabili già a tempo di compilazione. Infine si riscontra una **degradazione moderata** per il resto dei linguaggi: C, Java, Python e JavaScript mostrano degradazioni accettabili, da -16.3% (ossia pass rate di 83.7%) per C fino ad una degradazione del -9% per Java, con performance tollerabili relativamente al pass rate.

È quindi importante osservare come la scelta del linguaggio rappresenti un criterio dominante nella sicurezza e affidabilità del refactoring. L'ampia differenza tra C++ (peggiore) e Go (migliore) di

poco oltre il 30% indica che il grado di correttezza atteso dopo il refactoring via LLM è fortemente dipendente dal linguaggio considerato. Inoltre linguaggi fortemente tipati e di alto livello sembrano avere migliori risultati in termini di pass rate rispetto a linguaggi con altre caratteristiche. Infine, qualsiasi sia il linguaggio considerato, è consigliabile affiancare il processo di refactoring ad un'attenta analisi e validazione umana, in particolar modo per C++.

Pass Rate Aggregato per Linguaggio: Differenze tra Modelli

Mantenendo la disaggregazione tra i linguaggi considerati diviene ora interessante considerare il comportamento dei diversi modelli scelti in relazione proprio ai linguaggi. Al fine di comprendere le differenze tra i modelli, la Figura 4.2 mostra il pass rate considerando sia il linguaggio che lo specifico modello LLM, permettendo in tal modo di individuare i differenti comportamenti dei modelli scelti per lo stesso linguaggio e l'andamento dello stesso modello tra più linguaggi.

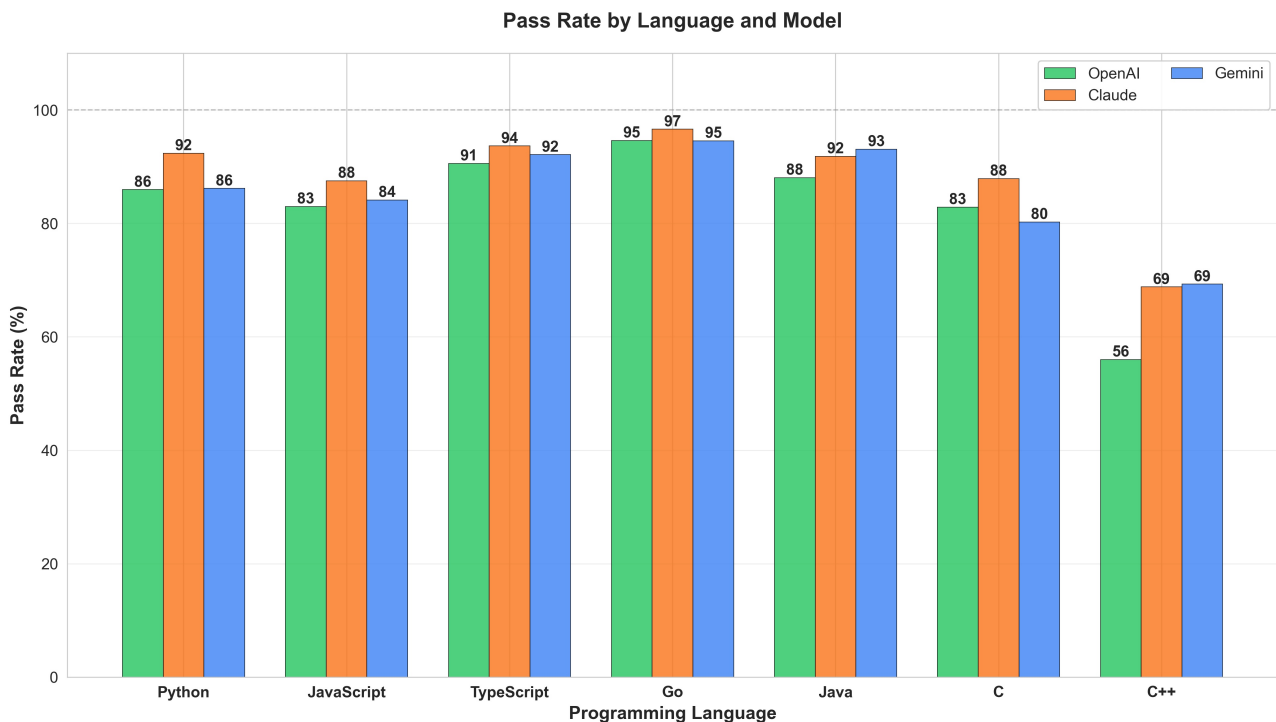


Figura 4.2: Pass Rate (%) per Linguaggio e Modello: Degradazioni rispetto alla baseline (100%). Tutto il codice di base mantiene una correttezza del 100%; valori inferiori rappresentano regressioni di correttezza specifiche della coppia linguaggio-modello.

Quest'analisi è volta a mostrare l'interazione tra i modelli e i linguaggi di programmazione in merito alla capacità degli LLM di preservare la correttezza del codice nel processo di refactoring. Da quanto emerge dalla Figura 4.2 è possibile osservare che C++, oltre al minor pass rate, mostra la maggior dipendenza dal modello: vi sono forti differenze tra il 56% di pass rate ottenuto con OpenAI e il 69% ottenuto con Gemini e Claude, una differenza di ben 13% tra modelli differenti; anche C mostra una significativa diversità tra il pass rate di Gemini (80%) e quello di Claude (88%). Al contrario TypeScript e Go rivelano minime differenze tra i modelli: una variazione massima del 2% per Go e una del 3% per TypeScript dimostrano come, per questi linguaggi, modelli diversi raggiungano risultati simili per il pass rate. Gli altri linguaggi considerati mostrano variazioni minori tra modelli, intorno al 5%.

Quindi da quest'analisi emerge come la capacità degli LLM di mantenere la correttezza del codice originario sia maggiormente legata alla scelta del linguaggio che alla scelta del modello, seppure per alcuni linguaggi (C e C++ in particolare) la scelta di un modello piuttosto che un altro possa risultare in variazioni considerevoli del pass rate.

Pass Rate per Modello e Versione del Prompt

Questa sezione si concentra sul ruolo del processo di ingegnerizzazione del prompt, analizzando il relativo impatto sulla correttezza dei codici generati dagli LLM. Per ottenere un quadro chiaro di tale situazione la Figura 4.3 mostra una vista disaggregata di tutte e dodici le coppie modello-versione del prompt, rendendo possibile la comparazione tra modelli diversi associati alla stessa versione del prompt e tra versioni associate allo stesso modello; confrontando ogni coppia con la baseline.

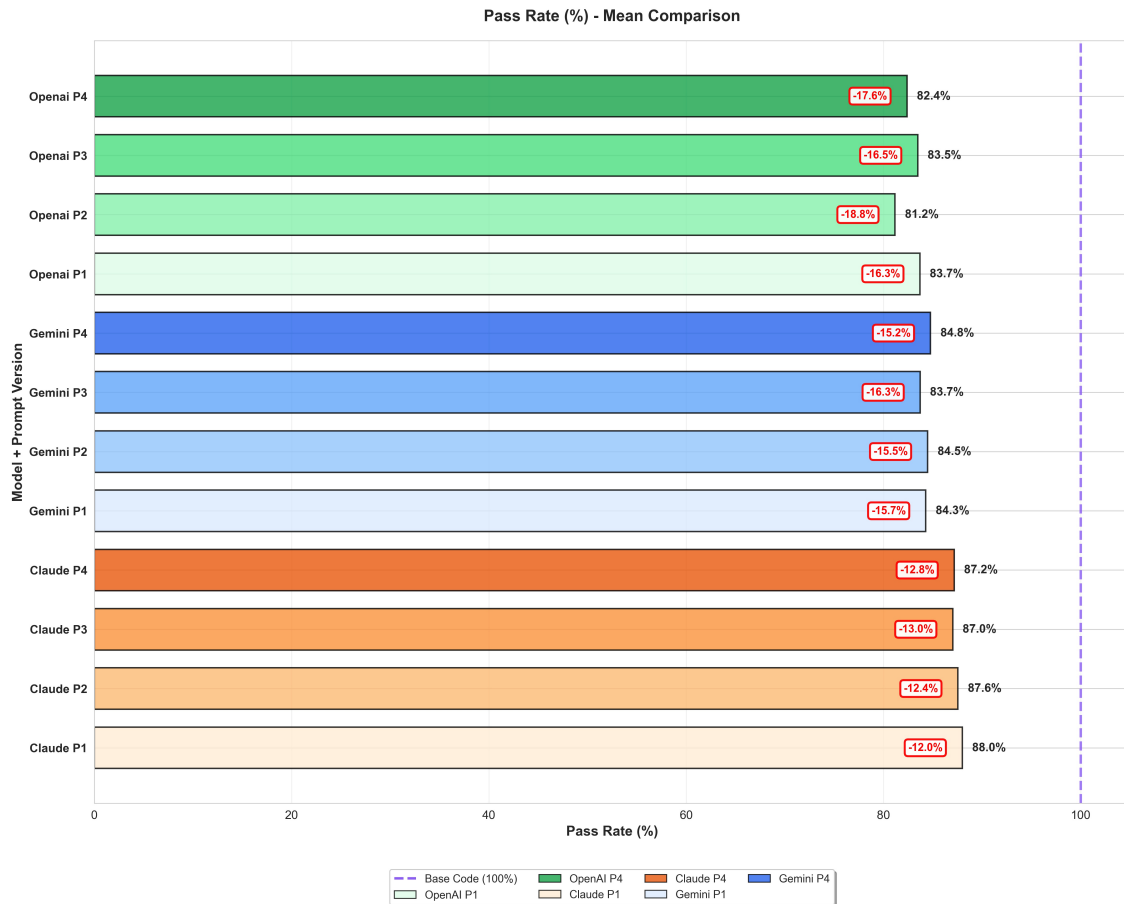


Figura 4.3: Pass Rate (%): Comparazione delle medie per modello e versione del prompt (aggregata su tutti i linguaggi).

Dall'analisi della Figura 4.3 emergono due estremi. Per quanto riguarda la **configurazione con il degrado minimo**, Claude con la prima versione del prompt raggiunge un degrado di correttezza del -12% , ossia un pass rate del 88% , indicando che anche la miglior combinazione di modello e versione del prompt fallisce circa una volta su dieci nel mantenere la correttezza nel processo di refactoring; OpenAI raggiunge il minimo degrado (-16.3%) sempre sfruttando la versione uno del prompt, mentre Gemini beneficia maggiormente dell'ultima versione del prompt, con cui raggiunge un pass rate del 84.8% (degrado del -15.2%). All'opposto, la **configurazione con il degrado massimo** è la coppia composta dal modello OpenAI e dalla seconda versione del prompt, che ottiene il degrado di correttezza più severo raggiungendo il -18.8% rispetto alla baseline (pass rate del 81.2%), ossia circa due codici ogni dieci falliscono nel mantenere la correttezza funzionale a seguito del procedimento di refactoring mediante LLM; Claude ottiene il peggior risultato con la terza versione del prompt (-13% di degrado per un pass rate del 87%), mentre Gemini ha un lieve peggioramento con la versione tre del prompt rispetto alle altre (degrado: -16.3% , pass rate: 83.7%).

Osservazioni Chiave

Attraverso l'analisi mirata in relazione alle versioni del prompt e riguardante la correttezza mantenuta dai codici generati dagli LLM rispetto agli originali è stato possibile individuare le seguenti importanti

scoperte:

- **Nessuna configurazione raggiunge la perfezione:** anche la coppia composta da Claude con la prima versione del prompt, seppure migliore delle altre, non raggiunge il 100% di pass rate, mantenendo un significativo degrado del -12% e rappresentando un upper-bound dell'affidabilità del refactoring via LLM, dimostrando inoltre che con i modelli considerati e i prompt utilizzati non è possibile raggiungere un tasso di correttezza se non identico quantomeno molto vicino alla perfezione della baseline;
- **Il processo di ingegnerizzazione del prompt ha un impatto modesto sulla correttezza:** lo spread tra il miglior (88%) e il peggior (81.2%) pass rate è di ben 6.8 punti percentuale, indicando che il prompt contribuisce discretamente alla correttezza del codice generato, seppure in minor modo rispetto alla scelta del linguaggio (Sezione 4.1).

A seguito di tali scoperte è stato deciso di procedere nelle analisi delle seguenti sezioni comparando sistematicamente le versioni del prompt marginalmente e maggiormente elaborate: la prima versione rispetto all'ultima. Ciò per poter isolare l'impatto del processo di ingegnerizzazione del prompt esaltandone l'importanza.

4.2 Analisi di Performance e Sostenibilità

Avendo stabilito una baseline per la correttezza nella Sezione 4.1, questa sezione è volta a rispondere alla principale domanda postasi in questo studio: "Il refactoring mediante LLM è in grado di migliorare le performance e la sostenibilità del codice?"

Comparazione Globale: Codice di Base vs Codice Generato dagli LLM

L'analisi in questione comincia mostrando l'effetto globale del refactoring via LLM tramite comparazioni volte a rivelare sia trend aggregati che specifici per modello.

Distribuzione del Consumo di Energia

Nella Figura 4.4 è possibile notare il confronto tra il consumo di energia dei codici di base e quello dei codici generati dagli LLM secondo le coppie modello-versione del prompt. L'uso dei box plot è volto ad evidenziare la presenza degli outlier, mentre per ogni coppia viene inoltre riportata la riduzione percentuale rispetto alla media del codice originario.

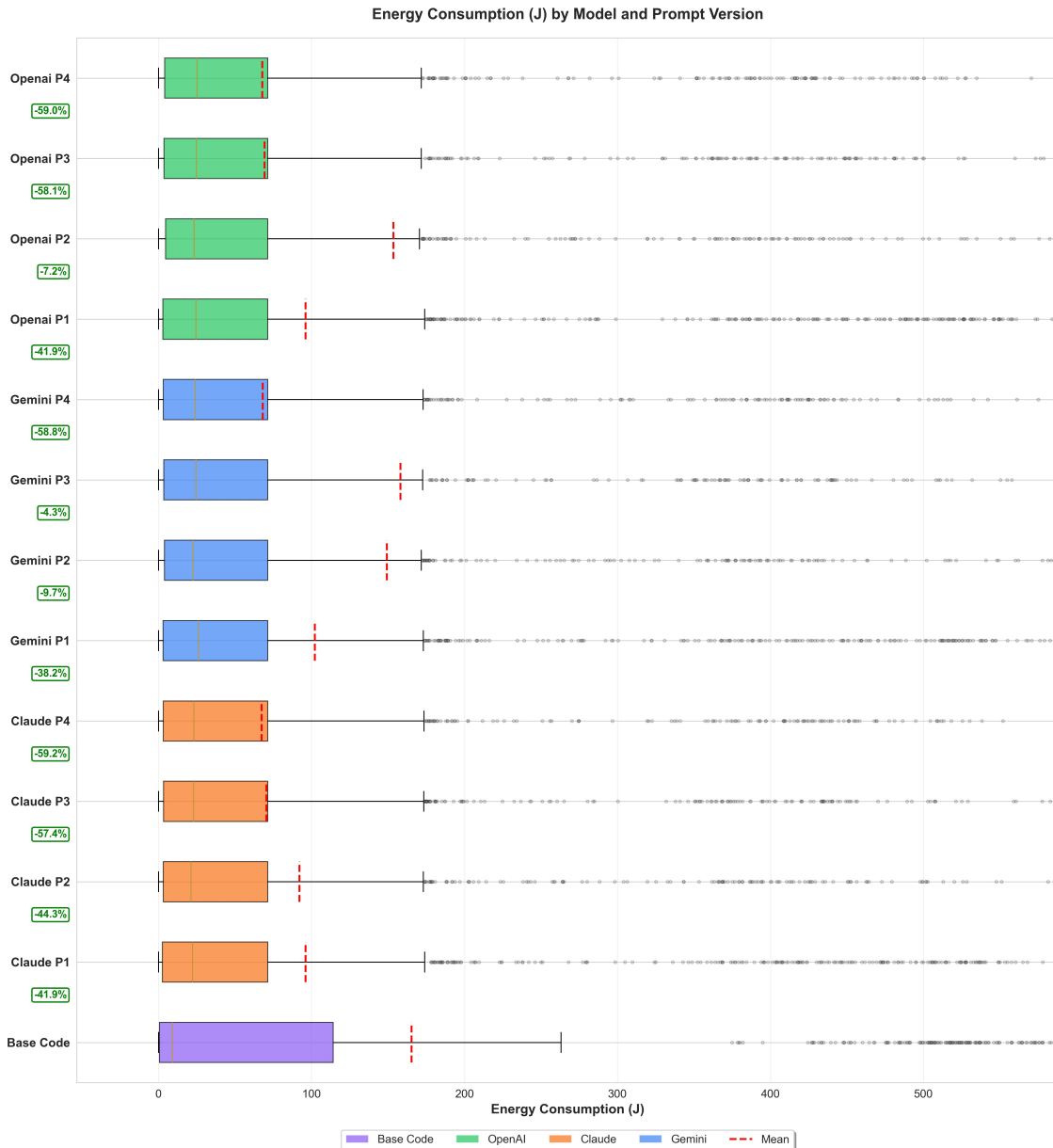


Figura 4.4: Distribuzione del consumo di energia (Joule): Codice di Base vs Codice generato dagli LLM. La figura mostra il codice di base (viola) e le 12 combinazioni di modello e versione del prompt. Le percentuali indicano il miglioramento degli LLM rispetto alla media del codice di base, ossia la riduzione del consumo energetico.

Osservazioni:

- In media gli LLM hanno un miglioramento rispetto ai codici di base, ossia riescono effettivamente a ridurre il consumo energetico: ogni combinazione modello-versione del prompt riesce a ridurre più o meno significativamente il consumo energetico rispetto al codice di base, ma la riduzione è consistente in tutte le combinazioni.
- **Variabilità tra modelli e versioni del prompt:** elencando le 12 possibili combinazioni di modello e versione del prompt è possibile notare una distribuzione abbastanza eterogenea:
 - *Migliori combinazioni:* Claude P4 (−59.2%), OpenAI P4 (−59.0%) e Gemini P4 (−58.1%) raggiungono le maggiori riduzioni per quanto riguarda il consumo energetico rispetto al codice originario.

- *Peggiori combinazioni*: Gemini P3 (−4.3%), OpenAI P2 (−7.2%), e Claude P1 (−41.9%) rappresentano le riduzioni minori, dimostrando che le prime versioni del prompt hanno un impatto indubbiamente minore nel miglioramento energetico rispetto all’ultima, oltre ad essere più dipendenti dal modello; tuttavia riescono comunque a diminuire il consumo energetico.
- *Consistenza nelle riduzioni*: nonostante la varianza tra versioni e modelli diversi, è facilmente osservabile come tutte e 12 le combinazioni riducano più o meno significativamente il consumo energetico rispetto al codice originale.
- **Variabilità**: i box plot e soprattutto la massiccia presenza di outlier rivelano che anche le migliori configurazioni presentano una forte varianza: esistono combinazioni che producono codici con un consumo di energia simile se non maggiore rispetto al codice originario. Ciò sottolinea come il comportamento degli LLM non sia deterministico ed evidenzia l’importanza di un’attenta analisi sui codici generati dagli LLM.

Questa prima visualizzazione delle riduzioni associate al consumo energetico dimostra l’efficacia degli LLM nel diminuire tale consumo mediante il refactoring del codice, suggerendo inoltre che il processo di ingegnerizzazione del prompt abbia un significativo impatto sui risultati finali.

Distribuzione di Frequenza delle Riduzioni

Per comprendere al meglio la distribuzione di frequenza delle riduzioni dei consumi, la Figura 4.5 mostra un istogramma di tali riduzioni percentuali per tutte le principali metriche oggetto di questo studio (uso di CPU, uso di RAM, tempo di esecuzione e consumo di energia).

Le quattro distribuzioni raffigurate nella Figura 4.5 presentano morfologie particolarmente differenti, rivelando come gli effetti del refactoring LLM varino a seconda della metrica da ottimizzare:

- **Uso di CPU**: l’uso della CPU mostra un principale picco intorno alla mediana del −18.57% e un secondo picco minore intorno al −50%. È inoltre presente una lunga coda positiva che indica la presenza, seppure rara, di casi in cui il refactoring introduce degradi fino al +200% (ossia in cui l’uso di CPU aumenta notevolmente nei codici generati dagli LLM).
- **Uso di RAM**: è presente una distribuzione bi-modale che concentra i risultati intorno a due principali picchi, uno dominante attorno allo 0% e l’altro attorno al −60% circa. La mediana del −0.97% conferma che la maggior parte dei codici generati dagli LLM non beneficia particolarmente del refactoring per quanto riguarda l’uso della RAM; tuttavia vi sono anche alcune entries per cui vi è una sostanziale riduzione di questa metrica.
- **Tempo di esecuzione**: la distribuzione di frequenze è più contenuta e suggerirebbe quasi una Gaussiana attorno allo 0 se non fosse per un picco verso il −80% circa. Ciò indica che spesso gli LLM riescono a ridurre significativamente l’overhead associato al tempo di esecuzione, diminuendo in modo consistente questa metrica. Una mediana del −22.54% indica che nella maggior parte dei casi i codici generati vengono eseguiti particolarmente più velocemente degli originali. Infine una coda positiva mostra come, in maniera simile all’uso di CPU, anche per il tempo di esecuzione siano stati generati codici particolarmente inefficienti se comparati alle relative entries di base.
- **Consumo energetico**: dall’analisi delle frequenze relativamente alle riduzioni del consumo energetico emerge subito che la mediana sia la più ampia in valore assoluto tra tutte le metriche: per oltre il 50% dei casi gli LLM riescono a generare codici che hanno una riduzione nel consumo energetico di almeno il −38.40% rispetto all’originale, un significativo risultato per il refactoring sostenibile del codice. È inoltre presente un picco di estrema efficienza attorno al −90%, il che dimostra l’abilità degli LLM nel generare codice particolarmente efficiente in termini energetici rispetto all’originale. Anche per il consumo di energia è presente una coda a destra rispetto allo 0 che indica la presenza di codici con un consumo maggiore rispetto alle entries di base.

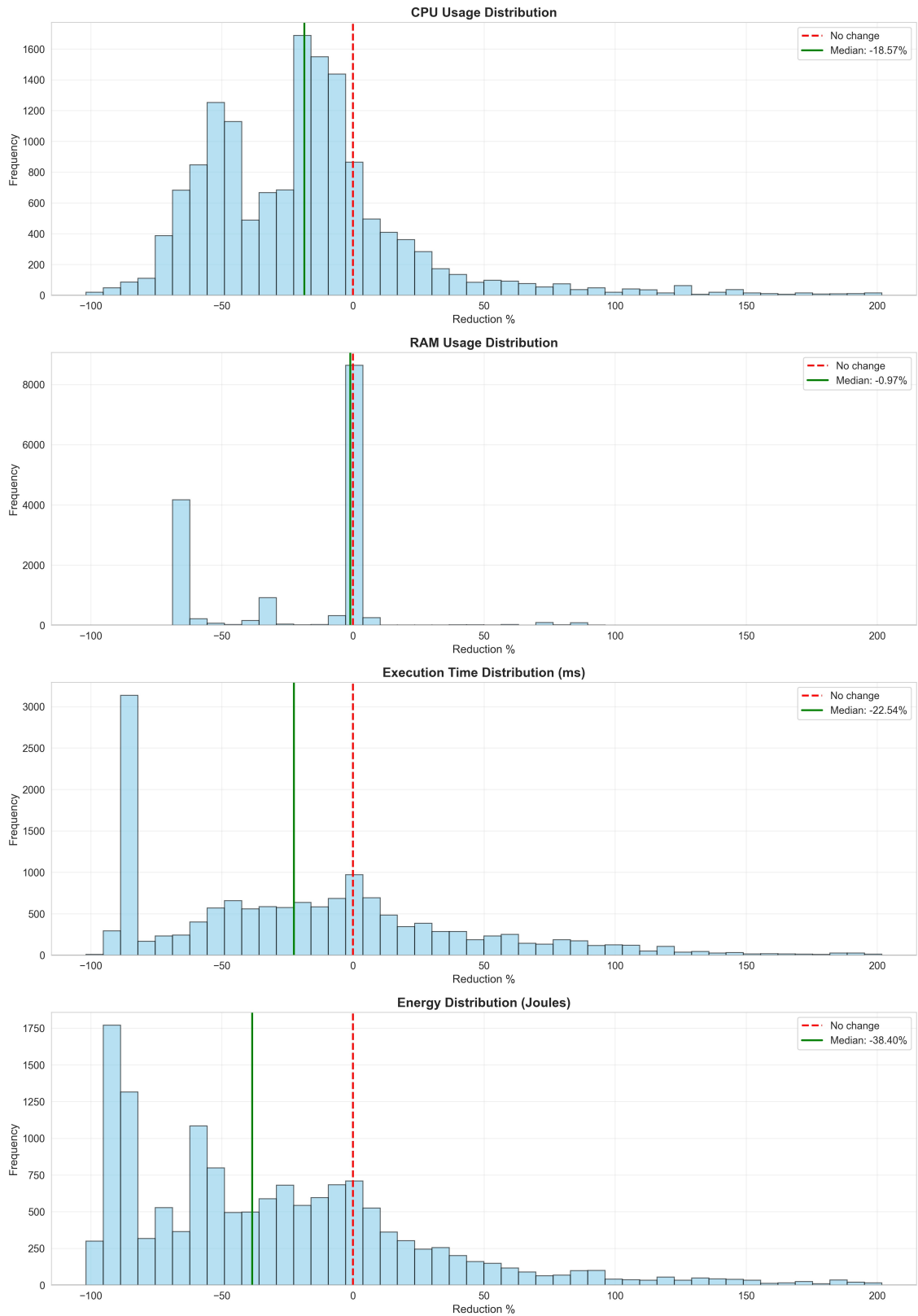


Figura 4.5: Distribuzione di frequenza delle riduzioni percentuali per uso di CPU, uso di RAM, tempo di esecuzione e consumo di energia (aggregati tra modelli, linguaggi e versioni del prompt). Una riduzione di $-x\%$ indica che il codice generato richiede $x\%$ in meno della metrica considerata rispetto all'originario, quindi rappresenta un miglioramento.

Trade-off: Correttezza vs Performance

Infine la Figura 4.6 raffigura le 12 combinazioni modello-versione del prompt su due dimensioni: pass rate (asse y) e riduzione energetica percentuale (asse x).

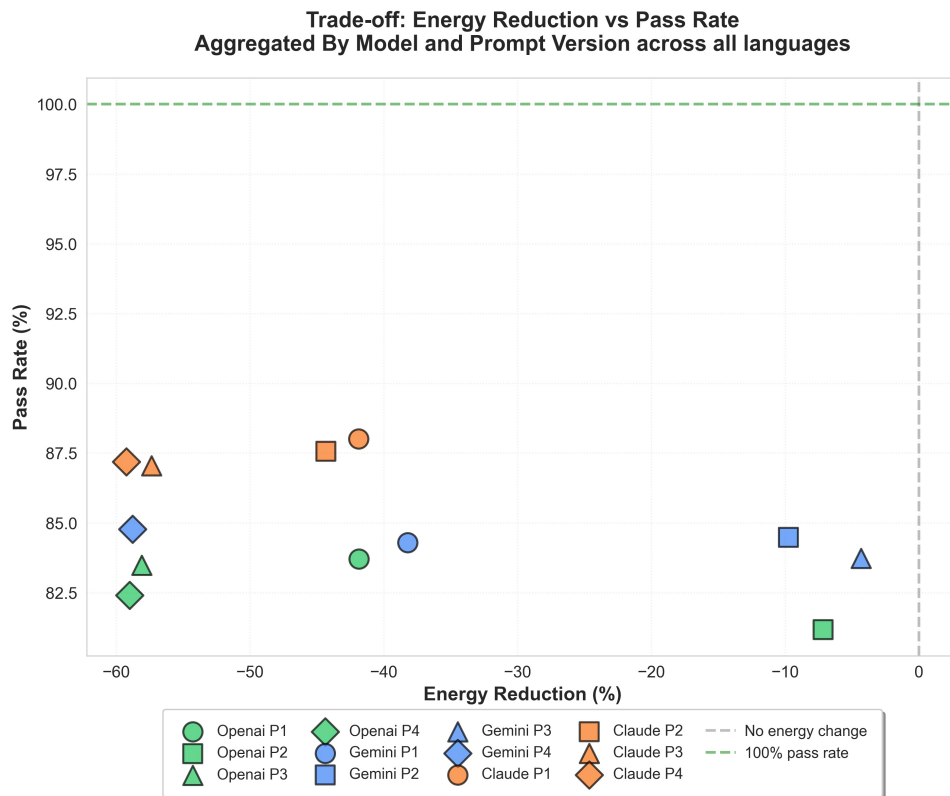


Figura 4.6: Trade-off: riduzione media (%) del consumo di energia vs. Pass rate medio (%) per tutte le combinazioni modello-versione del prompt.

Lo scatter plot raffigurato nella Figura 4.6 rivela una lieve correlazione tra la riduzione associata al consumo energetico e il pass rate, mostrando che a maggiori riduzioni corrispondono lievi aumenti del pass rate per i modelli e versioni del prompt considerate. Ciò suggerisce che vi sia una bassa dipendenza tra pass rate e riduzione energetica. A riguardo è importante osservare che:

- **Configurazioni migliori:** un modello su tutti sembra avere i migliori risultati: Claude è costantemente al di sopra del trend medio con tutte le versioni del prompt, dimostrando di essere il modello più indicato per un trade-off minimo tra pass rate e miglioramento nel consumo energetico. In particolare le versioni 1 e 2 del prompt con Claude raggiungono il maggior pass rate (vicino al 90%), mentre la versione 4 ottiene la maggior riduzione del consumo energetico (nonché il maggior miglioramento), circa del -60% rispetto al codice originale.
- **Andamento delle versioni del prompt:** è possibile notare come versioni del prompt in cui è stata posta maggior attenzione nel processo di ingegnerizzazione (P3, P4) ottengano sistematicamente una riduzione maggiore dell'energia utilizzata per l'esecuzione dei codici generati dagli LLM, talvolta a discapito del pass rate in cui, considerando modello per modello, la prima versione del prompt sembra avere risultati migliori rispetto alle altre. L'unica eccezione a questo trend è Gemini P3 che, nonostante la versione del prompt, ottiene la minor riduzione energetica (solo il -5% rispetto al codice originale).
- **Comparazione tra modelli:** OpenAI è indubbiamente il modello con le performance minori in questa analisi, raggiungendo il minor pass rate (inferiore al 82%) con la versione 2 del prompt e ottenendo costantemente risultati peggiori rispetto agli altri modelli per quanto riguarda questa metrica. Comparando quindi i modelli a parità di versione del prompt Claude risulta essere il migliore e OpenAI il peggiore, eccezione fatta per Gemini P3.

Da questa analisi emerge quindi che la scelta del modello può risultare un fattore importante per considerare il trade-off tra pass rate e riduzione energetica; tuttavia anche il processo di ingegnerizzazione

del prompt ha un ruolo chiave, specialmente nella riduzione energetica. Infine, seppure lieve, è presente una correlazione tra pass rate e riduzione energetica, la quale indica che coppie modello-versione del prompt con risultati migliori su una delle due dimensioni spesso hanno buoni o ottimi valori anche per l'altra.

Impatto del Processo di Ingegnerizzazione del Prompt

Come spesso precedentemente accennato e come è stato notato analizzando la Figura 4.6, la scelta del prompt e il processo volto al suo miglioramento e affinamento (descritto nella Sezione 3.3) è un fattore cruciale per la qualità del refactoring e la sua efficacia. Le seguenti sotto-sezioni sono volte ad evidenziare l'importanza del prompt nel processo di refactoring sostenibile mediante LLM, comparando le diverse versioni per ogni metrica considerata e sottolineando le differenze nei risultati.

La Versione del Prompt come Variabile Dominante nello Studio

La Figura 4.7 disaggrega le distribuzioni delle riduzioni percentuali secondo la versione del prompt, aggregando i tre modelli considerati e i sette linguaggi scelti.

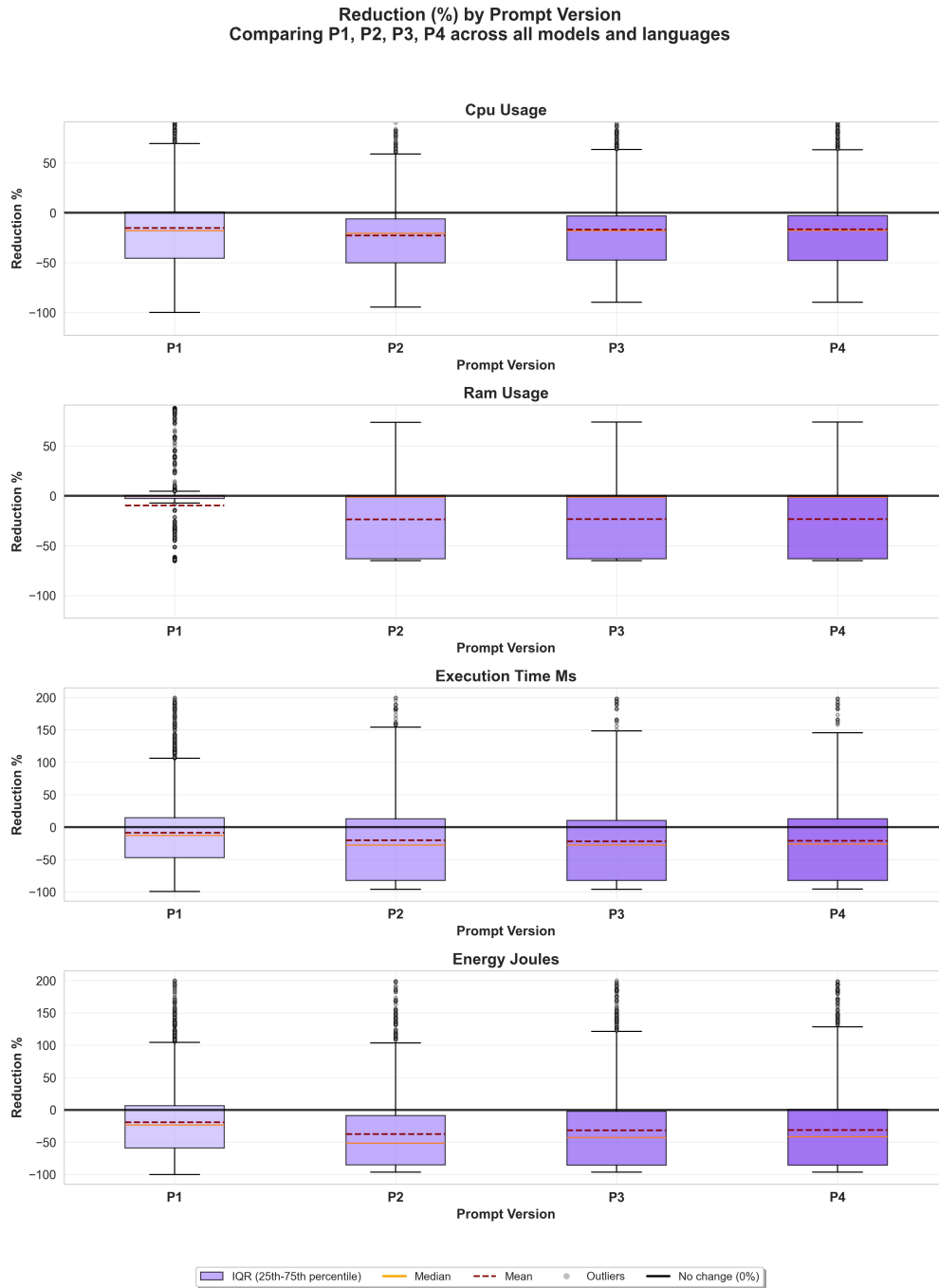


Figura 4.7: Distribuzione delle riduzioni percentuali per versione del prompt (aggregazione per linguaggi e modelli).

Dall'analisi della Figura 4.7 emerge un pattern ricorrente associato alle riduzioni percentuali per ogni metrica relativamente alle diverse versioni del prompt. Il **Prompt P1** (baseline) è sistematicamente il peggiore per ogni metrica, avendo la mediana superiore alle altre versioni in ogni box plot; inoltre ha una porzione del quartile al di sopra dello zero (in particolare per il tempo di esecuzione e la riduzione associata al consumo di energia), mostrando un leggero degrado in alcuni casi, ed è la versione che presenta la maggior quantità di outlier, indicando un'elevata varianza. Il **Prompt P2** mostra un importante e positivo cambiamento rispetto alla precedente, riuscendo non solo ad ottenere risultati

sistematicamente migliori rispetto alla prima versione, ma anche a raggiungere valori inferiori a tutte le altre versioni. Infine i **Prompt P3 e P4**, seppure rappresentino le versioni ultime del prompt, hanno performance simili rispetto alla seconda versione ma lievemente minori, mostrando così che nonostante la maggior attenzione nel processo di ingegnerizzazione queste due versioni non abbiano raggiunto riduzioni percentuali significativamente migliori.

Prompt	Energia (%)	Tempo Esec. (%)	RAM (%)	CPU (%)
P1	Basso	Basso	Basso	Basso
P2	Alto	Alto	Alto	Alto
P3	Medio-Alto	Medio-Alto	Medio-Alto	Medio-Alto
P4	Medio-Alto	Medio-Alto	Medio-Alto	Medio-Alto

Tabella 4.1: Riduzione % mediana per versione del prompt.

Dall’analisi e dalla forte differenza tra la versione 1 e 2 del prompt è facilmente comprensibile come il ruolo del design del prompt e relativo processo di ingegnerizzazione siano cruciali per l’efficacia del refactoring mediante LLM.

Interazioni tra Modello e Versione del Prompt – Analisi via Heatmap

Per comprendere ulteriormente e analizzare a fondo l'effetto del processo di ingegnerizzazione del prompt, la Figura 4.8 mostra una heatmap delle medie delle riduzioni percentuali associate ad ogni metrica per le 12 combinazioni modello-versione del prompt.

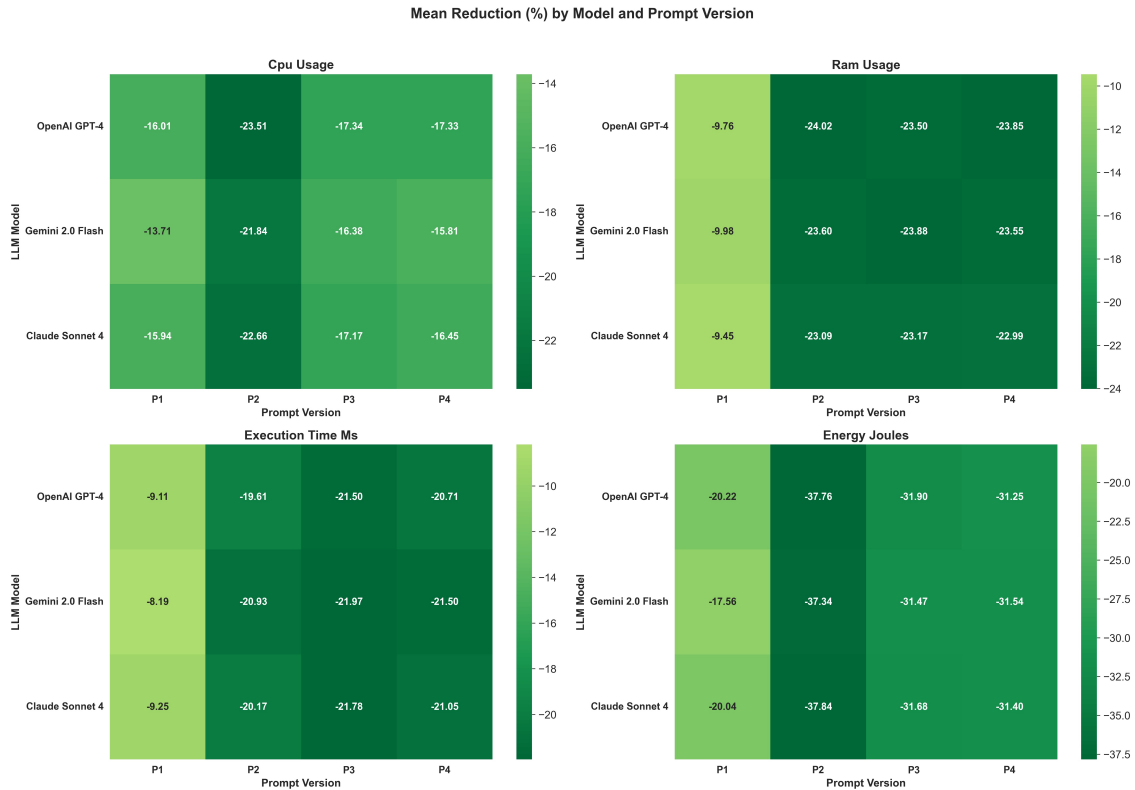


Figura 4.8: Heatmap delle medie delle riduzioni percentuali per ogni metrica, aggregazione per modello e versione del prompt.

L'heatmap conferma quanto già individuato mediante l'analisi dei boxplot, rivelando una forte correlazione tra la versione del prompt e le riduzioni per la metrica considerata. È importante osservare che l'heatmap in questione mostra le riduzioni relative e non le performance assolute.

Dall'analisi della Figura 4.8 è possibile comprendere come le **performance di P1 siano globalmente peggiori**: come già osservato dallo studio dei boxplot, la prima versione del prompt ha risultati sistematicamente inferiori rispetto alle altre versioni indipendentemente dal modello considerato. La **versione migliore** si conferma essere la seconda (P2), che ottiene miglioramenti (riduzioni) sostanziali e maggiori rispetto alle altre versioni, in particolare per l'uso della CPU e la riduzione del consumo di energia; questa versione raggiunge quindi la maggior riduzione relativa e comporta la variazione più elevata tra versioni del prompt successive, rappresentando un significativo cambiamento rispetto alla prima versione del prompt. Le ultime due versioni del prompt (P3, P4) mantengono invece un **comportamento stabile** e simile a P2, seppure con riduzioni relative leggermente minori; un'interessante eccezione è visibile riguardo al tempo di esecuzione, in cui P3 ha riduzioni lievemente maggiori rispetto alle altre.

Prima di procedere oltre è tuttavia utile notare che i risultati di quest'analisi sono associati a riduzioni relative, ossia anche se la seconda versione del prompt sembra avere riduzioni relative maggiori rispetto alle altre è da tenere in considerazione che la versione 4 ha performance assolute migliori. Ciò è particolarmente visibile dalla precedente analisi sui trade-off e dalle seguenti analisi specifiche sulle versioni del prompt.

Implementazione Strategica: Isolamento della Variabile Associata alla Versione del Prompt

Ciò che è stato osservato in questa sezione determina l'importanza e la dominanza della versione del prompt come variabile sperimentale, dato che influenza maggiormente la qualità dei risultati rispetto alla scelta del modello.

Questo aspetto ha contribuito notevolmente alla scelta della seguente analisi: aggregare assieme le versioni del prompt offusca e nasconde le effettive capacità di ogni modello nel refactoring sostenibile dei codici.

Inglobando P1 del prompt otterremmo costantemente performance sottostimate, mentre considerando unicamente P4 ci concentreremmo solo sul miglior scenario, non tenendo in conto l'importanza del processo di ingegnerizzazione del prompt. Per tali motivazioni i seguenti approfondimenti (le analisi per modello e per linguaggio) si concentrano su un'analisi sistematica e comparativa tra la peggiore (P1) e la migliore (P4) versione del prompt, ciò ai fini di:

1. Quantificare l'intero intervallo di varianza associato all'impatto dell'ingegnerizzazione del prompt per ogni modello/linguaggio
2. Isolare le capacità intrinseche di ogni modello mantenendo costante la qualità del prompt e considerando unicamente gli estremi
3. Rivelare ottimizzazioni specifiche per linguaggio, meno affette dal prompt utilizzato

Analisi per Modello: Comparazione di Performance P1 vs. P4

Questa sezione è volta all'analisi delle performance valutando specificamente ogni modello considerato e comparando i due estremi: la prima versione del prompt (worst case) contro la quarta (best case). Ciò isola le capacità intrinseche di ogni modello evidenziando le differenze di qualità associate alla versione del prompt utilizzata.

Consumo di Energia: Comparazione per Modello

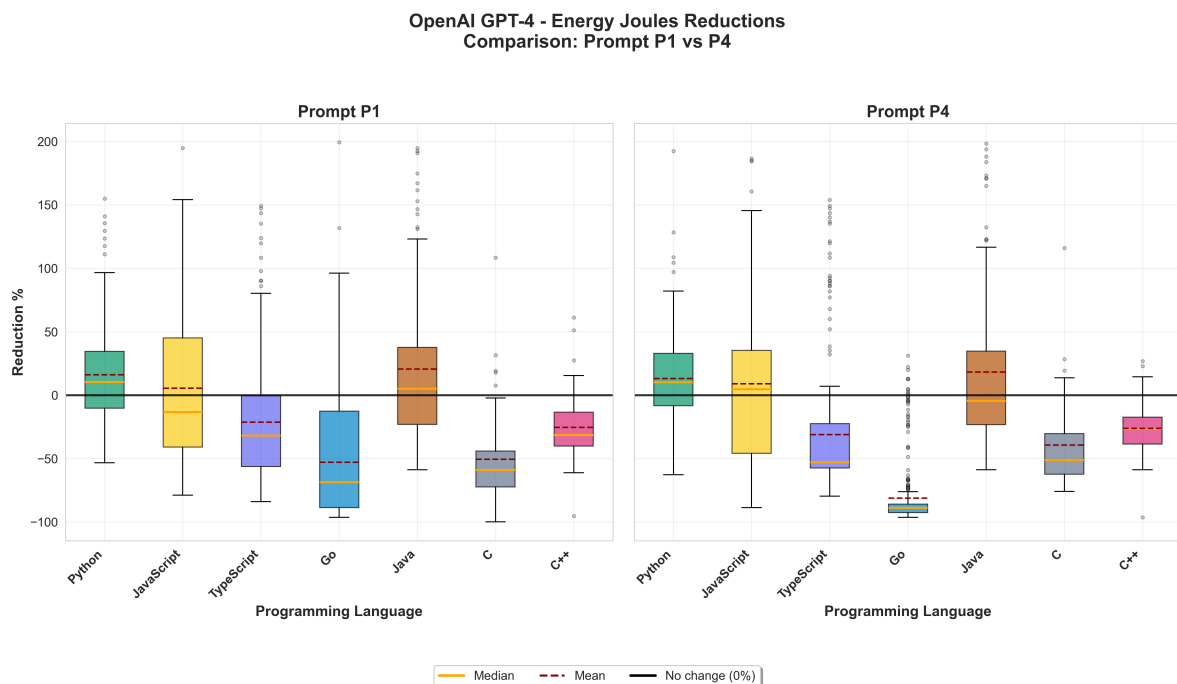


Figura 4.9: Distribuzione delle riduzioni energetiche percentuali: OpenAI P1 vs. P4 (aggregato per linguaggio).

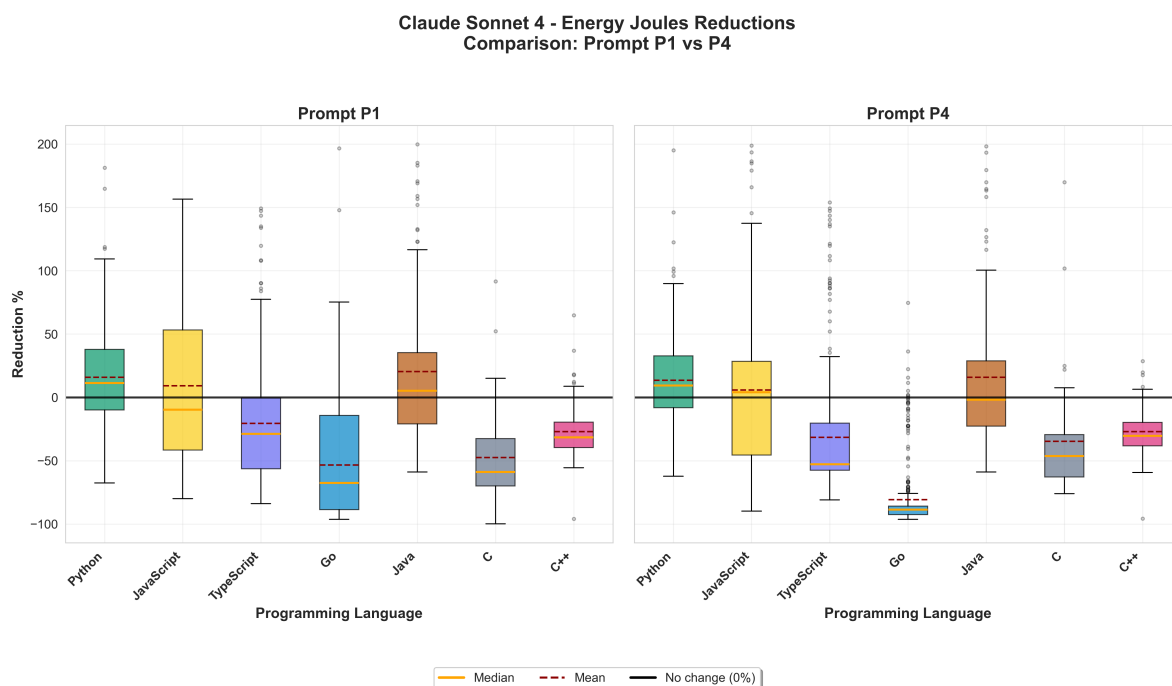


Figura 4.10: Distribuzione delle riduzioni energetiche percentuali: Claude P1 vs. P4 (aggregato per linguaggio).

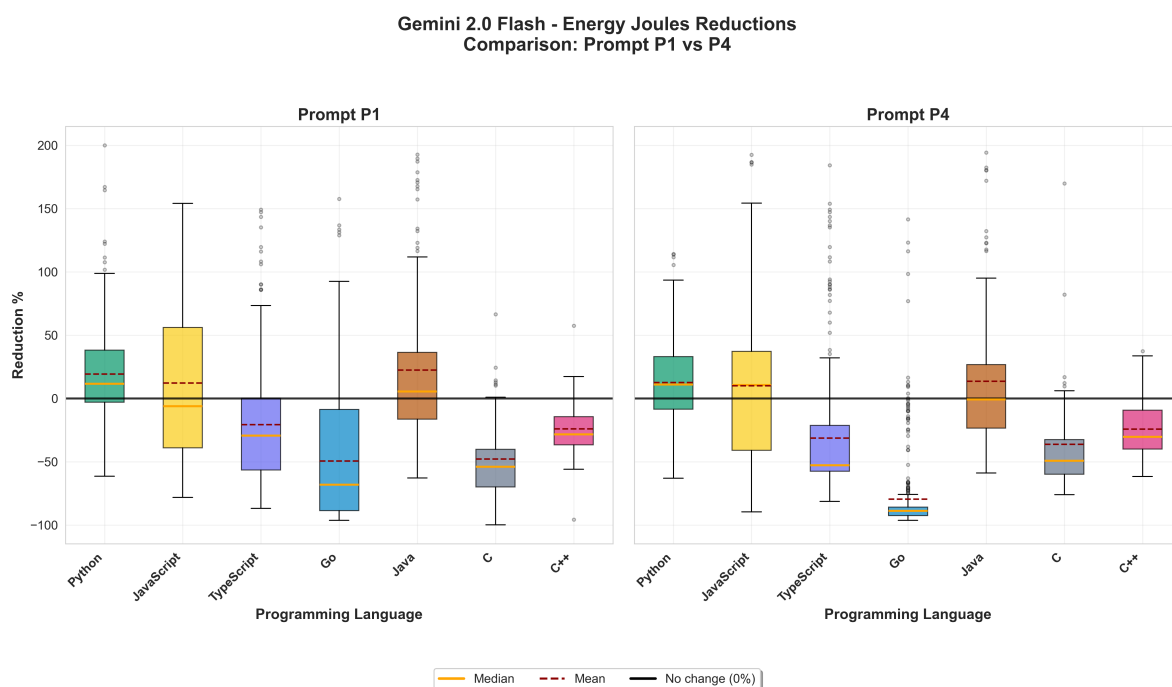


Figura 4.11: Distribuzione delle riduzioni energetiche percentuali: Gemini P1 vs. P4 (aggregato per linguaggio).

L'analisi della distribuzione delle riduzioni relative al consumo energetico rivela un comportamento eterogeneo e fortemente legato al linguaggio di programmazione. Anche le differenze associate alle versioni del prompt mostrano risultati saldamente annessi al linguaggio considerato. Si osserva una **considerevole ottimizzazione per Go e TypeScript**, che sono i linguaggi che beneficiano maggiormente del processo di ingegnerizzazione intercorso tra la prima e l'ultima versione del prompt; il comportamento è simile per ogni modello considerato: Go aumenta la riduzione media da circa il -50% (P1) a circa il -75% (P4), tuttavia presenta una quantità maggiore di outlier con l'ultima ver-

sione del prompt, mentre TypeScript ha una riduzione media più lieve (da circa il -20% al -25%) ma migliora particolarmente la mediana, che passa da circa -25% ad oltre il -50% (per tutti i modelli). Si riscontra invece una **stagnazione o leggero peggioramento** per linguaggi come C e C++, che presentano variazioni minime sia tra modelli diversi che tra le due versioni del prompt considerate, mentre JavaScript presenta una leggera diminuzione della riduzione percentuale portando la mediana nei pressi della media nella versione 4 del prompt, a differenza di P1 in cui la mediana si trovava costantemente al di sotto dello 0 per ogni modello. Infine si notano **variazioni lievi** per Python, che presenta una lieve diminuzione della riduzione percentuale dalla versione 1 alla versione 4, e per Java, che beneficia del processo di ingegnerizzazione del prompt; in particolare è interessante notare come Java sia il principale linguaggio per cui vi sono differenze anche tra modelli diversi, nello specifico OpenAI ha una riduzione più elevata con la versione 4 del prompt rispetto agli altri modelli.

Quest'analisi evidenzia come le differenze tra le versioni del prompt introducano un trade-off dipendente dal linguaggio, in cui linguaggi fortemente tipati come TypeScript e Go beneficiano maggiormente del processo di ingegnerizzazione del prompt rispetto a linguaggi debolmente tipati come JavaScript (che addirittura presentano lievi peggioramenti).

Uso di CPU, Uso di RAM e Tempo di Esecuzione: Comparazione per Modello

Seppur il consumo energetico sia una metrica che aggrega le altre e possa essere considerato come il perfetto indicatore della sostenibilità del codice, è utile e informativo analizzare anche le altre metriche prese in considerazione in questo studio per comprendere nel dettaglio come ogni modello LLM migliora le performance del codice originario attraverso il refactoring.

A causa di limiti di spazio solamente i grafici relativi alle riduzioni energetiche sono presentati sopra, ma l'analisi volta all'individuazione di pattern ricorrenti nelle riduzioni percentuali per ogni metrica (i cui grafici sono mostrati nell'Appendice C) rivela come anche le metriche quali uso di RAM, uso di CPU e tempo di esecuzione presentino comportamenti dipendenti dal linguaggio, già osservati e descritti in relazione alle riduzioni energetiche:

- **Ottimizzazioni nell'uso della CPU:** il confronto tra modelli diversi non mostra particolari differenze, mentre considerando i linguaggi emerge il seguente comportamento eterogeneo: TypeScript e Go si confermano essere i linguaggi che traggono maggior beneficio dal passaggio tra la prima e l'ultima versione del prompt; C, C++ invece diminuiscono le riduzioni associate all'uso della CPU, passando da valori negativi a positivi nel caso di C++; gli altri linguaggi considerati mostrano una stabilità maggiore tra le due versioni del prompt considerate.
- **Ottimizzazioni nell'uso della RAM:** la RAM è la metrica maggiormente affetta dalle differenze tra i linguaggi: mentre è ben chiaro come per Go e TypeScript vi siano miglioramenti netti tra la prima e la quarta versione del prompt, per gli altri linguaggi la riduzione rispetto all'uso della RAM rimane attorno allo 0; fa eccezione Java per cui si osserva un leggero peggioramento, passando da circa 0 a circa $+5\%$ (degrado) tra la prima e l'ultima versione del prompt.
- **Ottimizzazioni del tempo di esecuzione:** segue strettamente quanto osservato in relazione alle riduzioni riguardo al consumo energetico in quanto il tempo di esecuzione è un fattore chiave nella formula del calcolo dell'energia che impiega un codice per la sua esecuzione. TypeScript e Go mantengono il gap maggiore tra P1 e P4, mentre JavaScript peggiora nella versione del prompt più avanzata. A differenza delle altre metriche C, C++ e Python presentano lievi miglioramenti tra le due versioni del prompt considerate (nella versione 4 rispetto a P1), mentre Java tende a peggiorare la media delle riduzioni con l'ultima versione del prompt.

Pass Rate: Comparazione per Modello

Le Figure 4.12, 4.13 e 4.14 mostrano la comparazione del pass rate per linguaggio tra la versione 1 e la 4 del prompt, per ogni modello.

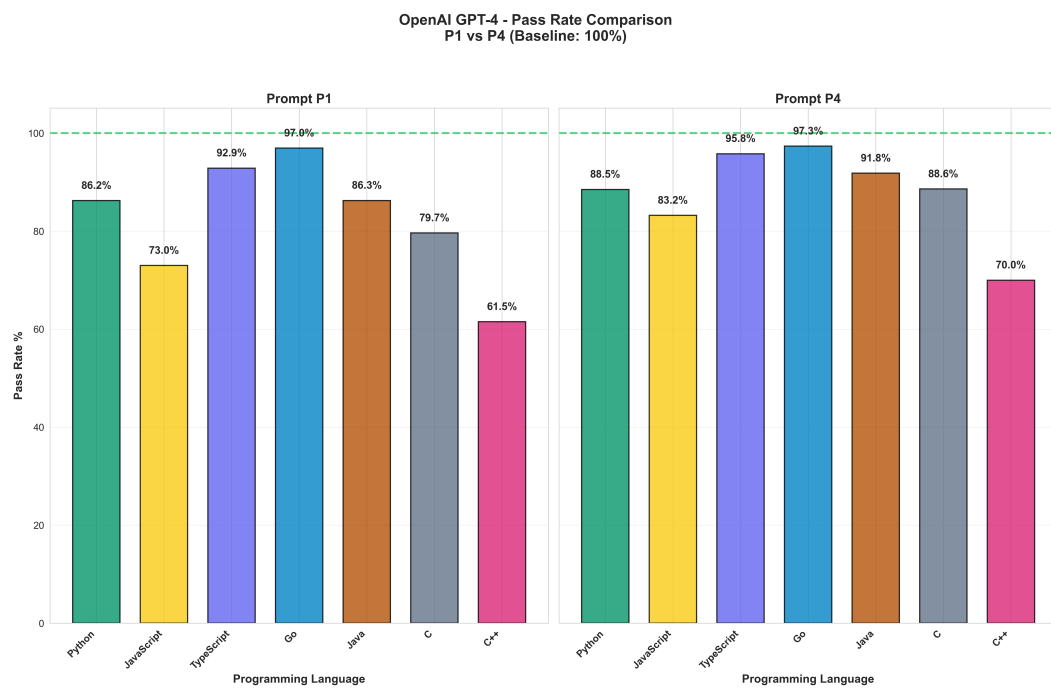


Figura 4.12: Pass Rate (%) per Linguaggio: OpenAI, prompt P1 vs. P4.

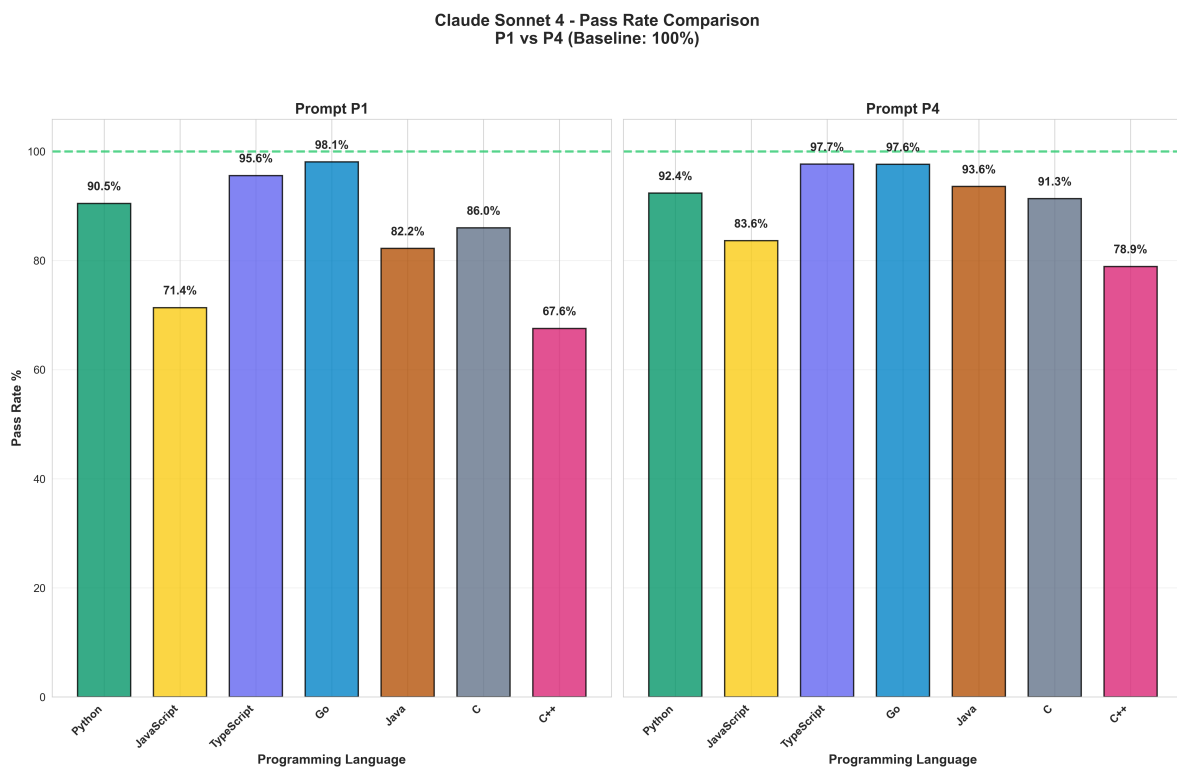


Figura 4.13: Pass Rate (%) per Linguaggio: Claude, prompt P1 vs. P4.

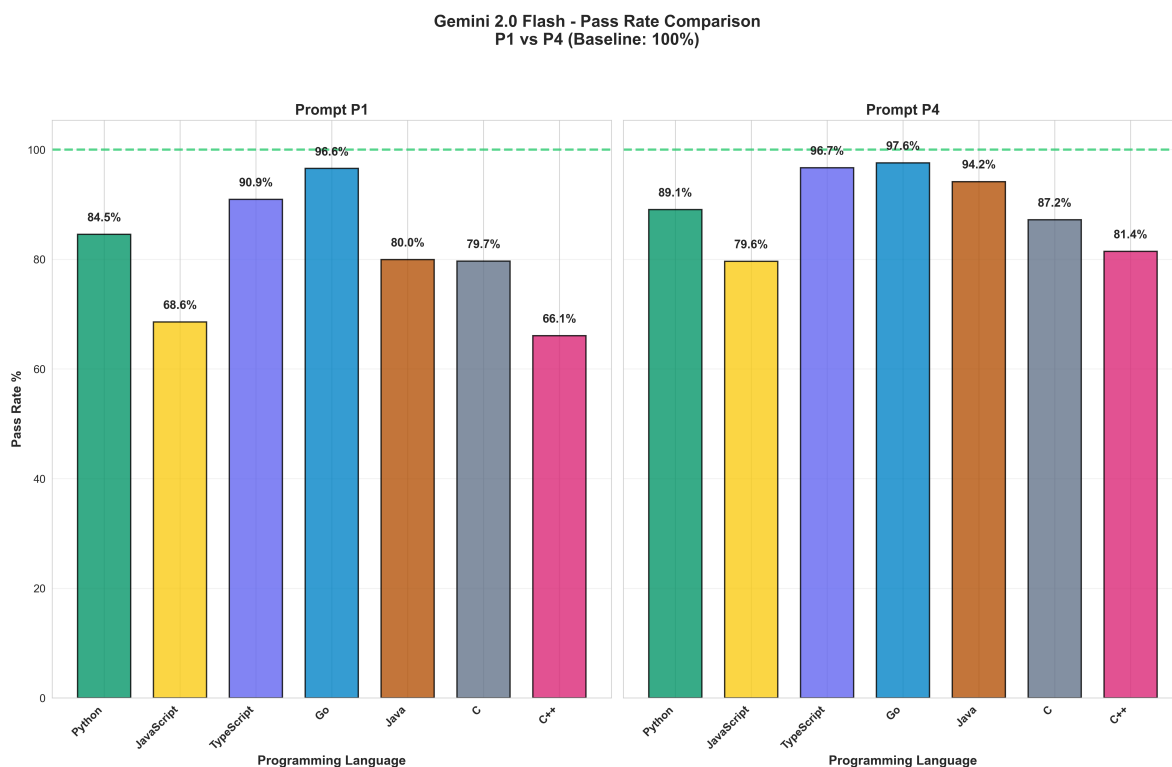


Figura 4.14: Pass Rate (%) per Linguaggio: Gemini, prompt P1 vs. P4.

Questi grafici a barre mostrano il pass rate (e il relativo degrado dalla baseline) assoluto raggiunto da ogni modello in relazione ai linguaggi, comparando la prima e l'ultima versione del prompt. Il degrado percentuale contenuto nei riquadri rossi mostra la differenza rispetto alla perfezione associata alla baseline (codice originale).

Dall'analisi di tali grafici emerge come il pass rate, in modo simile alle altre metriche, dipenda dal linguaggio considerato e anche le differenze tra le versioni del prompt considerate variano tra i diversi linguaggi. Si riscontrano **aumenti significativi** per linguaggi di basso livello come C++ e C, che beneficiano particolarmente dell'ingegnerizzazione del prompt passando rispettivamente da circa il 79/86% e 61/66% al 87/91% e 70/81%, mostrando inoltre differenze legate anche ai modelli (dove Claude ha performance migliori); anche Java (da circa 80-86% fino a 91-94%) e JavaScript (da circa il 70% fino a circa l'83%) denotano elevati aumenti del pass rate nella versione 4 del prompt rispetto alla 1. Si osservano invece **aumenti lievi** per Go e Python, che migliorano lievemente tra la prima e l'ultima versione del prompt per ogni modello, ad eccezione di Go con Claude che peggiora di 0.5%; anche TypeScript migliora in modo consistente per ogni modello. Infine emergono **differenze tra modelli**: diversamente da quanto osservato per le altre metriche, il pass rate ha una varianza maggiore considerando più modelli (ad esempio OpenAI P4 ottiene un pass rate del 70% per C++, mentre Gemini P4 raggiunge 81.4% di pass rate per lo stesso linguaggio).

Classificazione per Modello, Considerando la Versione 4 del Prompt

Sintetizzando la correttezza:

Modello	Pass Rate (P4, media)	Miglior Qualità (P4)
Claude	90.7%	Maggior equilibrio complessivo (picco per TypeScript al 97.7%)
OpenAI	87.9%	Ottimo su TypeScript/Go, basso su C++
Gemini	89.4%	Migliore su C++ (81.4%) e Java (94.2%), peggiore su JavaScript (79.6%)

Tabella 4.2: Comparazione del pass rate per modello (Prompt P4, media su 7 linguaggi).

- **Claude** emerge come il modello più bilanciato per la versione 4 del prompt, ottenendo i risultati migliori con una media del pass rate del 90.7%
- **OpenAI** mostra buone performance complessive, seppure le più basse rispetto agli altri modelli (87.9%), con particolari problematiche relative a C++ per cui raggiunge solo il 70% di pass rate.
- **Gemini** dimostra ottime capacità per C++ nonostante sia il peggiore su C (87.2%), ottenendo un pass rate complessivo di 89.4%.

Analisi per Linguaggio di Programmazione

In questa sezione vengono aggregati modelli e versioni del prompt per poter apprezzare le differenze presenti tra i linguaggi di programmazione considerati in merito alle riduzioni per ogni metrica. La Figura 4.15 mostra la distribuzione delle riduzioni percentuali mediante box plot per ogni metrica, disaggregando solo per linguaggio.

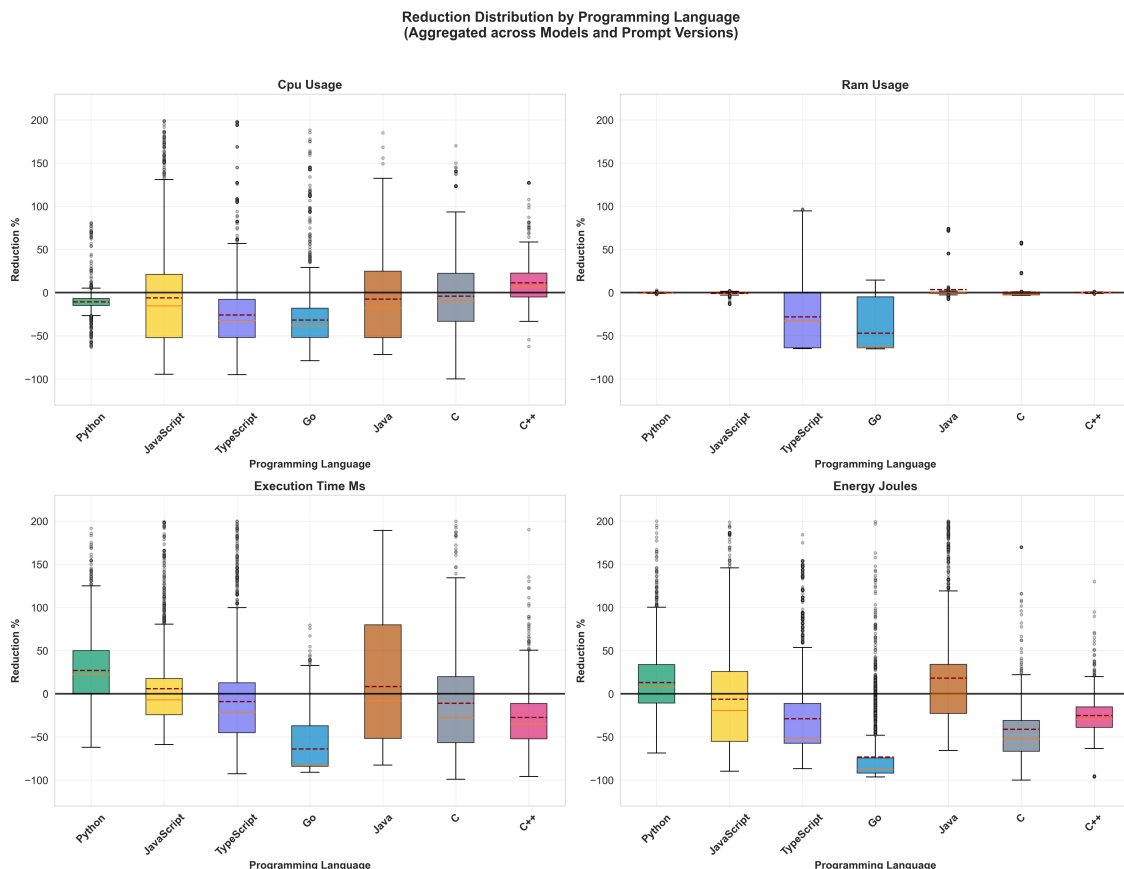


Figura 4.15: Distribuzione delle riduzioni (%) per linguaggio di programmazione (aggregazione per modello e versione del prompt).

La Figura in questione permette di comprendere come vi sia un'elevata varianza degli effetti associati al refactoring tra linguaggi e metriche diverse:

- **Il miglior risultato:** Go risulta ottenere i maggiori benefici dal processo di refactoring, mostrando riduzioni percentuali rispetto ai codici di base sempre al di sotto dello 0 e particolarmente significative per la maggior parte delle metriche. Nonostante la presenza di outlier (massiccia per quanto riguarda le riduzioni del consumo di energia) Go sembra essere il perfetto candidato per il refactoring via LLM, superando costantemente sia in media che con il valore mediano gli altri linguaggi e mostrando di conseguenza le maggiori riduzioni rispetto al codice originale.
- **Miglioramenti significativi:** C++ e TypeScript ottengono entrambi ottimi risultati, seppure C++ mostri degrado nell'uso della CPU piuttosto che un miglioramento e TypeScript abbia parte del proprio quartile al di sopra dello 0 per quanto riguarda le riduzioni associate al tempo di esecuzione, oltre che ad un'elevata presenza di outlier.
- **Utilizzo della RAM:** viene confermato quanto già precedentemente osservato mediante le analisi comparative delle versioni 1 e 4 del prompt, ossia che le riduzioni riguardanti la RAM sono pressoché nulle per tutti i linguaggi ad eccezione di Go e TypeScript.
- **Deterioramento:** vi sono linguaggi quali Java, Python (prevalentemente per il tempo di esecuzione) e JavaScript in cui il refactoring via LLM sembra peggiorare le performance rispetto ai codici originari. Ciò dipende strettamente dalla metrica considerata, ma concentrandosi sull'utilizzo di energia (che aggrega nel calcolo le altre metriche) è facilmente comprensibile come per questi linguaggi vi siano dei deterioramenti anziché miglioramenti nei codici generati dagli LLM.

Osservazioni chiave: I risultati dell'analisi confermano che il refactoring basato su LLM è particolarmente efficace per Go e TypeScript, moderatamente efficiente per C/C++, ma talvolta controproducente in linguaggi come Python e Java, spesso risultando in un codice più lento e più dispendioso in termini di energia. Tuttavia, come stabilito nella precedente analisi sull'impatto del prompt, questa visualizzazione aggregata comprende sia le scarse prestazioni di P1 del prompt che le ottime prestazioni di P4.

Identificazione di Pattern Associati al Refactoring

Le precedenti sezioni erano volte a quantificare quanto gli LLM migliorino le performance del codice originario e quali siano i modelli e i linguaggi per cui tale miglioramento è maggiore. In quest'ultima sezione l'analisi è invece dedicata a comprendere *come* gli LLM riescano ad introdurre migliorie nei codici generati mediante il processo di refactoring. A tal proposito è stato svolto un lavoro di ricerca di pattern di refactoring noti all'interno dei codici generati; lavoro di ricerca i cui risultati sono riportati in questo paragrafo al fine di mostrare quali siano le effettive modifiche applicate dagli LLM.

Metodologia: Filtraggio per Alto Impatto e Bassa Similarità

Per poter identificare correttamente pattern associati al refactoring via LLM, è stato deciso di concentrare la ricerca su un sottoinsieme mirato di codici generati, ossia casi per cui gli LLM hanno introdotto significative ottimizzazioni modificando ampiamente il codice originario.

Per ottenere tale sottoinsieme è stato necessario creare un indice di similarità da associare ad ogni codice generato per poter comprendere quanto tale codice differisca effettivamente dall'originale. Questo indice è volto a catturare i seguenti due aspetti della somiglianza testuale: la struttura sintattica sequenziale (tramite approccio Fuzzy) e la sovrapposizione lessicale (mediante Cosine Similarity).

Calcolo dell'indice di similarità:

L'indice di similarità composto è stato calcolato combinando quindi le seguenti due metriche:

- **Similarità Fuzzy:** viene calcolata sfruttando un algoritmo basato sulla distanza di Levenshtein [25] (implementazione RapidFuzz), la quale misura la somiglianza a livello di caratteri singoli e sequenze, esprimendo il grado di similarità in una scala da 0 a 100.
- **Cosine Similarity:** sia il codice originario che quello generato dagli LLM vengono vettorializzati mediante il modello TF-IDF (Term Frequency-Inverse Document Frequency) [26]. Sui vettori ottenuti viene poi calcolata la similarità del coseno, normalizzata in seguito in scala 0-100.

- **Indice di similarità:** si tratta di una combinazione delle precedenti metriche che utilizza una media ponderata, assegnando pesi lievemente differenti e incentrandosi quindi sulla componente semantica rappresentata dalla cosine similarity:

$$\text{Similarity Index} = 0.45 \times \text{Fuzzy Score} + 0.55 \times \text{Cosine Score} \quad (4.1)$$

Criterio di selezione:

Per isolare i casi significativi di refactoring, ossia quelli in cui gli LLM hanno contribuito sostanzialmente modificando profondamente il codice originario, sono stati applicati i seguenti filtri:

- **Elevata riduzione:** È stata considerata la riduzione associata al consumo energetico rispetto al codice originario in quanto aggrega assieme le altre metriche e sono state selezionate unicamente le entries che raggiungevano almeno il -15% per tale riduzione.
- **Bassa similarità:** l'indice di similarità precedentemente descritto è stato sfruttato per filtrare tutti i codici generati particolarmente somiglianti agli originali, mantenendo solamente codici con un indice di similarità $\leq 75\%$.

L'applicazione di questo duplice filtraggio assicura di ottenere unicamente esempi con interventi sostanziali di refactoring da parte degli LLM, perfetti per poter individuare i pattern introdotti dagli LLM al fine di migliorare le performance dei codici originari. Il sottoinsieme risultante conta più di 7400 coppie di codice (base-LLM) (nel dataset ogni codice di base ha associate 12 versioni generate dagli LLM, tuttavia non tutte vengono considerate in questa analisi a causa del criterio di filtraggio applicato).

Scelta dei Pattern: Nove Categorie di Pattern Generici

Al fine di comprendere quali siano le strategie di ottimizzazione adottate dagli LLM, sono state scelte le seguenti categorie di pattern generici, ossia non specifici per linguaggio, e sono poi state ricercate all'interno dei codici generati dagli LLM mediante l'uso di Abstract Syntax Trees (AST) ed euristiche fondate su regex, assicurandosi che i pattern individuati nei codici generati non fossero già presenti in quelli originari. I pattern individuati sono stati scelti basandosi sulle ottimizzazioni classiche dei compilatori [27], sulle best practices dell'ambito di Ingegneria del Software [8] e su ricerche associate al Green Software Engineering [28, 29].

1. **Loop Invariant Code Motion:** sposta al di fuori dei cicli i calcoli che ritornerebbero lo stesso risultato, riducendo di conseguenza la quantità di cicli di clock associata all'esecuzione del codice.
2. **Branch Optimisation:** semplifica alcune condizioni logiche migliorando la gestione di costrutti switch, molteplici rami if-else e riordinando i casi per aumentare l'efficienza della branch prediction [10].
3. **Reduced Allocations:** riduce la quantità di chiamate a new, malloc o a costruttori degli oggetti, migliorando di conseguenza la gestione della memoria.
4. **Nested Loops:** individuazione di logiche con complessità almeno cubica $O(n^3)$ modificate per ottenere complessità quadratica o inferiore [30].
5. **Lower Complexity Algorithm:** sostituzione di algoritmi inefficienti dal punto di vista computazionale con alternative più performanti (es. utilizzo di HashMaps/Sets $O(1)$ o Binary Search $O(\log n)$).
6. **Allocation in Loop:** individua per poi segnalare l'anti-pattern relativo all'allocazione di oggetti all'interno di cicli iterativi. Muove l'allocazione al di fuori di tali cicli, diminuendo il lavoro del Garbage Collector e la frammentazione della memoria.
7. **Cache Locality:** sfrutta cache L1 e L2 per migliorare la località spaziale lavorando sulle strutture dati utilizzate nel codice [31].

8. **Fine Grained Locking**: migliora l'implementazione di meccanismi di gestione della concorrenza a grana grossa con metodologie di sincronizzazione più precise e/o strutture dati lock-free, riducendo la contesa tra threads e migliorando la scalabilità [32].
9. **Memoization**: introduzione di tecniche di caching al fine di memorizzare i risultati di funzioni computazionalmente costose, evitando calcoli ridondanti.

Distribuzione dei Pattern: Analisi di Frequenza

La Figura 4.16 presenta la distribuzione dei pattern generici di refactoring individuati tra i codici del dataset filtrato secondo la metodologia precedentemente descritta.

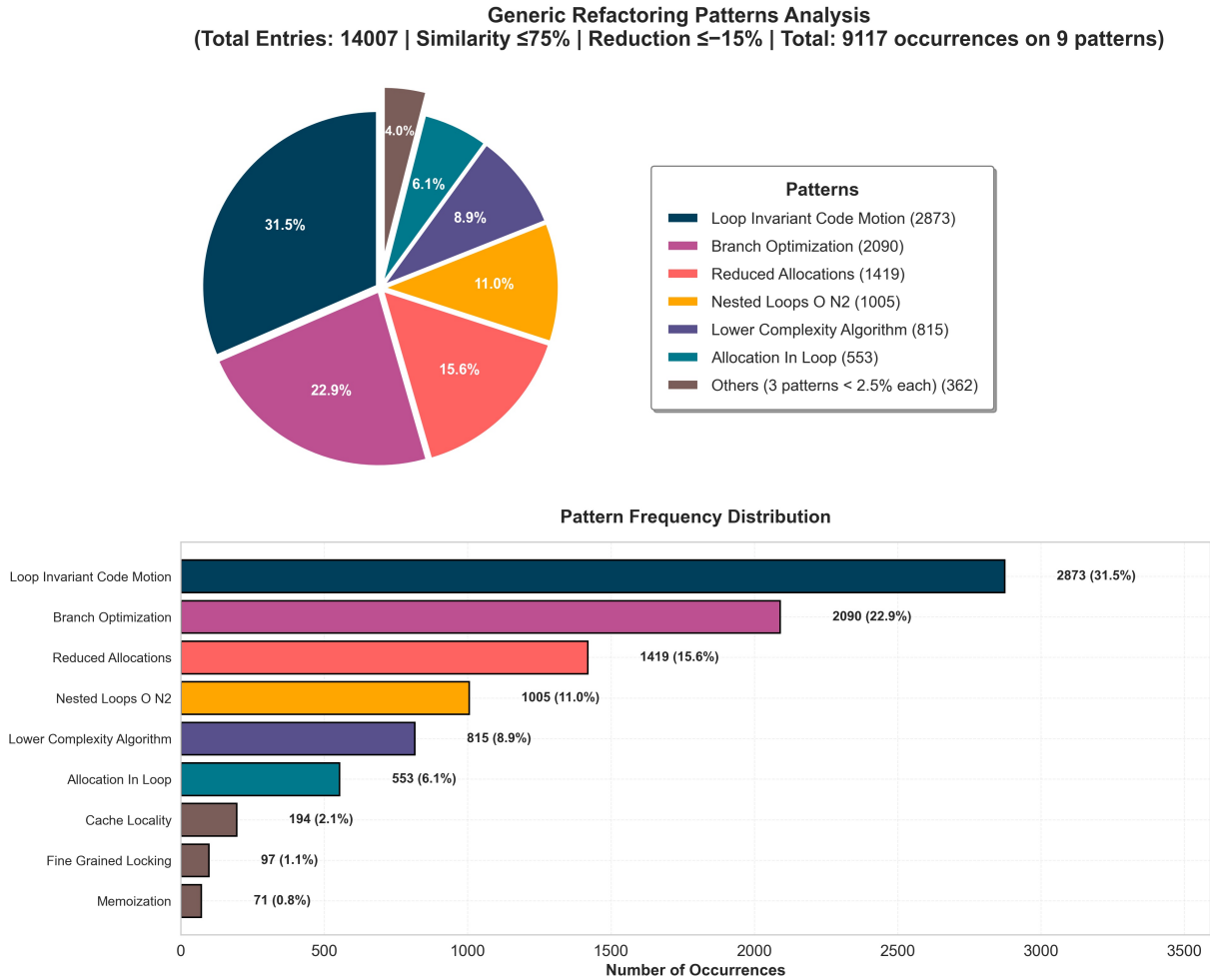


Figura 4.16: Distribuzione dei pattern generici individuati all'interno del dataset filtrato (riduzione energetica $\leq -15\%$, similarità $\leq 75\%$). A sinistra vi è un grafico a torta con le percentuali relative ad ogni pattern, mentre sotto è raffigurato un grafico a barre con le quantità assolute delle occorrenze per ogni pattern.

- **Loop Invariant Code Motion (2.873 occorrenze, 31.5%)**: è il pattern con la frequenza maggiore, presente in circa un terzo dei codici generati dagli LLM. Ciò conferma come gli LLM abbiano ottime capacità di identificazione di operazioni ridondanti all'interno dei loop.
- **Branch Optimisation (2.090 occorrenze, 22.9%)**: presente nel 22.9% dei codici rifattorizzati dimostra come gli LLM riescano ad individuare e migliorare efficacemente le condizioni ridondanti o disorganizzate all'interno del codice, impattando positivamente sulla riduzione del consumo energetico.

- **Reduced Allocations (1.419 occorrenze, 15.6%)**: i modelli considerati nello studio riescono spesso ad effettuare un refactoring attento e mirato al miglioramento della gestione della memoria, con relativi benefici per quanto riguarda il consumo energetico.
- **Nested Loops (1.005 occorrenze, 11.0%)**: ai fini del refactoring gli LLM all'incirca una volta su dieci vanno a ridurre la complessità del codice agendo su cicli iterativi annidati, ottenendo performance migliori nel codice generato rispetto all'originale.
- **Lower Complexity Algorithm (815 occorrenze, 8.9%)**: seppur meno frequente rispetto ad altri pattern introdotti dagli LLM, questi ultimi riescono talvolta ad individuare correttamente la complessità algoritmica dei codici in input generando in output snippet di codice in cui vengono sfruttate strutture più efficienti in grado di ridurre la complessità computazionale.
- **Allocation in Loop (553 occorrenze, 6.1%)**: in determinati scenari gli LLM riescono ad individuare allocazioni ripetute interne a cicli iterativi e riescono a riorganizzare il codice per gestire al meglio tali allocazioni, migliorando di conseguenza le performance.
- **Cache Locality (194 occorrenze, 2.1%)**: di rado gli LLM sfruttano strutture dati volte ad ottenere vantaggi computazionali mediante l'utilizzo delle cache L1 e L2.
- **Fine Grained Locking (97 occorrenze, 1.1%)**: in poco più dell'1% dei casi, anche e soprattutto a causa della natura dei codici considerati, gli LLM individuano e migliorano un'imprecisa gestione della concorrenza, creando codice maggiormente efficiente rispetto all'originale.
- **Memoization (71 occorrenze, 0.8%)**: nella minor parte dei casi gli LLM hanno (ed individuano) l'occasione di sfruttare cache nel codice per il salvataggio di risultati che, altrimenti, andrebbero calcolati nuovamente (come avviene nei codici originari), portando benefici in termini di performance.

Osservazioni:

La distribuzione della presenza di pattern introdotti nei codici generati mostra come vi sia una forte concentrazione nei primi 3 pattern (Loop Invariant Code Motion, Branch Optimisation, Reduced Allocations), i quali costituiscono circa il 70% del totale.

Esempi di refactoring:

All'interno dell'Appendice [B](#) è possibile visualizzare esempi concreti di pattern introdotti dagli LLM nei codici generati, con annesse comparazioni dettagliate tra codice originale e codice rifattorizzato e relative analisi dei miglioramenti ottenuti.

Capitolo 5

Discussione

Questo capitolo è volto a sintetizzare e riassumere i risultati presentati nel Capitolo 4, andando a rispondere alle principali domande poste in questo studio.

Inizialmente vengono riassunte le scoperte in merito alle performance e alla correttezza (Sezione 5.1), in seguito la Sezione 5.2 approfondisce le implicazioni dei risultati individuati nel campo dell’Ingegneria del Software e Green Computing. La Sezione 5.3 esamina criticità e possibili minacce di validità, oltre che le limitazioni, di questo studio. Infine la Sezione 5.4 fornisce un riepilogo dei principali risultati ottenuti.

5.1 Risposte alle Principali Domande di Ricerca

L’obiettivo primario di questa tesi è rispondere alla seguente domanda di ricerca: “Se, come e quanto i differenti LLM considerati (OpenAI, Gemini, Claude) riescano a migliorare le performance di snippet di codice reale (scritto da umani) mediante un processo di refactoring sostenibile?”

Gli LLM Sono in Grado di Migliorare il Codice Preservandone la Correttezza?

Come stabilito nelle Sezioni 3.1 e 3.5, tutti i codici di base devono ottenere il 100% di pass rate su cinque esecuzioni delle relative test case per essere considerati validi ed essere aggiunti al dataset. Ciò permette quindi di misurare i miglioramenti associati alle performance introdotte dal refactoring e il mantenimento della correttezza piuttosto che analizzare le capacità degli LLM di risolvere problematiche ed errori già presenti nel codice originario.

Riguardo alle difficoltà nel mantenere la correttezza, l’analisi presente nella Sezione 4.1 dimostra come nessun linguaggio e nessuna combinazione di modello-versione del prompt riesca a preservare la correttezza nel 100% dei casi (Figura 4.3). Perfino la miglior configurazione (Claude con la prima versione del prompt) ha un degrado significativo del -12% . Ciò indica come gli LLM considerati non siano completamente affidabili nel processo di refactoring, il quale è consigliabile che sia seguito e revisionato.

Tramite i risultati emersi nella Sezione 4.2 è facilmente comprensibile come il linguaggio scelto per effettuare refactoring mediante LLM rappresenti una variabile critica: linguaggi come Go, TypeScript e Java superano il 90% di pass rate, mentre linguaggi come C++ faticano a raggiungere il 65%. Questo divario dimostra come la capacità degli LLM di mantenere la correttezza nel processo di refactoring dipenda fortemente dal linguaggio considerato.

In conclusione sì, gli LLM riescono a migliorare il codice ma con le seguenti 3 osservazioni critiche:

1. **La correttezza non viene mantenuta incondizionatamente:** anche le migliori configurazioni falliscono nel mantenere una baseline di correttezza perfetta.
2. **Affidabilità strettamente correlata al linguaggio:** il pass rate dei codici generati dagli LLM subisce grandi variazioni tra i linguaggi considerati, dimostrando come la scelta del linguaggio sia un punto critico nel processo di refactoring.
3. **I miglioramenti in termini di performance non sono garantiti:** nonostante la correttezza venga mantenuta, non si ha necessariamente un miglioramento nelle metriche considerate con la versione di codice generata dagli LLM, come dimostrato dalla presenza degli outlier (Sezione 4.2).

Riassumendo, la risposta alla domanda “Gli LLM possono migliorare il codice?” è: “Sì, in particolare per linguaggi staticamente e fortemente tipati, seppure la validazione dei risultati sia sempre necessaria in quanto non vi è un’affidabilità assoluta nel mantenimento della correttezza da parte degli LLM nel processo di refactoring”.

Quanto gli LLM Migliorano il Codice.

Considerando il sottoinsieme dei codici rifattorizzati in cui viene mantenuta la correttezza, i guadagni in termini di prestazioni e sostenibilità sono sostanziali. Come mostrato nel confronto globale (Sezione 4.2), i dati mostrano miglioramenti significativi, specialmente in termini di riduzione energetica, come mostrato dai seguenti miglioramenti mediani:

- **Uso di CPU (%)**: -18.57%;
- **Uso di RAM (KB)**: -0.97%;
- **Tempo di esecuzione**: -22.54%;
- **Consumo energetico (Joule)**: -38.40%.

Ciò è facilmente comprensibile dalla distribuzione di frequenze delle riduzioni mostrata nella Figura 4.5, la quale mostra inoltre che la maggior parte delle riduzioni si trova a sinistra rispetto allo 0 (valori negativi). Tuttavia sono presenti code di degrado a destra dello 0 (valori positivi) che indicano la presenza di casi di refactoring che peggiorano le performance del codice originario, mostrando come i guadagni associati alle metriche considerate non siano garantiti.

In sintesi, gli LLM riescono ad introdurre significative miglirie nei codici generati nella maggior parte dei casi, rivelandosi uno strumento valido per il refactoring sostenibile, in particolar modo per quanto riguarda le riduzioni associate al consumo energetico.

Come gli LLM Riescono ad Ottenere i Miglioramenti delle Performance mediante Refactoring.

Il “come” viene spiegato considerando la congiunzione delle principali variabili sperimentali incontrate durante la fase di analisi: il prompt, il modello e il linguaggio considerati.

In primo luogo, **il processo di ingegnerizzazione del prompt è critico**: il grande divario di miglioramenti tra la prima versione del prompt e le altre raggiunge picchi del 20% (confrontando Gemini P1: -17.65% con Gemini P2: -37.34% riguardo alla riduzione energetica). Ciò, assieme ai risultati delle analisi svolte nelle Sezioni 4.2 e 4.2, dimostra come le performance siano fortemente influenzate dal prompt, il quale diviene uno dei punti chiave su cui programmatori e ingegneri del software dovrebbero porre particolare attenzione quando si apprestano ad effettuare refactoring sfruttando gli LLM.

In secondo luogo, **la scelta del modello è spesso secondaria**: le principali e maggiormente significative differenze tra i modelli considerati si notano prevalentemente per quanto riguarda il pass rate, in relazione ai diversi linguaggi di programmazione. In tale analisi, come dimostrato nella Sezione 4.1, emerge una configurazione abbastanza eterogenea tra modelli diversi considerando il medesimo linguaggio: in particolare per C++ vi è una buona differenza del 13% tra OpenAI e gli altri modelli (56% di OpenAI vs 69% di Claude e Gemini). Ciò tuttavia non riflette anche il comportamento legato alle metriche: i miglioramenti associati alle performance hanno lievi variazioni tra i modelli di LLM considerati, mostrando che le capacità degli LLM scelti per questo studio sono piuttosto simili tra loro e che non è presente un modello che prevalga sugli altri.

Infine, **il linguaggio di programmazione considerato ha un ruolo chiave**: come già discusso e come mostra la Figura 4.15, i risultati relativi alle performance e al pass rate variano sensibilmente tra i linguaggi di programmazione considerati. Sono presenti sia linguaggi come TypeScript e Go in cui vi sono sensibili miglirie per quanto riguarda le metriche considerate e con pass rate elevati, sia linguaggi quali C++ ad esempio in cui vi è un trade-off tra pass rate e riduzioni (specialmente energetiche): minor pass rate ma buone riduzioni. Infine vi sono anche linguaggi, come Java, che presentano elevati pass rate (91.0%), ma performance talvolta negative (massiccia presenza di outlier, talvolta media delle riduzioni sopra lo 0, indicando peggioramenti). Questa elevata varianza associata alla scelta del linguaggio indica come questa sia una variabile da tenere in forte considerazione quando si effettua refactoring sostenibile mediante LLM.

Oltre alle variabili sperimentali precedentemente considerate, un importante ruolo in relazione al “come” gli LLM migliorino le performance mediante refactoring lo hanno anche i pattern introdotti nei codici generati, analizzati nella Sezione 4.2.

Il Ruolo dei Pattern Individuati nel Refactoring del Codice. Considerando i pattern aggiunti più frequentemente dagli LLM nel processo di refactoring, ossia Loop Invariant Code Motion (31.5%), Branch Optimisation (22.9%) e Reduced Allocation (15.6%), e la relativa analisi è possibile ipotizzare come:

- Loop Invariant Code Motion contribuisca alla riduzione del tempo di esecuzione e dei cicli di CPU, con conseguente diminuzione dell'uso % di CPU;
- Reduced Allocation sia volto ad abbassare la quantità di RAM utilizzata;
- Branch Optimisation aiuti a diminuire sia il tempo di esecuzione che l'uso di CPU, con relativo miglioramento riguardo consumo energetico.

La massiccia presenza di questi pattern nei codici generati dagli LLM permette di comprendere più a fondo come linguaggi quali Go e TypeScript possano ottenere miglioramenti elevati nelle performance, probabilmente dovuti alla diminuzione delle allocazioni e alla diversa gestione dei cicli iterativi. È inoltre ipotizzabile che pattern relativi alla diversa gestione della memoria possano aver aumentato il rischio di errori in linguaggi come C e principalmente C++, i quali presentano i minori pass rate.

In conclusione, questo lavoro di tesi è riuscito a dimostrare che gli LLM possono essere strumenti utili nell'ambito del refactoring sostenibile: sono in grado (*se*) di migliorare (*quanto*) significativamente le performance mediante (*come*) l'introduzione di pattern durante il processo di refactoring. Tuttavia i risultati ottenuti dipendono fortemente dal linguaggio considerato e necessitano di un accurato e attento processo di ingegnerizzazione del prompt, mentre la scelta del modello rimane secondaria. Infine l'intero processo di refactoring non è affidabile al 100% e necessita sistematicamente di validazione e controllo umano, in quanto anche nelle migliori condizioni (linguaggi precisamente scelti, versione del prompt curata e uso del miglior modello disponibile) non viene mai raggiunto il 100% del pass rate insito nei codici originali.

5.2 Lezioni Apprese

Dalle sfide affrontate in questo lavoro di tesi emergono una serie di “lezioni apprese”, di cui in seguito vengono discusse le principali criticità riscontrate, le relative soluzioni adottate e le raccomandazioni per future ricerche in merito.

Implicazioni per l'Ingegneria del Software.

La principale lezione imparata relativamente al campo dell'Ingegneria del Software è il fatto che gli LLM possano essere visti come strumenti potenti e utili, seppure imperfetti, per la collaborazione relativa all'ottimizzazione del codice. Il workflow tradizionale in cui lo sviluppo del software e il relativo refactoring sono processi guidati da figure competenti ed esperte può essere affiancato dall'uso degli LLM. A riguardo da questo studio si possono evincere le seguenti tre lezioni.

La prima lezione è che **la validazione è necessaria**: una problematica incontrata in questo studio riguarda l'elevata percentuale di regressione funzionale legata al refactoring mediante LLM, specialmente in linguaggi complessi come C++, la quale conferma che gli LLM introducano errori e non siano completamente affidabili. Perciò in qualsiasi utilizzo pratico che si avvalga di tali tecnologie è necessario strutturare un processo di test (automatizzabile) necessario a controllare e garantire che la correttezza venga mantenuta anche in seguito agli interventi degli LLM. Per tale motivo in questo lavoro è stata strutturata una pipeline di validazione volta non solo ad individuare gli errori introdotti dal refactoring via LLM, ma anche a comprenderne le cause. Questo livello di attenzione diviene quindi fondamentale per coloro che abbiano l'intenzione di affiancare l'uso di questi strumenti al processo di refactoring del codice.

La seconda lezione è che **il prompt engineering è una vera e propria abilità**: le elevate differenze individuate tra le versioni diverse del prompt rafforzano il fatto che il processo di ingegnerizzazione del prompt sia determinante per ottenere buoni risultati. Ciò implica le abilità necessarie a scrivere un prompt chiaro e conciso, in grado di spiegare contesto e obiettivo, guidando gli LLM nel lavoro richiesto per l'ottenimento del risultato finale. Per questo è stata posta particolare attenzione

e cura alla scrittura dei prompt, risolvendo il problema di prompt generici e inefficienti mediante un processo di ingegnerizzazione in più fasi, una per ciascuna versione prodotta.

La terza lezione riguarda il **comportamento dipendente dal linguaggio**: non è possibile generalizzare il comportamento degli LLM assumendo che un modello funzioni bene e indifferentemente su tutti i linguaggi. Bensì, come ampiamente dimostrato e analizzato, vi sono modelli che ottengono risultati particolarmente migliori su determinati linguaggi e vi sono poi alcuni linguaggi (C++) per cui è particolarmente arduo ottenere correttezza funzionale e/o miglioramento in termini di performance. È stato quindi necessario assicurarsi che l'ambiente di esecuzione comprendesse quanto necessario per supportare tutti i linguaggi scelti e le diverse tipologie di test per ogni linguaggio, ciò al fine di accertarsi che gli errori nei codici rifattorizzati fossero responsabilità unicamente degli LLM e non fossero invece causati da altri fattori.

Implicazioni per il Green Computing e la Sostenibilità Ambientale.

La lezione emersa da questo studio è che gli LLM riescono concretamente a ridurre considerevolmente l'impiego di energia necessario all'esecuzione del codice mediante refactoring. Con una riduzione mediana di quasi il 40% (Sezione 4.2) sui codici corretti è possibile considerare i seguenti punti:

Il primo aspetto riguarda la **riduzione dell'impronta di carbonio legata al software**: migliorando codici già esistenti sfruttando gli LLM è possibile ridurre il consumo energetico e di conseguenza l'impronta di carbonio; ciò su larga scala può avere un considerevole e positivo impatto ambientale, dimostrando come "green pattern" presenti nella teoria abbiano un riscontro pratico e considerevole. Per misurare e quantificare le riduzioni energetiche introdotte dal refactoring via LLM è stato fondamentale l'utilizzo di container Docker per la creazione di ambienti indipendenti e isolati e per evitare bias e problematiche annesse alla macchina su cui sono state eseguite le test suite.

Il secondo aspetto concerne la **riduzione del divario e traslazione delle competenze**: ottimizzazioni associate alle performance e all'energia richiedono abilità elevate e specializzate. L'impiego degli LLM, quando svolto con attenzione, può abbassare il livello di competenze specifiche richiesto, a discapito della capacità d'uso di tali modelli: mentre non chiunque è in grado di ottimizzare il codice al fine di ottenere significative migliorie, la maggior parte dei developer possono essere capaci di sfruttare gli LLM per questo intento. Tuttavia per ottenere risultati ottimali dagli LLM è necessario porre particolare attenzione al processo di scrittura del prompt, su cui verte fortemente la qualità dell'output generato. In questo studio è stato quindi effettuato un accurato e attento processo di ingegnerizzazione del prompt, come descritto nella Sezione 3.3, per far fronte a risultati insoddisfacenti che invece sarebbero stati generati utilizzando prompt vaghi e generici.

5.3 Minacce di Validità dello Studio

Al fine di contestualizzare correttamente i risultati ottenuti in questo lavoro di tesi è cruciale e necessario valutare le minacce alla validità di questo studio. Tale valutazione è stata organizzata seguendo la tassonomia di Wohlin [33].

Validità Interna.

Una prima minaccia alla validità interna riguarda il *filtro sul pass rate*. La scelta di richiedere il 100% del pass rate per le cinque esecuzioni delle base entries (Sezione 3.5) garantisce una baseline deterministica e stabile per misurare la degradazione introdotta dagli LLM, ma introduce un bias di selezione: il dataset risultante esclude codici con comportamenti dipendenti dall'ambiente o con sporadici fallimenti di esecuzione, allontanandosi da scenari reali in cui sono spesso presenti inesattezze e comportamenti instabili. Lo studio è dunque volto a valutare l'abilità degli LLM di mantenere la correttezza piuttosto che migliorare codici inesatti, un vincolo più stringente che potrebbe sottostimare le capacità degli LLM in contesti in cui la correttezza iniziale non è garantita. Ciononostante, tale scelta è stata necessaria per concentrarsi sulle capacità di refactoring sostenibile, per cui la correttezza iniziale del codice rappresenta un requisito fondamentale.

Una seconda minaccia concerne il *bias di modello*. Determinati modelli tra quelli considerati possono risultare più performanti su specifici linguaggi poiché maggiormente addestrati su di essi; a titolo esemplificativo, Gemini potrebbe aver ottenuto risultati particolarmente buoni su Go in virtù di una specializzazione su tale linguaggio. Questo bias, indicativo delle differenti caratteristiche dei

modelli stessi (Sezione 3.2), è stato mitigato adottando un approccio comparativo tra più modelli anziché incentrare lo studio su un'unica scelta, identificando in tal modo anche pattern indipendenti dal singolo modello. Tale aspetto rappresenta inoltre un'opportunità per futuri sviluppi, ad esempio nella creazione di framework che associno il linguaggio al modello più adatto (Sezione 6.2).

Validità Esterna.

La principale minaccia alla validità esterna è legata alla *rappresentatività del dataset*. Il dataset è composto prevalentemente da entries ottenute dalla piattaforma Exercism, scelta che ha consentito di costruire una baseline comprensiva di test suite ma che può non rappresentare al meglio sistemi software complessi e di larga scala. Tale limitazione è coerente con l'obiettivo della tesi, focalizzata sul refactoring a livello di singola funzione o classe: l'analisi su ampi progetti avrebbe alterato significativamente il paradigma di valutazione, aumentando notevolmente la complessità del lavoro.

Un'ulteriore minaccia riguarda la *specificità dei linguaggi*, in particolare C++, che ha costantemente mostrato il minor pass rate in tutte le analisi (Sezione 4.1). Con un pass rate aggregato del 64.7% — disaggregato in 53% per OpenAI, 69% per Claude e 69% per Gemini — il divario rispetto a Go (95.3%) e TypeScript (92.1%) è rilevante e persiste nonostante il processo di ingegnerizzazione del prompt. Le principali cause risiedono nella gestione manuale della memoria, dove i pattern “Reduced Allocation” e “Allocation in Loop” (Sezione 4.2) possono introdurre errori specifici del linguaggio, e nella separazione tra file header e file di implementazione, che espone il codice rifattorizzato a incongruenze nelle firme delle funzioni con conseguenti errori di compilazione. Queste problematiche rendono il refactoring automatico di C++ particolarmente rischioso, richiedendo interventi di controllo e validazione manuali. Per mitigare questa criticità è stata posta particolare attenzione all'apparato sperimentale, assicurandosi che non vi fossero errori nella valutazione ed esecuzione dei codici in questo linguaggio; il risultato finale è quindi indicativo di un limite intrinseco degli LLM nella gestione di C++, per cui sarebbero necessarie accortezze specifiche quali, ad esempio, l'inserimento dei file header nei prompt.

Validità di Costrutto.

La minaccia principale alla validità di costrutto è legata all'*ingegnerizzazione del prompt*. L'intero studio ha dimostrato come la formulazione del prompt sia una variabile cruciale per l'efficacia del refactoring via LLM (Sezione 4.2). Le quattro versioni utilizzate (Sezione 3.3) sono state progettate con specifici obiettivi di ricerca, ed è quindi possibile che prompt strutturati diversamente avrebbero prodotto risultati differenti; le scoperte emerse sono pertanto legate ai prompt utilizzati, rappresentando un potenziale bias. Tale bias non è totalmente eliminabile, poiché riflette la soggettiva interazione umana con gli LLM, ma è stato limitato utilizzando il medesimo prompt per tutti i modelli e linguaggi (Sezione 3.3). Un'ulteriore mitigazione è stata ottenuta attraverso un'analisi di sensibilità su più versioni del prompt, dimostrando che i risultati non sono casuali ma consistenti su diverse formulazioni.

Una seconda minaccia riguarda l'*ambiente di misurazione*. Le metriche di performance (uso di CPU, uso di RAM, tempo di esecuzione) sono state raccolte in container Docker mediante Colima (Sezione 3.4), una scelta che garantisce isolamento e riproducibilità ma introduce un livello di astrazione rispetto a misure hardware dirette, specialmente per il calcolo del consumo energetico. Il rischio di imprecisioni nei valori assoluti è stato accettato per mancanza di strumentazione specifica, ed è stato in parte mitigato dall'uso di un ambiente containerizzato indipendente dalle caratteristiche della macchina host, in grado di garantire la validità dei confronti relativi tra codice originale e codice rifattorizzato.

5.4 Riepilogo dei Principali Risultati

Di seguito vengono riepilogati i principali risultati emersi dallo studio, sintetizzati nella Tabella 5.1.

Tabella 5.1: Riepilogo dei principali risultati dello studio

Aspetto	Risultato Principale
Efficacia del refactoring	Gli LLM riducono il consumo energetico con una riduzione mediana del -38.40% , confermando la loro utilità come strumenti per il refactoring sostenibile.
Riduzioni mediane per metrica	Uso di CPU: -18.57% Uso di RAM: -0.97% Tempo di esecuzione: -22.54% Consumo energetico: -38.40%
Correttezza funzionale	Nessuna configurazione mantiene il 100% di pass rate. Il degrado varia fortemente per linguaggio: Go e TypeScript $>90\%$, C++ $\approx 65\%$.
Impatto del prompt	L'ingegnerizzazione del prompt ha un effetto significativo: l'ultima versione (P4) produce risultati più consistenti e riduzioni energetiche maggiori rispetto alla prima (P1), con differenze fino al 20% .
Differenze tra modelli	Le differenze tra i modelli (OpenAI, Claude, Gemini) sono limitate per quanto riguarda le performance; emergono principalmente nel pass rate per linguaggi specifici.
Dipendenza dal linguaggio	Il linguaggio è una variabile critica sia per la correttezza che per le performance. TypeScript e Go ottengono i migliori risultati complessivi; C++ presenta le maggiori criticità.
Pattern di refactoring	I pattern più frequenti sono Loop Invariant Code Motion (31.5%), Branch Optimisation (22.9%) e Reduced Allocation (15.6%), che spiegano le principali ottimizzazioni introdotte dagli LLM.

In sintesi, i risultati confermano che gli LLM rappresentano uno strumento efficace per il refactoring sostenibile del codice, con significative riduzioni del consumo energetico nella maggior parte dei casi. Tuttavia, l'efficacia dipende fortemente dal linguaggio di programmazione e dalla qualità del prompt, mentre la correttezza funzionale non è mai garantita e richiede sistematicamente una validazione umana.

Capitolo 6

Conclusioni

Questo capitolo finale sintetizza i principali risultati di questa ricerca e li situa nel contesto più ampio dell'Ingegneria del Software assistita da AI e della sostenibilità computazionale. La Sezione 6.1 fornisce un sommario conciso dei contributi empirici della tesi, affrontando le domande di ricerca centrali riguardanti l'efficacia del refactoring basato su LLM per migliorare le prestazioni del codice e ridurre l'impatto ambientale. La Sezione 6.2 delinea direzioni promettenti per estendere questo lavoro, includendo miglioramenti metodologici, analisi più profonda dei pattern e lo sviluppo di framework di refactoring intelligenti adattati a contesti di programmazione specifici.

6.1 Sintesi dei Contributi

Questa tesi ha condotto un'indagine sistematica, empirica e su larga scala riguardo all'impatto del refactoring via LLM sulle performance, la correttezza e la sostenibilità del codice, cercando di colmare le lacune della letteratura attuale riguardo all'intersezione dei campi del Green Computing con l'Ingegneria del Software guidata dall'AI. In risposta alle domande di ricerca poste nella Sezione 1.3 — se, quanto e come gli LLM possano migliorare l'efficienza e la sostenibilità dei codici mediante refactoring — questo lavoro ha fornito tre contributi principali, in linea con quanto anticipato nella Sezione 1.4.

Anzitutto la realizzazione di un **dataset strutturato e multi-linguaggio per analisi di sostenibilità**, comprendente oltre 2.300 snippet di codice in sette linguaggi di programmazione, ciascuno corredato da una test suite per la validazione automatica della correttezza funzionale. Il dataset, organizzato in cluster per tipo di esercizio e affiancato da una pipeline automatizzata per l'espansione e la raccolta delle metriche, costituisce una risorsa riproducibile per future valutazioni del refactoring sostenibile via LLM.

Un secondo apporto da questo studio è il **design e l'utilizzo di una metodologia sperimentale rigorosa**, la quale prevede l'invocazione sistematica di tre modelli LLM con quattro versioni del prompt per ogni codice di base, l'esecuzione in ambienti containerizzati e la raccolta completa di metriche grezze (CPU, RAM, tempo di esecuzione, pass rate) e derivate (consumo energetico). L'infrastruttura, completamente automatizzata, consente la replicazione e l'estensione dell'esperimento.

Infine, l'ultimo contributo risiede nelle **analisi quantitative su correttezza, performance e impatto ambientale**. Sono stati raccolti oltre 150.000 record di esecuzione che hanno permesso di fornire risposte basate sui dati alle domande di ricerca, caratterizzando empiricamente l'efficacia del refactoring LLM e identificando interazioni complesse tra modello, linguaggio e formulazione del prompt. Tali interazioni mettono in discussione l'ipotesi di prestazioni uniformi degli LLM e forniscono prove a sostegno di strategie di selezione specifiche per contesto.

Dai risultati emergono implicazioni dirette per la pratica dell'Ingegneria del Software. In primo luogo, gli sviluppatori e le organizzazioni non dovrebbero utilizzare gli LLM per il refactoring senza un approccio critico: l'utilizzo dovrebbe essere selettivo e basato sul contesto, dando priorità a linguaggi particolarmente adatti (quali Go e TypeScript) e sottoponendo ogni codice generato a rigorosi test di regressione. In secondo luogo, la qualità del refactoring è fortemente sensibile al design del prompt: mentre P2 ha ottenuto le riduzioni percentuali più elevate, P4 produce il codice finale più efficiente con il consumo energetico assoluto più basso, il che suggerisce che organizzazioni e società debbano porre particolare attenzione all'ingegnerizzazione del prompt. Infine, per sfruttare al meglio gli LLM ai fini del refactoring, occorre scegliere con criterio il modello in base al linguaggio considerato, poiché per alcuni di essi (primo tra tutti C++) può risultare troppo rischioso e/o inefficiente automatizzare il processo.

In sintesi, questa tesi dimostra che il refactoring basato su LLM è una tecnica valida e con un impatto considerevole per migliorare le prestazioni del codice e la relativa sostenibilità, a condizione che sia applicata con giudizio, convalidata rigorosamente e adattata a specifici contesti di programmazione.

Punti di forza.

L'efficacia comprovata, visibile aggregando tutti i codici rifattorizzati e osservando che gli LLM raggiungono una riduzione mediana del -38.40% per il consumo energetico, con riduzioni significative anche per uso di CPU (-18.57%) e tempo di esecuzione (-22.54%).

Il ruolo chiave dell'ingegnerizzazione del prompt, il cui design influenza significativamente la qualità del refactoring. P4 produce i codici finali più efficienti in termini assoluti, mentre P2 raggiunge le più elevate riduzioni percentuali, confermando l'importanza di un attento processo di ingegnerizzazione.

Il successo su linguaggi fortemente tipati come Go e TypeScript, che emergono come casi con riduzioni consistenti su tutte le metriche e pass rate elevati (rispettivamente 95.3% e 92.1%).

L'identificazione dei nove pattern generici di refactoring, individuati mediante AST e Regex. Essi mostrano come gli LLM applichino prevalentemente euristiche di ottimizzazione a livello di ciclo e di allocazione.

Punti di debolezza e limitazioni.

È stato osservato che la correttezza non è garantita, in quanto nessuna configurazione preserva perfettamente la correttezza funzionale. Il degrado è particolarmente grave per C++ ($\approx 65\%$ di pass rate), rendendo il refactoring automatico rischioso senza validazione umana.

Dalle analisi emerge inoltre che l'efficacia è fortemente dipendente dal linguaggio: C e C++ rappresentano i linguaggi più ardui per gli LLM, indicando come l'efficacia del refactoring non sia universale ma dipenda dal sistema di tipi, dalle caratteristiche di runtime e dalla rappresentazione del linguaggio nei dati di addestramento.

La stima indiretta dell'energia è un'ulteriore limitazione di questo studio, in quanto il consumo energetico è derivato da modelli basati su TDP anziché misurato direttamente tramite hardware, introducendo potenziali imprecisioni nei valori assoluti.

Un'altra limitazione è data dal dataset utilizzato, ristretto a snippet isolati. I codici analizzati provengono da Exercism e non rappresentano la complessità di codebase reali su larga scala.

Infine la correlazione pattern-performance non è quantificata poiché l'analisi identifica i pattern più frequenti ma non stabilisce un legame causale quantitativo tra singoli pattern e miglioramenti osservati.

Le limitazioni sopra elencate definiscono le principali direzioni per i lavori futuri presentati nella Sezione [6.2](#).

6.2 Lavori Futuri

Questa tesi ha fornito evidenza empirica per l'efficacia del refactoring basato su LLM per quanto riguarda il miglioramento delle performance e la sostenibilità del codice, al contempo rivela molteplici opportunità per ulteriori indagini, studi e approfondimenti. Di seguito vengono discusse alcune direzioni promettenti per la ricerca futura.

Una prima direzione riguarda l'**espansione del dataset, dei modelli e/o dei linguaggi**: il dataset corrente è concentrato sui sette linguaggi di programmazione considerati e su una specifica fonte (Exercism). Lavori futuri potrebbero e dovrebbero estendere il dataset per includere nuovi linguaggi (ad esempio Rust, Kotlin, Swift, C#) e magari aggiungere codici da ulteriori e diverse fonti, comprese repository open source, piattaforme di programmazione (ad esempio, LeetCode, Codeforces) e codebase. In modo simile, anche l'insieme dei modelli potrebbe essere espanso per comprendere altre alternative (ad esempio DeepSeek, Coder, CodeLlama). Tali ampliamenti del dataset metterebbero alla prova la generalizzabilità dei risultati, comprendendo se i modelli più recenti presentino migliori capacità di refactoring o se introducano una maggior quantità di errori.

Una seconda direzione concerne le **misure a livello hardware per l'energia**: le metriche riguardanti il consumo energetico riportate in questo lavoro sono state derivate dal consumo di CPU,

dall'uso della RAM e dal tempo di esecuzione. Anche se questo approccio è stato validato dalla letteratura precedente (Sezione 3.5), introduce errori di stima e astrae le specifiche caratteristiche hardware. Lavori futuri dovrebbero incorporare, quando possibile, misure dirette a livello hardware per misurare l'effettivo consumo energetico mediante appositi strumenti, ottenendo di conseguenza valori più affidabili e privi di errori di approssimazione.

Un'ulteriore prospettiva è l'**analisi su ampie codebase**: questo studio si concentra su code snippet isolati (di massimo 100-150 righe di codice) e associate test suite. Sebbene questa progettazione consenta un controllo preciso e una robustezza statistica, non riflette la complessità dei sistemi software reali, in cui il codice interagisce con più moduli, librerie esterne e sottosistemi di I/O. Lavori futuri dovrebbero estendere la metodologia a basi di codice più ampie, analizzando l'impatto del refactoring LLM a livello di modulo, pacchetto o sistema. Tali analisi rivelerebbero se le ottimizzazioni localizzate si propagano anche a miglioramenti delle prestazioni a livello di sistema o se le interazioni tra moduli annullano i benefici di refactoring isolati. Inoltre, l'analisi a livello di sistema consentirebbe la valutazione di attributi olistici di qualità del software che vanno oltre le prestazioni, come modularità, accoppiamento e coerenza architetturale.

I risultati di questa tesi motivano inoltre lo sviluppo di un **framework per il refactoring intelligente** che selezioni dinamicamente la configurazione ottimale di LLM e prompt per un dato frammento di codice e obiettivo di ottimizzazione. Un framework del genere analizzerebbe le caratteristiche del codice (ad esempio linguaggio, complessità algoritmica, utilizzo delle strutture dati, bottleneck prestazionali) e indirizzerebbe l'attività di refactoring alla combinazione modello-prompt consigliata per massimizzare il miglioramento riducendo al minimo il rischio funzionale. Il framework potrebbe essere implementato come plugin IDE, integrazione CI/CD o strumento CLI autonomo, fornendo agli sviluppatori suggerimenti di refactoring automatizzati e sensibili al contesto. Tecniche di apprendimento automatico (ad esempio classificazione supervisionata o apprendimento per rinforzo con feedback sulle prestazioni) potrebbero essere impiegate per addestrare il modello di routing sui risultati storici del refactoring.

Si propone inoltre un'**analisi estesa della qualità del software**: questa tesi si è concentrata esclusivamente su metriche di prestazioni e sostenibilità, tuttavia la qualità del codice comprende anche altre metriche, tra cui sicurezza, robustezza, manutenibilità, leggibilità e aderenza ai principi di progettazione. Lavori futuri dovrebbero estendere il framework di valutazione per considerare l'impatto del refactoring LLM su queste ulteriori dimensioni. Tale analisi fornirebbe una visione più equilibrata del refactoring mediante LLM, riconoscendo che l'ottimizzazione delle prestazioni è solo uno tra gli aspetti relativi alla qualità del software.

Infine si suggerisce un'**analisi profonda della correlazione tra pattern e performance**: sebbene l'analisi attuale abbia identificato quali pattern si verificano e la loro frequenza relativa, non ha stabilito collegamenti causali quantitativi tra pattern specifici e miglioramenti delle prestazioni osservati. Lavori futuri dovrebbero sviluppare tassonomie di pattern specifiche per linguaggio, che tengano conto delle ottimizzazioni proprie di ciascun linguaggio, e condurre esperimenti controllati in cui singoli pattern vengano applicati in isolamento per misurarne l'effetto causale sulle prestazioni. Sarebbe inoltre opportuno indagare le interazioni tra pattern applicati simultaneamente (interazioni sinergiche, ossia super-additive, o antagoniste, ovvero sub-additive) e sviluppare modelli di attribuzione in grado di scomporre un miglioramento osservato nei contributi dei singoli pattern. Infine, una volta identificati i pattern migliori (ossia quelli che producono costantemente notevoli miglioramenti con un rischio funzionale minimo), si potrebbero progettare prompt mirati che guidino esplicitamente gli LLM verso precise strategie di ottimizzazione. Nel complesso, tale linea di ricerca trasformerebbe l'attuale identificazione descrittiva dei pattern in un framework predittivo, permettendo di comprendere non solo che gli LLM possono ristrutturare il codice, ma anche quali strategie di ristrutturazione sono più efficaci per specifici obiettivi di ottimizzazione.

Bibliografia

- [1] European Commission. The European Green Deal. COM(2019) 640 final, 2019. Comunicazione della Commissione al Parlamento Europeo, al Consiglio Europeo, al Consiglio, al Comitato Economico e Sociale Europeo e al Comitato delle Regioni. URL: https://commission.europa.eu/strategy-and-policy/priorities-2019-2024/european-green-deal_en.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020. URL: <https://arxiv.org/abs/2005.14165>.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017. URL: <https://arxiv.org/abs/1706.03762>.
- [4] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. A prompt pattern catalog to enhance prompt engineering with ChatGPT. In *Proceedings of the 30th Conference on Pattern Languages of Programs (PLoP)*. ACM, 2023. URL: <https://dl.acm.org/doi/10.5555/3721041.3721046>.
- [5] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. URL: <https://arxiv.org/abs/2206.07682>.
- [6] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arber, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021. URL: <https://arxiv.org/abs/2108.07258>.
- [7] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. CodeGen: An open large language model for code with multi-turn program synthesis. In *International Conference on Learning Representations (ICLR)*, 2023. URL: <https://arxiv.org/abs/2203.13474>.
- [8] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley Professional, 2nd edition, 2018.
- [9] San Murugesan. Harnessing green IT: Principles and practices. *IT Professional*, 10(1):24–33, 2008. URL: <https://ieeexplore.ieee.org/document/4446673>, doi:10.1109/MITP.2008.10.
- [10] John L Hennessy and David A Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition, 2017.
- [11] Jonathan Koomey. Growth in data center electricity use 2005 to 2010. Technical report, Analytics Press, 2011. URL: <https://koomey.com/post/8323374335>.
- [12] International Energy Agency. Electricity 2024 – analysis and forecast to 2026. Technical report, IEA, 2024. License: CC BY 4.0. URL: <https://www.iea.org/reports/electricity-2024>.
- [13] Changan Niu, Ting Zhang, Chuanyi Li, Bin Luo, and Vincent Ng. On evaluating the efficiency of source code generated by llms. In *Proceedings of the 2024 IEEE/ACM 1st International Conference on AI Foundation Models and Software Engineering (FORGE)*, pages 103–107. ACM, 2024. Co-located with ICSE 2024, Lisbon, Portugal. URL: <https://dl.acm.org/doi/10.1145/3650105.3652295>, doi:10.1145/3650105.3652295.

- [14] Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob Gardner, Yiming Yang, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. Learning performance-improving code edits. In *International Conference on Learning Representations (ICLR)*, 2024. Spotlight. URL: <https://openreview.net/forum?id=ix7rLVHXYy>.
- [15] Tina Vartziotis, Ippolyti Dellatolas, George Dasoulas, Maximilian Schmidt, Florian Schneider, Tim Hoffmann, Sotirios Kotsopoulos, and Michael Keckeisen. Learn to code sustainably: An empirical study on green code generation. In *Proceedings of the 1st International Workshop on Large Language Models for Code (LLM4Code)*, pages 30–37. ACM, 2024. Co-located with ICSE 2024, Lisbon, Portugal. URL: <https://dl.acm.org/doi/10.1145/3643795.3648394>, doi:10.1145/3643795.3648394.
- [16] Vlad-Andrei Cursaru, Laura Duits, Joel Milligan, Damla Ural, Berta Rodriguez Sanchez, Vincenzo Stoico, and Ivano Malavolta. A controlled experiment on the energy efficiency of the source code generated by code llama. In *Proceedings of the 17th International Conference on the Quality of Information and Communications Technology (QUATIC)*, pages 161–176. Springer, 2024. URL: https://link.springer.com/chapter/10.1007/978-3-031-70245-7_12, doi:10.1007/978-3-031-70245-7_12.
- [17] Kevin Lano, Zishan Rahman, and Lyan Alwakeel. Design patterns for software sustainability. In *Proceedings of the 31st Conference on Pattern Languages of Programs (PLoP)*, Skamania Lodge, Columbia River Gorge, Washington, USA, 2024. ACM. ISBN 978-1-941652-20-6. doi:10.64346/PLoP2024p10.
- [18] Olivia Poy, Ma Ángeles Moraga, Félix García, and Coral Calero. Impact on energy consumption of design patterns, code smells and refactoring techniques: A systematic mapping study. *Journal of Systems and Software*, 222:112303, 2025. URL: <https://www.sciencedirect.com/science/article/abs/pii/S0164121224003479>, doi:10.1016/j.jss.2024.112303.
- [19] Roy Schwartz, Jesse Dodge, Noah A Smith, and Oren Etzioni. Green AI. *Communications of the ACM*, 63(12):54–63, 2020. URL: <https://dl.acm.org/doi/10.1145/3381831>, doi:10.1145/3381831.
- [20] OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023. URL: <https://arxiv.org/abs/2303.08774>.
- [21] Anthropic. Introducing Claude 4. Technical report, Anthropic, 2025. Model ID: claude-sonnet-4-20250514. URL: <https://www.anthropic.com/news/claude-4>.
- [22] Google DeepMind. Gemini 2.0: A new AI model for the agentic era. Technical report, Google DeepMind, 2024. Official announcement, December 2024. URL: <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024/>.
- [23] Green Software Foundation. Software carbon intensity (SCI) specification. Technical report, Green Software Foundation, 2023. Now ISO/IEC 21031:2024. URL: <https://sci.greensoftware.foundation>.
- [24] Snehal Bhatia et al. Calculating software’s energy use and carbon emissions: A survey. In *Proceedings of the 8th International Workshop on Green and Sustainable Software (GREENS)*, 2025. Co-located with ICSE 2025. URL: <https://ieeexplore.ieee.org/document/11039301/>.
- [25] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966. URL: <https://nymity.ch/sybilhunting/pdf/Levenshtein1966a.pdf>.
- [26] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK, 2008. Disponibile gratuitamente dagli autori. URL: <https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf>.

- [27] Alfred V Aho, Monica S Lam, Ravi Sethi, and Jeffrey D Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2006.
- [28] Gustavo Pinto and Fernando Castor. Energy efficiency: A new concern for application software developers. *Communications of the ACM*, 60(12):68–75, 2017. URL: <https://dl.acm.org/doi/10.1145/3154384>, doi:10.1145/3154384.
- [29] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. Energy efficiency across programming languages: How do energy, time, and memory relate? In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*, pages 256–267. ACM, 2017. URL: <https://dl.acm.org/doi/10.1145/3136014.3136031>, doi:10.1145/3136014.3136031.
- [30] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.
- [31] Ulrich Drepper. What every programmer should know about memory. Technical report, Red Hat, Inc., 2007. URL: <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>.
- [32] Brian Goetz, Tim Peierls, Joshua Bloch, Joseph Bowbeer, David Holmes, and Doug Lea. *Java Concurrency in Practice*. Addison-Wesley Professional, 2006.
- [33] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in software engineering*. Springer Science & Business Media, Berlin, Heidelberg, 2012. doi:10.1007/978-3-642-29044-2.

Appendice A

Testi Completi dei Prompt (P1-P4)

Questa appendice presenta i testi completi delle quattro versioni del prompt utilizzate in questo studio. In ogni prompt, il placeholder `{language}` viene sostituito con il nome del linguaggio di programmazione specifico (es. “Python”, “Go”, “C++”).

Prompt Versione 1 (Baseline)

```
1 You are an experienced software engineer. Below is a code snippet
2 written in {language} delimited by triple backticks (``), without
3 the tests. Your task is to return an optimized version of the
4 snippet, which keeps the original behavior and interface
5 (functions' signatures), while reducing the runtime energy
6 consumption:
7
8 Optimize the code according to these criteria:
9 1. Minimize execution times
10 2. Minimize resource usage (e.g., memory, CPU)
11 3. Avoid modifying functions' signatures
12 4. Follow sustainable software engineering best practices,
13    guidelines and pattern
14
15 Respond only with updated code, without further explanation.
16
17 ```${code}```
```

Prompt Versione 2 (Semplificato)

```
1 You are an experienced software engineer. Below is a code snippet
2 written in {language}, delimited by triple backticks (``), it does
3 not include any tests. Your task is to return an optimized version
4 of the code, that preserves the functionality and the function
5 signatures, while reducing the runtime energy consumption.
6
7 Follow these optimization guidelines:
8 - Improve execution efficiency, minimizing execution time
9 - Reduce resource consumption (e.g., memory, CPU)
10 - Apply sustainable software engineering principles where relevant
11
12 Respond only with updated code, without further explanation.
13
14 ``
15 {code}
16 ``
```

Prompt Versione 3 (Ruolo Senior)

```
1 You are a senior-level software engineer with expertise in
2 sustainable computing and performance optimization. A code snippet
3 written in {language} is provided below, marked by triple
4 backticks ('``'). Your task is to optimize it while preserving its
5 original functionality and interface.
6
7 Follow these instructions:
8 - Refactor the code to improve runtime performance and reduce
9   overall energy consumption
10 - Limit memory and CPU usage where possible
11 - Preserve all existing function names and signatures
12   (do not alter the external interface)
13 - Apply clean and maintainable coding practices in line with
14   sustainable software engineering standards
15
16 Respond only with the updated code, without explanations or
17 comments.
18
19 ````
20 {code}
21 ````
```

Prompt Versione 4 (Strutturato)

```
1 You are a senior software engineer with expertise in performance
2 optimization and sustainable computing. Below is a code snippet
3 written in {language}, delimited by triple backticks ('``'), and it
4 does not include any tests.
5
6 Your task is to return an optimized version of the code that:
7 - "Preserves the original functionality"
8 - "Does not alter any function names or signatures"
9 - "Maintains the external interface exactly as provided"
10
11 Optimize the code following these clearly defined goals:
12 1. "Improve execution performance (minimize execution time)"
13 2. "Reduce runtime resource usage (e.g., CPU and memory)"
14 3. "Minimize energy consumption"
15 4. "Apply clean, maintainable coding practices"
16 5. "Follow sustainable software engineering principles and
17   standards"
18
19 Respond only with the updated and optimized code, without any
20 explanation or comments.
21
22 ````
23 {code}
24 ````
```

Appendice B

Esempi Notevoli di Refactoring

Questa appendice raccoglie esempi significativi di refactoring effettuato dagli LLM, selezionati per mostrare alcuni dei pattern di ottimizzazione più frequentemente applicati. È presente inoltre un esempio che mostra come gli LLM possano introdurre errori, anche critici, nei codici generati, degradando la correttezza funzionale.

Criteri di Selezione degli Esempi

Gli esempi di corretta applicazione dei pattern presentati sono stati selezionati secondo i seguenti criteri:

- **Riduzione significativa:** riduzione energetica $\leq -15\%$
- **Differenza dal codice originale:** similarity index $\leq 85\%$ per garantire modifiche sostanziali
- **Diversità linguistica:** rappresentazione di diversi linguaggi di programmazione

Esempi di Applicazione dei Pattern

Le seguenti figure mostrano esempi concreti di applicazione dei pattern analizzati in questo studio da parte degli LLM, comparando il codice originale (a sinistra) con il codice rifattorizzato (a destra). Le righe evidenziate in verde indicano le porzioni di codice in cui i pattern sono stati introdotti.

Esempio 1 – Go (OpenAI, Prompt P2):

Nel primo esempio viene mostrato un codice in Go in cui OpenAI ha ridotto l'uso di CPU agendo sulla rimozione della ricorsione (riga 8 del codice originale), riducendo allocazioni nello stack. Inoltre il codice generato pre-alloca la memoria necessaria mediante l'uso di **make** (righe 4 e 6), evitando allocazioni dinamiche mediante il pattern *Reduced Allocation*.

Infine viene mantenuta l'allocazione in loop, ma viene ottimizzata anch'essa mediante l'uso di **make** (riga 6), evitando sprechi di memoria e alleggerendo il lavoro del Garbage Collector.

I cambiamenti implementati nel codice rifattorizzato da parte di OpenAI portano sostanziali migliorie in termini di performance, arrivando quasi a dimezzare il tempo di esecuzione e riducendo drasticamente sia l'uso di CPU che di RAM.

ORIGINAL CODE	REFACTORED (OPENAI (PROMPT_V2))
<pre> 1 package pascal 2 3 func Triangle(n int) [][]int { 4 if n == 1 { 5 return [][]int{{1}} 6 } 7 8 previousRows := Triangle(n - 1) 9 lastRow := previousRows[len(previousRows)-1] 10 11 currentRow := []int{} 12 for i := 0; i < n; i++ { 13 if i == 0 i == n-1 { 14 currentRow = append(currentRow, 1) 15 } else { 16 sum := lastRow[i-1] + lastRow[i] 17 currentRow = append(currentRow, sum) 18 } 19 } 20 return append(previousRows, currentRow) 21 } 22 </pre>	<pre> 1 package pascal 2 3 func Triangle(n int) [][]int { 4 triangle := make([][]int, n) 5 for i := 0; i < n; i++ { 6 triangle[i] = make([]int, i+1) 7 triangle[i][0], triangle[i][i] = 1, 1 8 for j := 1; j < i; j++ { 9 triangle[i][j] = triangle[i-1][j-1] + trian... 10 } 11 } 12 return triangle 13 } </pre>
Reduction: CPU -68.6% RAM -63.4% Time -88.7%	

Figura B.1: Primo esempio di codice rifattorizzato: OpenAI, seconda versione del prompt. Dalla figura è possibile comprendere l'integrazione di molteplici pattern volti a migliorare le performance del codice.

Esempio 2 – C (Claude, Prompt P1):

Nel secondo esempio è ben chiaro l'uso del pattern *Early Return* (riga 8) nel codice generato da Claude, che introduce un comportamento “fail fast” nella funzione `aliquot_sum` e alla riga 32 ne previene la chiamata dalla funzione `classify_number` nel caso in cui il parametro `number` abbia valore ≤ 0 .

Viene inoltre introdotta un'ottimizzazione importante mediante il pattern *Algorithmic Optimization*: la complessità scende da $O(N)$ ad $O(\sqrt{N})$ andando ad iterare da 2 alla radice di n all'interno del ciclo `for` della funzione `aliquot_sum` (riga 19).

Infine il calcolo manuale della radice (`sqrt_num`) (righe 14-16) e l'inizializzazione di `result` ad 1 nella funzione `aliquot_sum` (riga 10) introducono ottimizzazione dei branch.

I cambiamenti apportati al codice originale permettono quindi di dimezzare il tempo di esecuzione e avere elevati miglioramenti anche nell'uso della CPU.

ORIGINAL CODE	REFACTORED (CLAUDE (PROMPT_V1))
<pre> 1 #include "perfect_numbers.h" 2 3 #include <stdio.h> 4 5 int aliquot_sum(int number); 6 7 int aliquot_sum(int number) { 8 int result = 0; 9 for (int count_nb = 1; count_nb < number; count_nb++) 10 if (number % count_nb == 0) result += count_nb; 11 } 12 return (result); 13 } 14 15 int classify_number(int number) { 16 int aliquot_sum_result = aliquot_sum(number); 17 if (number <= 0) return (ERROR); 18 if (aliquot_sum_result == number) { 19 return (PERFECT_NUMBER); 20 } else if (aliquot_sum_result > number) { 21 return (ABUNDANT_NUMBER); 22 } 23 return (DEFICIENT_NUMBER); 24 } 25 </pre>	<pre> 1 #include "perfect_numbers.h" 2 3 #include <stdio.h> 4 5 int aliquot_sum(int number); 6 7 int aliquot_sum(int number) { 8 if (number <= 1) return 0; 9 10 int result = 1; // 1 is always a divisor for number... 11 int sqrt_num = 1; 12 13 // Find approximate square root 14 while (sqrt_num * sqrt_num <= number) { 15 sqrt_num++; 16 } 17 sqrt_num--; 18 19 for (int i = 2; i <= sqrt_num; i++) { 20 if (number % i == 0) { 21 result += i; 22 int complement = number / i; 23 if (complement != i && complement != number...) 24 result += complement; 25 } 26 } 27 return result; 28 } 29 30 31 ... [12 more lines] </pre>
Reduction: CPU -86.8% RAM -0.0% Time -99.0%	

Figura B.2: Secondo esempio di codice rifattorizzato: Claude, prima versione del prompt. Dall'immagine è visibile l'integrazione dei pattern Early Return, Branch Optimization e soprattutto Algorithmic Optimization, i quali riducono sostanzialmente uso di CPU e tempo di esecuzione nel codice generato.

Esempio 3 – JavaScript (Claude, Prompt P4):

In questo terzo esempio in cui il codice è stato rifattorizzato da Claude tramite la quarta versione del prompt sono stati introdotti molteplici pattern. Un *Early Return* è stato inserito all'inizio della funzione (riga 2), mediante un controllo sulla lunghezza dell'array **start** passato come parametro.

L'uso dei costruttori **new Array** (righe 5 e 8) sfrutta il pattern *Reduced Allocation*, evitando allocazioni dinamiche mediante l'uso di **push**.

Infine il pattern *Loop Invariant Code Motion* viene applicato per evitare l'uso di **result.map** (riga 15 del codice originale), costruendo il risultato progressivamente mediante **result[c] = column.join('')** (riga 21).

ORIGINAL CODE	REFACTORED (CLAUDE (PROMPT_V4))
<pre> 1 export default function(start) {}; 2 const max_col = start.reduce((max, row) => Math.max... 3 var result = []; 4 for(var c = 0; c < max_col; c++) { 5 var prefix = true; 6 result.push([]); 7 for(var r = 0; r < start.length; r++) { 8 result[c][r] = start[r][c]; 9 if(prefix && result[c][r] == undefined) 10 result[c][r] = ' '; 11 else 12 prefix = false; 13 } 14 } 15 return result.map(row => row.join('')); 16 } 17 </pre>	<pre> 1 export default function(start) { 2 if (!start.length) return []; 3 4 const max_col = start.reduce((max, row) => Math.max... 5 const result = new Array(max_col); 6 7 for (let c = 0; c < max_col; c++) { 8 const column = new Array(start.length); 9 let prefix = true; 10 11 for (let r = 0; r < start.length; r++) { 12 const cell = start[r][c]; 13 if (prefix && cell === undefined) { 14 column[r] = ' '; 15 } else { 16 column[r] = cell; 17 prefix = false; 18 } 19 } 20 21 result[c] = column.join(''); 22 } 23 24 return result; 25 } </pre>
Reduction: CPU -74.0% RAM -0.6% Time -58.9%	

Figura B.3: Terzo esempio di codice rifattorizzato: Claude, quarta versione del prompt. Dalla figura è osservabile l'introduzione dei pattern Early Return, Reduced Allocation e Loop Invariant Code Motion. Anche in questo esempio vi sono importanti benefici in termini di tempo di esecuzione e uso di CPU nel codice rifattorizzato mediante LLM.

Esempi di Errori Introdotti dagli LLM

La Figura B.4 è un chiaro esempio della possibilità di introduzione di errori da parte degli LLM nel processo di refactoring. Come visto dalle precedenti analisi, gli LLM faticano a mantenere correttezza funzionale dei codici originari e talvolta introducono problematiche che possono causare anche la completa impossibilità d'esecuzione del codice in questione.

A riguardo viene mostrato come Claude (utilizzando la versione 1 del prompt) introduca un errore già a tempo di compilazione, inserendo nel codice generato l'uso di `std::vector` alla riga 18, ma non importando la necessaria libreria (`#include <vector>`). Ciò causa un problema critico a livello di compilazione che rende il codice non eseguibile, rappresentando quindi in questo caso una regressione funzionale grave del 100%: nessun regression test passato dal codice generato a causa della mancata possibilità di esecuzione.

Seppur l'errore sia banale, facilmente individuabile e risolvibile, questo caso è un chiaro esempio della necessità di validazione umana a posteriori del refactoring automatizzato mediante Large Language Model.

Broken Refactoring: Largest-Series-Product Toemm (CPP) — LLM: CLAUDE

ORIGINAL CODE	LLM REFACTORED (CLAUDE)
<pre> 1 #include "largest_series_product.h" 2 3 #include <stdexcept> 4 #include <algorithm> 5 6 namespace largest_series_product { 7 8 int largest_product(const std::string& number, int span... 9 10 auto length = number.size(); 11 12 int biggest_series{}; 13 14 if (span > static_cast<int>(length) length == 0 ... 15 16 if (std::any_of(number.begin(), number.end(), 17 [](aut... 18 ret... 19))) 20 throw std::domain_error("invalid num"); 21 22 for (std::size_t i{}; i < length - span + 1; i++) { 23 24 int product = 1; 25 26 for (int j{}; j < span; j++) { 27 int num = number.at(i+j) - '0'; 28 product *= num; 29 } 30 31 ... [12 more lines] </pre>	<pre> 1 #include "largest_series_product.h" 2 3 #include <stdexcept> 4 #include <algorithm> 5 6 namespace largest_series_product { 7 8 int largest_product(const std::string& number, int span... 9 const auto length = static_cast<int>(number.size())... 10 11 if (span > length length == 0 span < 0) { 12 throw std::domain_error("invalid"); 13 } 14 15 if (span == 0) return 1; 16 17 // Single pass validation and conversion 18 std::vector<int> digits; 19 digits.reserve(length); 20 21 for (char ch : number) { 22 if (ch < '0' ch > '9') { 23 throw std::domain_error("invalid num"); 24 } 25 digits.push_back(ch - '0'); 26 } 27 28 int biggest_series = 0; 29 30 // Calculate first window 31 ... [28 more lines] </pre>

ERROR: Compilation error: uses std::vector without #include <vector>

Figura B.4: Esempio di errore introdotto dal processo di refactoring via LLM. In questo caso Claude, tramite la versione 1 del prompt, introduce un problema a tempo di compilazione (mancato import di `<vector>`) che rende il codice impossibile da eseguire.

Appendice C

Grafici Supplementari: Comparazione P1 vs P4 per Metrica

Questa appendice presenta i grafici supplementari menzionati nella Sezione 4.2, mostrando le distribuzioni dei miglioramenti percentuali per uso di CPU, uso di RAM e tempo di esecuzione. Nel testo principale sono stati presentati solo i grafici relativi al consumo energetico; qui vengono fornite le comparazioni complete per ogni metrica, disaggregate per modello e linguaggio di programmazione.

Uso di CPU: Comparazione P1 vs P4 per Modello

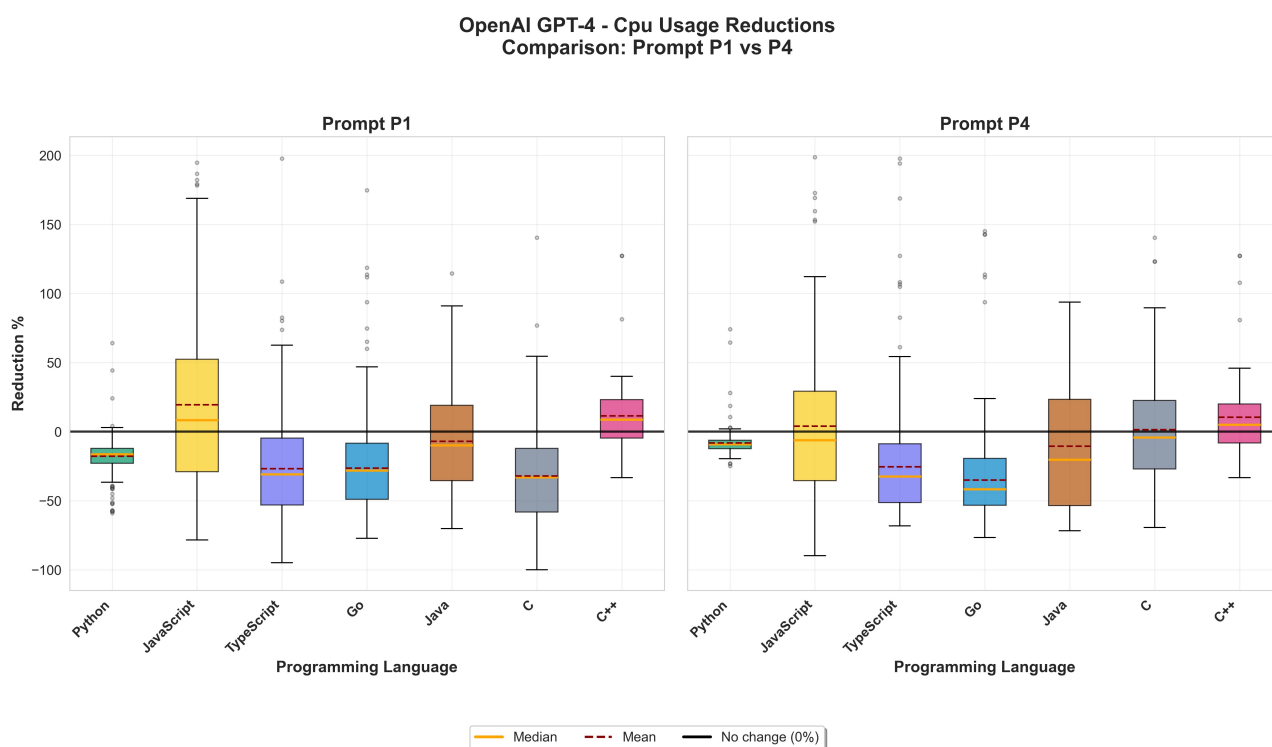


Figura C.1: Distribuzione della riduzione nell'uso di CPU (%): OpenAI P1 vs. P4 (per linguaggio).

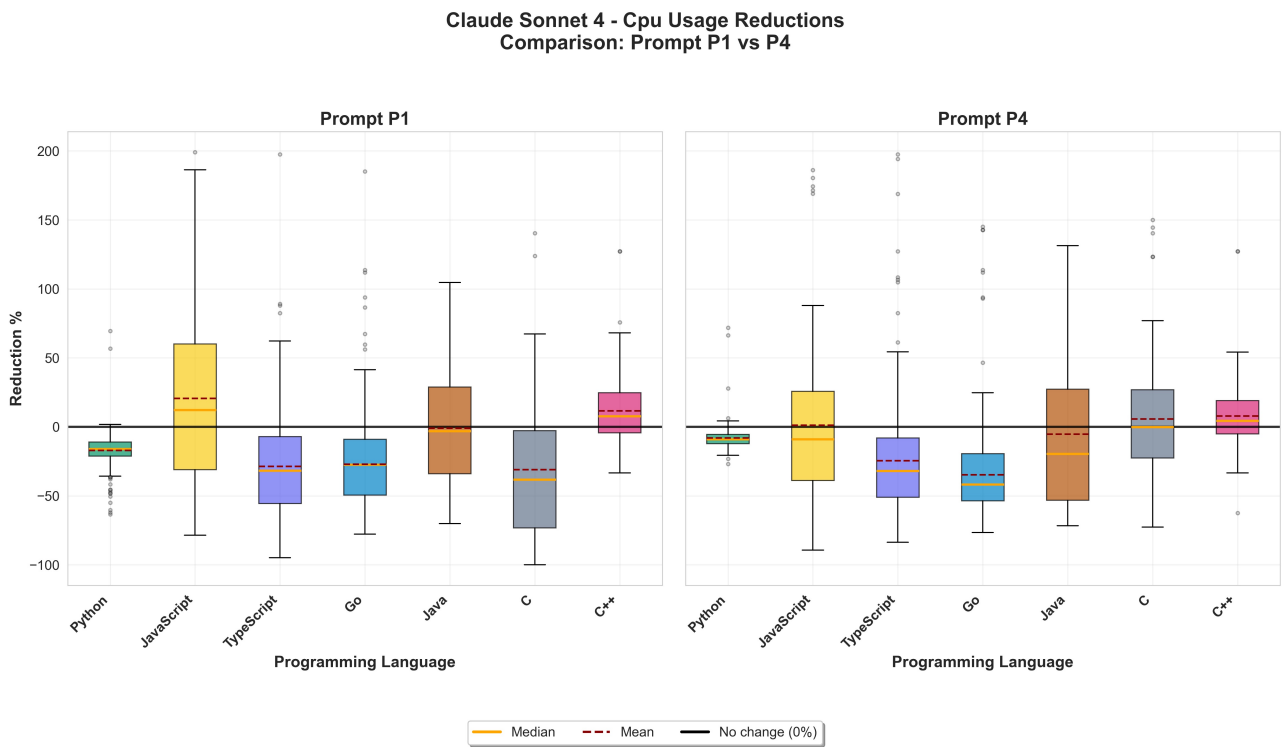


Figura C.2: Distribuzione della riduzione nell'uso di CPU (%): Claude P1 vs. P4 (per linguaggio).

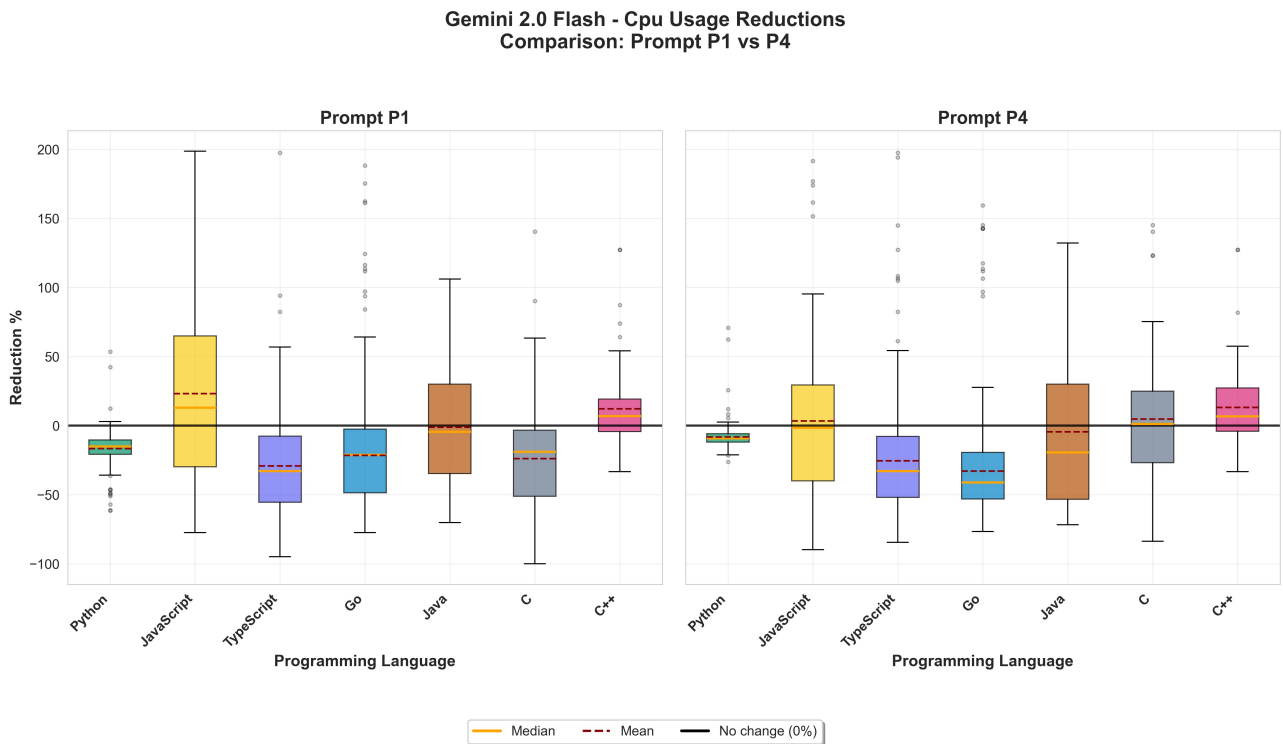


Figura C.3: Distribuzione della riduzione nell'uso di CPU (%): Gemini P1 vs. P4 (per linguaggio).

Uso di RAM: Comparazione P1 vs P4 per Modello

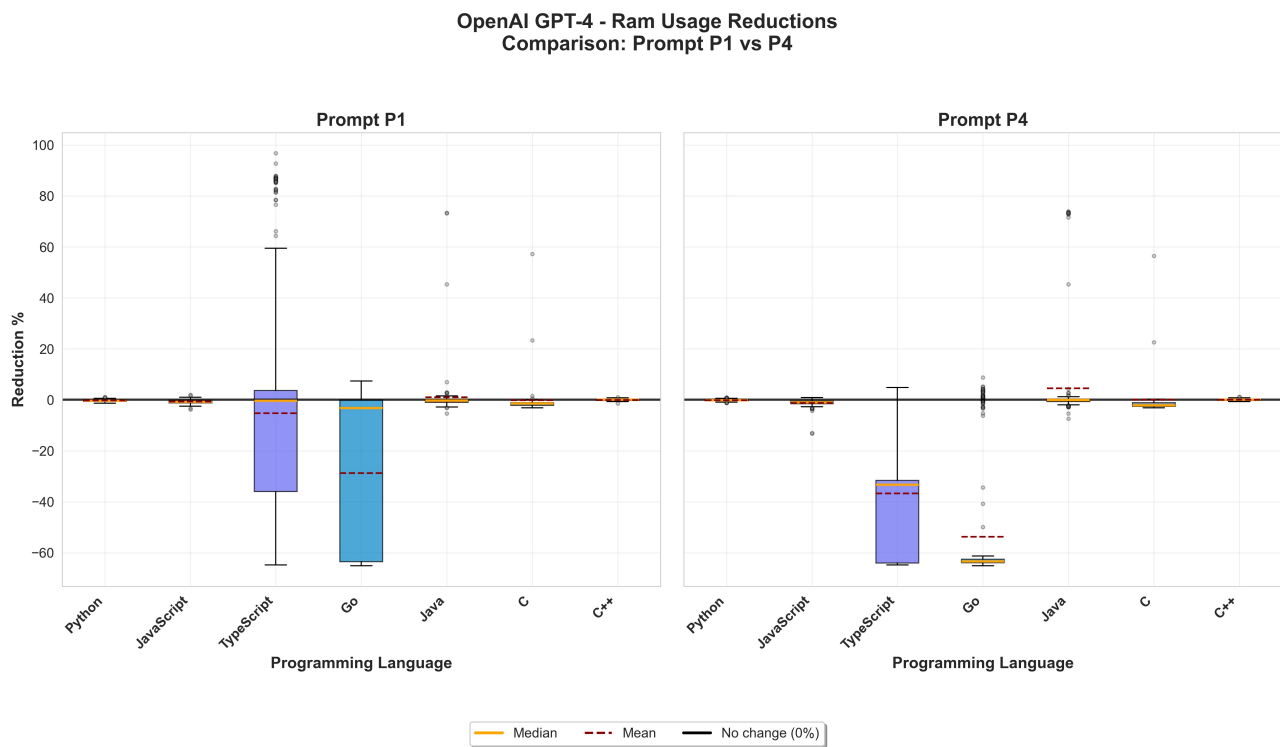


Figura C.4: Distribuzione della riduzione nell'uso di RAM (%): OpenAI P1 vs. P4 (per linguaggio).

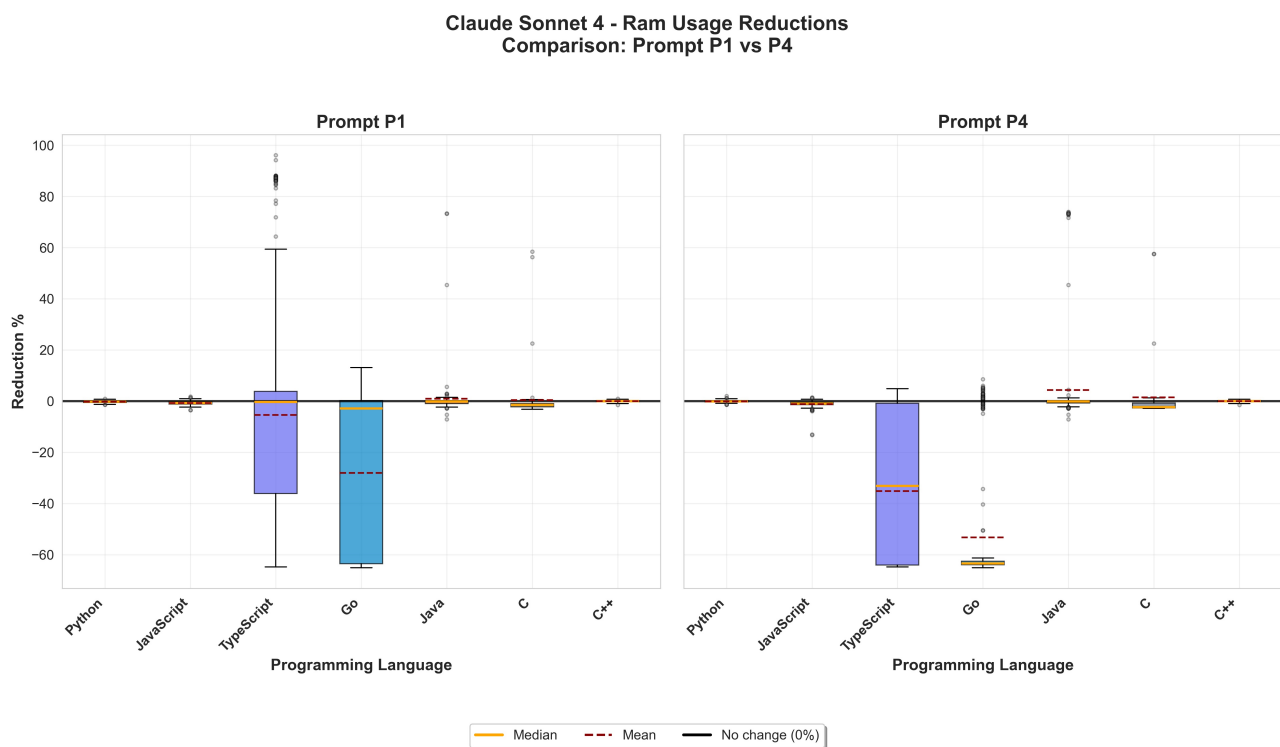


Figura C.5: Distribuzione della riduzione nell'uso di RAM (%): Claude P1 vs. P4 (per linguaggio).

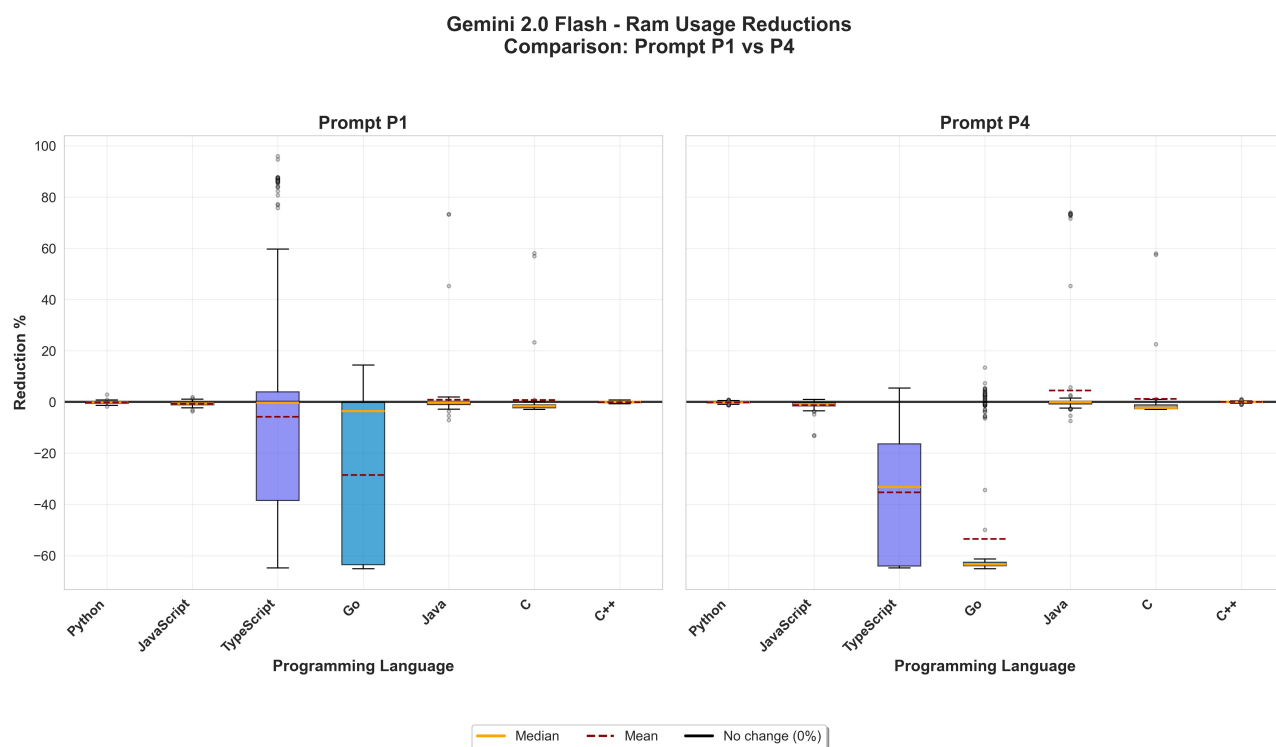


Figura C.6: Distribuzione della riduzione nell'uso di RAM (%): Gemini P1 vs. P4 (per linguaggio).

Tempo di Esecuzione: Comparazione P1 vs P4 per Modello

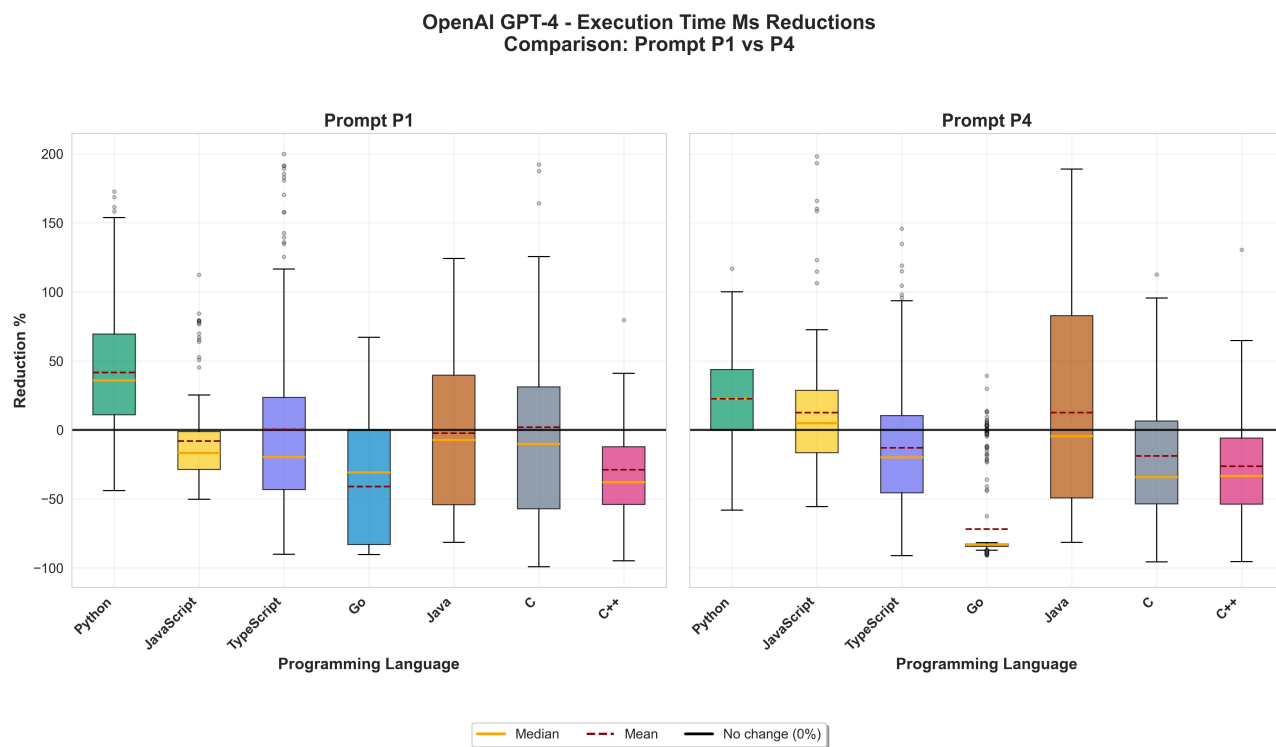


Figura C.7: Distribuzione della riduzione nel tempo di esecuzione (%): OpenAI P1 vs. P4 (per linguaggio).

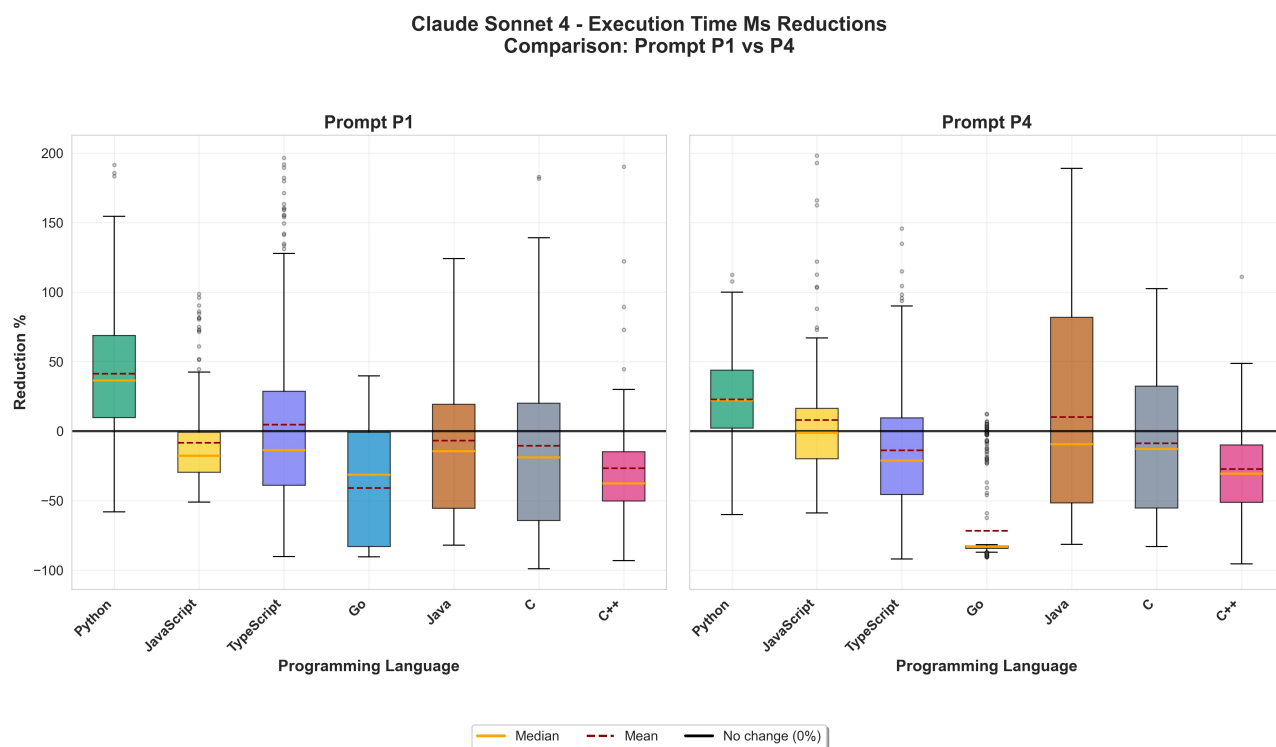


Figura C.8: Distribuzione della riduzione nel tempo di esecuzione (%): Claude P1 vs. P4 (per linguaggio).

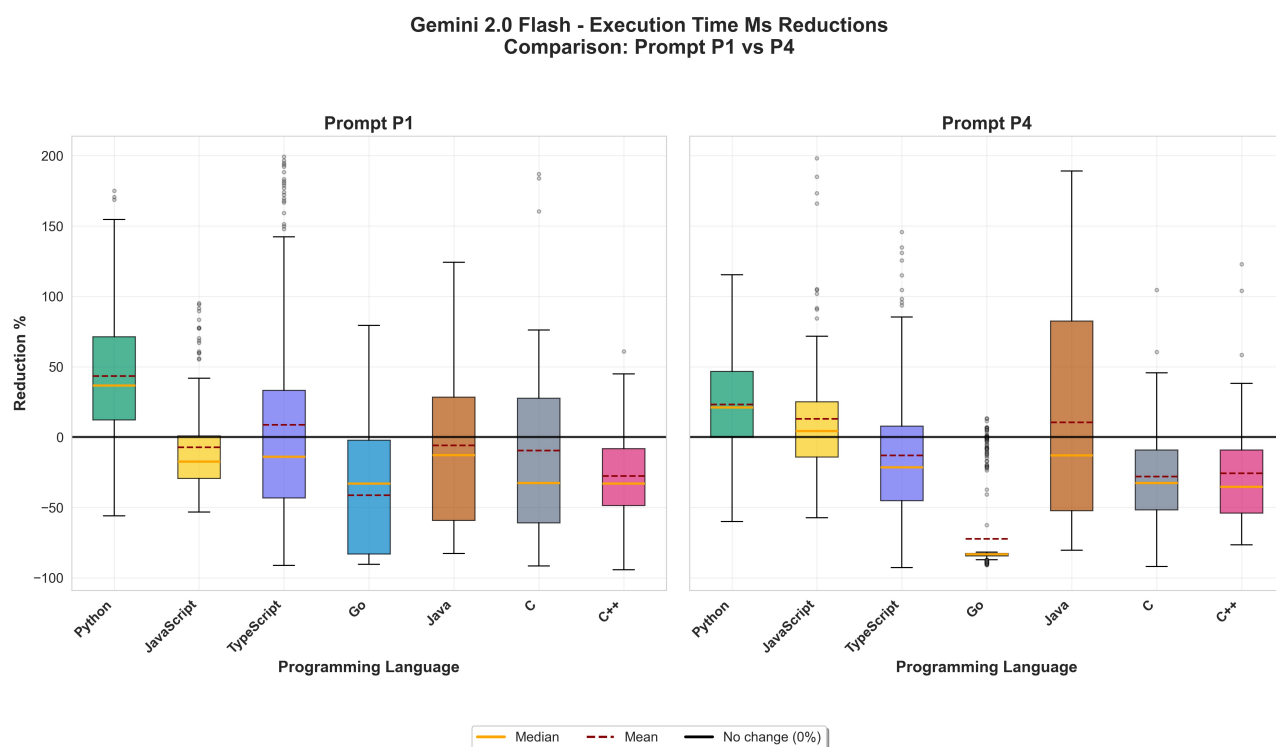


Figura C.9: Distribuzione della riduzione nel tempo di esecuzione (%): Gemini P1 vs. P4 (per linguaggio).